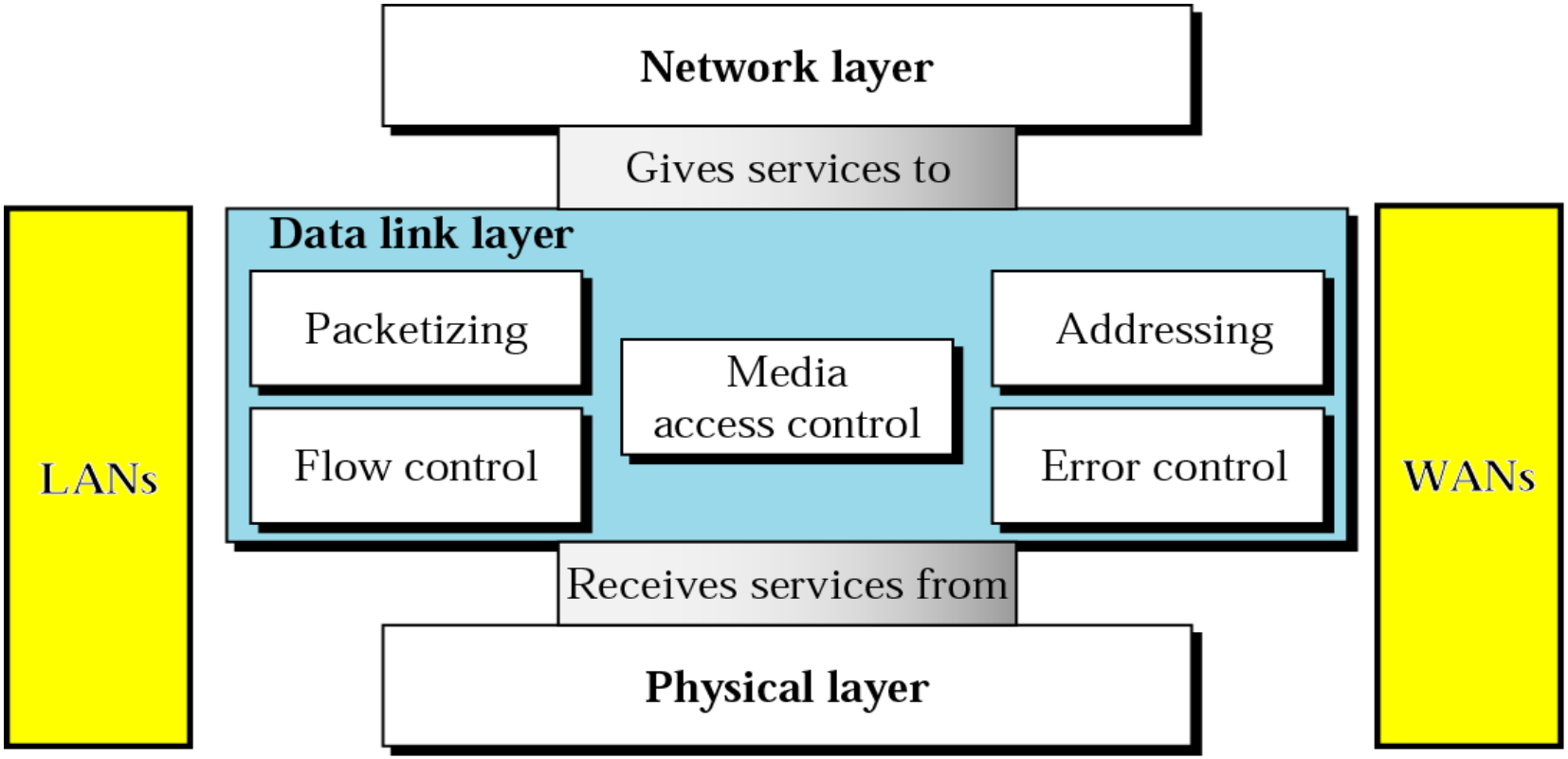
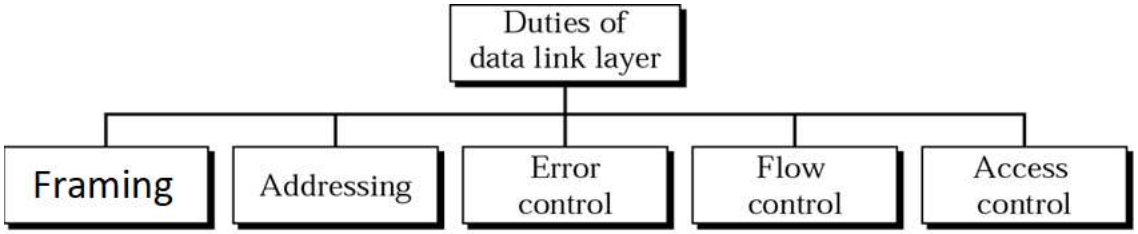
DATA LINK LAYER

Compiled by- Mrs. Savita Raut

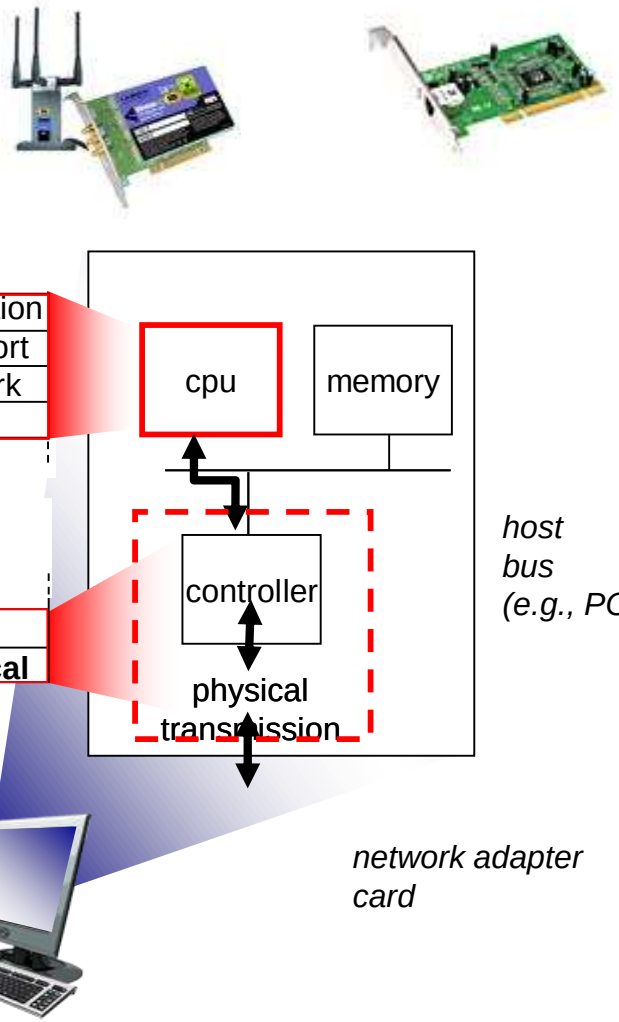
EXTC Dept, KJSCE


Network layer
Gives services to
Data link layer
Packetizing
Media access control
Addressing
Flow control
Error control
LANs
WANs
Receives services from
Physical layer

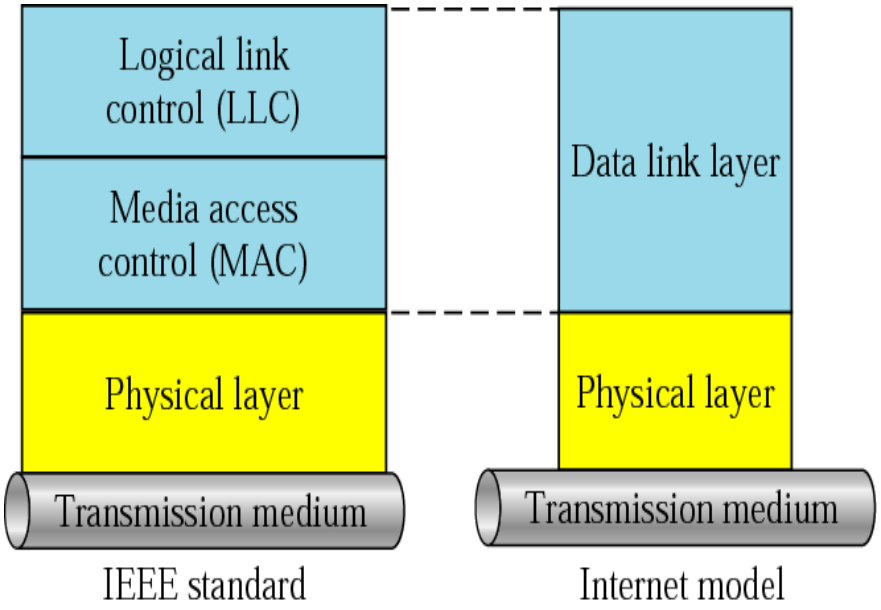

Duties of data link layer
Framing
Addressing
Error control
Flow control
Access control

Where is the link layer implemented

- **In each and every host**
- **link layer implemented in "adaptor" (*network interface card* NIC) or on a chip**
  - **Ethernet card, 802.11 card; Ethernet chipset**
  - **implements link, physical layer**
- **attaches into host's system buses**
- **combination of hardware, software, firmware**



The DLL is divided into two sublayers:



- LLC deals with all issues common to both point-to-point and broadcast links
- MAC sublayer deals with only broadcast links

DATA LINK LAYER :Functions

- It is responsible for transmitting frames from one node to the next.
- ***Service to the N/w layer***: To transfer data from N/w layer on source m/c to the N/w layer on destination m/c.
- ***Framing*** : The data received from network layer is divided into manageable data units called frames.
- ***Physical addressing*** [MAC (Media access control) or H/w address]: When frames are to be sent to different LAN's , the data link layer adds a header to the frame to define sender or receiver.
- ***Flow control*** : When data transmitted rate and data received is not same; some data may be lost.
- The Data link layer imposes a flow control mechanism to prevent loss of data.
- ***Error control*** : It helps to detect the error and retransmit the damaged frame or the lost frame.

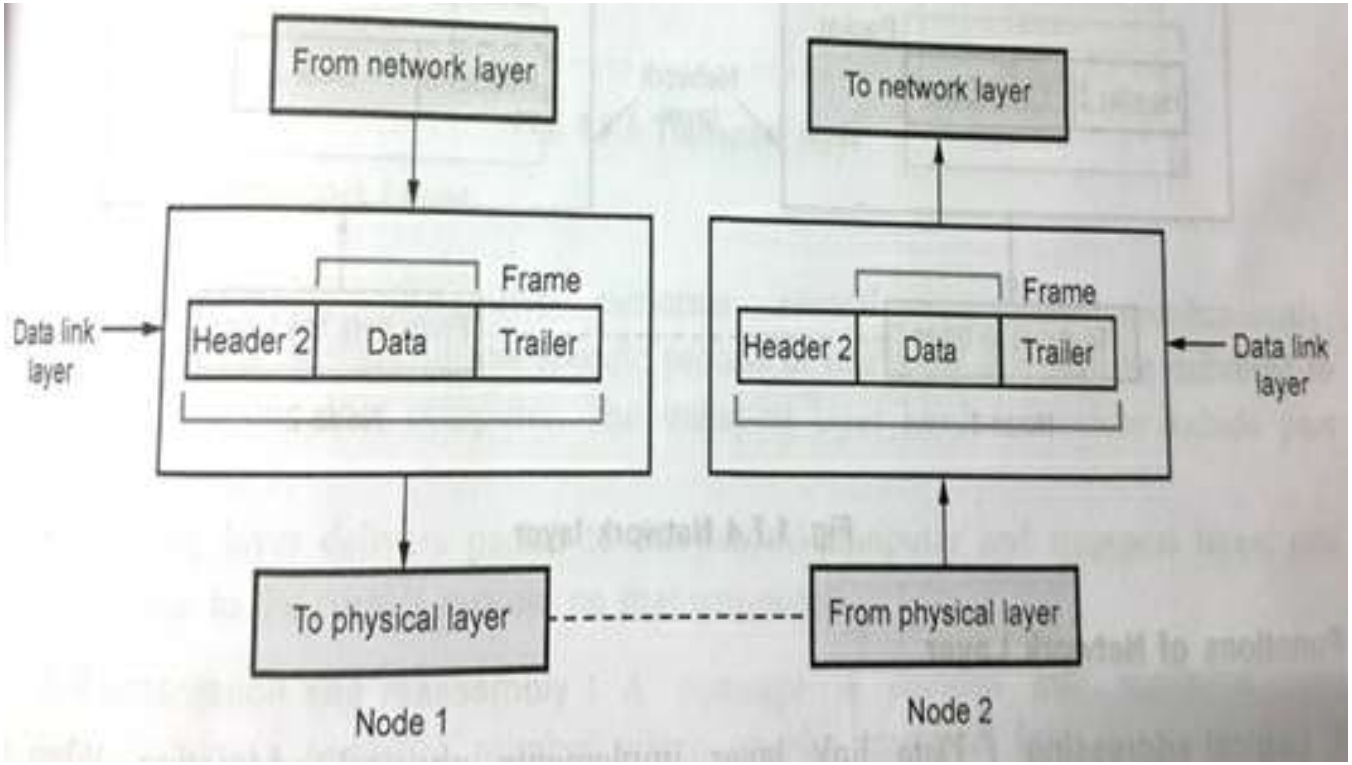
DATA LINK LAYER :Functions …cntd

- ***Medium Access Control (Link Management)*** : The nodes are connected by transmission medium (cable or Air).The DLL controls how medium is used .The link can be point-to point (dedicated)or Broadcast (shared)link.

Protocols and procedures are required for Link management.

FRAMING :

- Bits to be transmitted is broken into Frames at DLL.
- To guarantee the bit stream is error free , the Checksum of each frame is computed.
- When the frame is received, the DLL re-computes the checksum, and compares with the one present in the frame. If the checksum is different, then the DLL comes to know that an error has occurred. It discards the frame, and sends back a request for retransmission.
- Breaking bit stream in frames is called as “Framing”.


From network layer
To network layer
Frame
Header 2
Data
Trailer
Data link layer
Frame
Header 2
Data
Trailer
Data link layer
To physical layer
From physical layer
Node 1
Node 2

FRAMING METHODS :

1.***Character Count*** :

A field in header is used to specify the number of characters in the Frame.

This number helps the receiver to know the no. of characters in the frame, following this count.

Five frames are of 4, 6, 4, 5, and 3 characters respectively.

If wrong character count number is received, then the receiver will get out of synchronization and will be unable to locate the start of next frame.

Disadvantage: an error can change the character count. This method is rarely used in practice.

2. Starting and Ending character with Character Stuffing :

The problem of character count method is solved here.

It uses a starting character before the start of each frame and an ending character at the end of each frame.

Each frame is preceded by the transmission of ASCII character sequence DLE STX.

DLE → Data Link Escape.

STX → Start of text.

After the frame, the ASCII character sequence DLE ETX is transmitted.

DLE → Data Link Escape.

ETX → End of text.

So if the receiver looses the synchronization, it just has to search for the DLE STX or DLE ETX, characters to return back on track..

Data to be transferred is KJSCE

Frame will be

STX DLE KJSCE DLE ETX

after adding start and end Character.

Problem :

If the syntax is a part of data. It would be interpreted as start or end.

This problem is solved by using a technique called "**Character Stuffing**".

The DLL at the sending side, inserts an ASCII DLE character, just before any accidental DLE character in the data.

The DLL at the receiving end will remove these additional DLE character before handing over the data to the N/w layer.

Data to be sent A DLE B

Frame will be

STX DLE A DLE DLE B DLE ETX

after adding start and end Character + DLE stuffed before DLE in the Data.

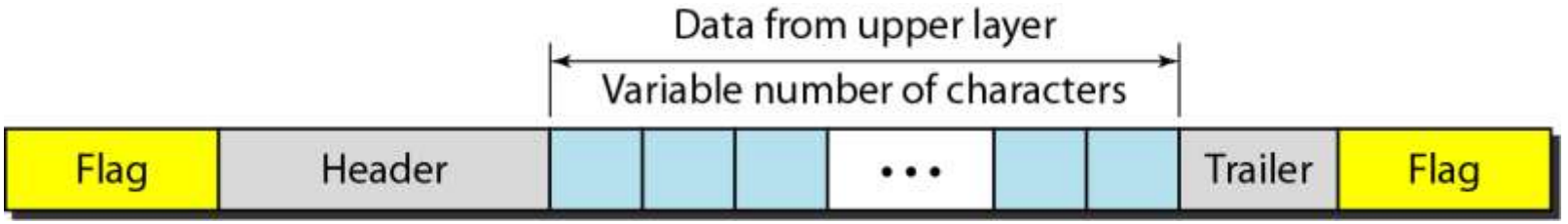
At the receiver side additional DLE will be removed, and A DLE B will be given to the N/w layer.

Thus framing DLE STX and DLE ETX can be easily distinguished from the one in data, because DLE's in the data is always Doubled.

Starting and Ending Flags with Bit Stuffing :

It allows frames to contain an arbitrary no of bits and codes different from ASCII code.

At the beginning and end of each frames a specific bit pattern " 01111110 ", called "Flag-byte" is transmitted. A concept of bit stuffing. (as there are 6 consecutive 1's).



Data to be transferred :

0 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 0 1 1 0.

The above data has 6 consecutive 1's. So the sender adds a '0' bit after continuous 5, 1's.

Data:0 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 0 1 1 0.

New data created

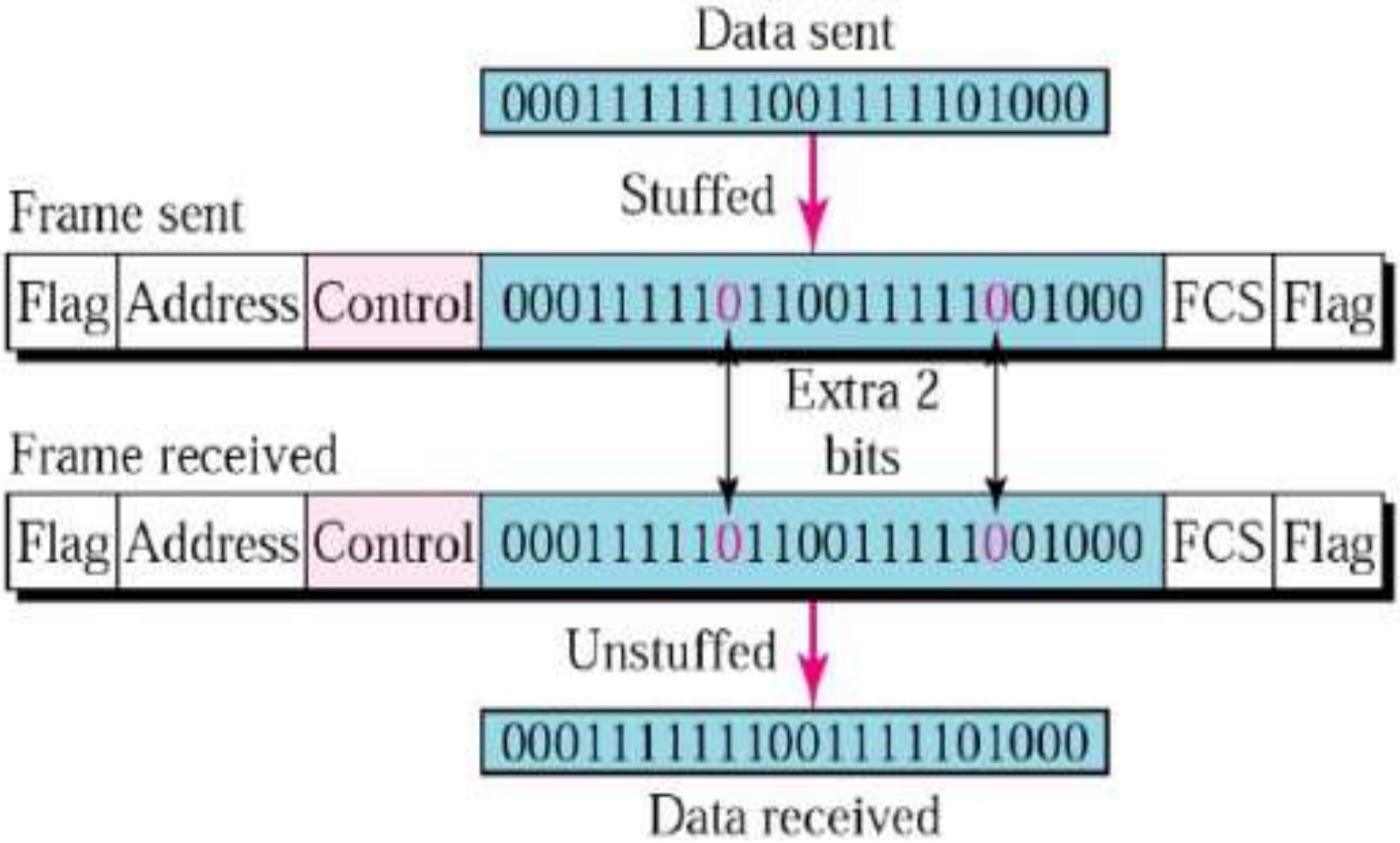
01111110 0 1 0 0 1 1 1 1 1 0 1 0 0 1 1 1 1 1 0 1 0 1 0 1 1 0 01111110.

Pink 0 → indicates the '0' bit stiffed, after consecutive 1's, in the data.

When the receiver receives this new data, with 5 consecutive 1's, it automatically deletes the '0' following the 5th bit.

This is called "**De-stuffing**". Due to bit stuffing, the problem of flag arising in the data is eliminated.

11.24 Bit stuffing and removal



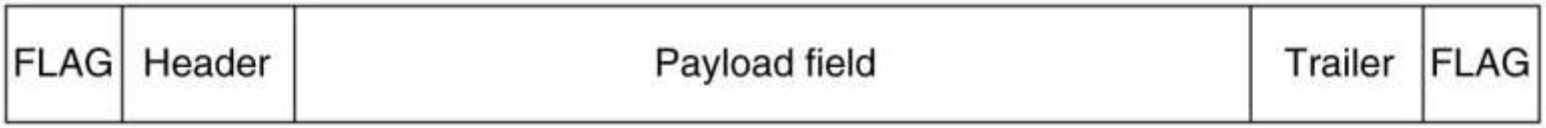
BYTE STUFFING : “ Escape character” :

A special byte is added to the data section of the frame, when there is a character with the same pattern as the flag.

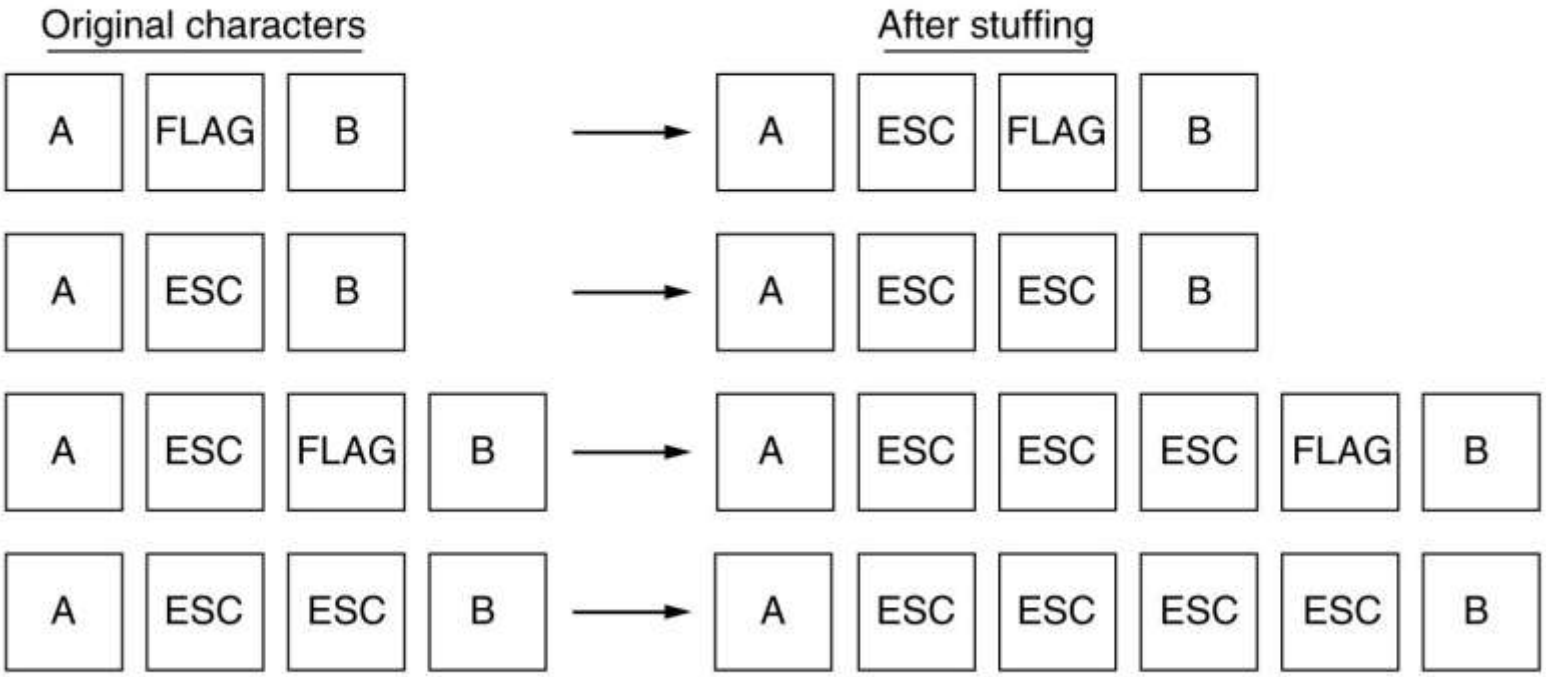
The data section is stuffed with an extra byte.

This byte is called Escape character. (ESC).

At receiver these ESC bytes are removed from the data section and the next character is treated as data.


FLAG
Header
Payload field
Trailer
FLAG

(a)


Original characters
After stuffing
A
FLAG
B
A
ESC
FLAG
B
A
ESC
B
A
ESC
ESC
B
A
ESC
FLAG
B
A
ESC
ESC
ESC
FLAG
B
A
ESC
ESC
B
A
ESC
ESC
ESC
ESC
B

(b)

Error Detection and Correction

Data can be corrupted during transmission. For reliable communication, errors must be detected and corrected.

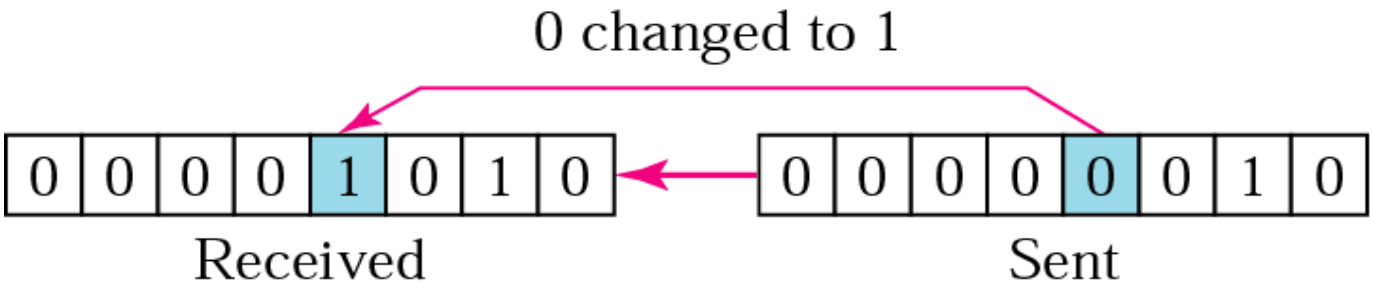
Types of Error

Single-Bit Error

Burst Error

In a single-bit error, only one bit in the data unit has changed.

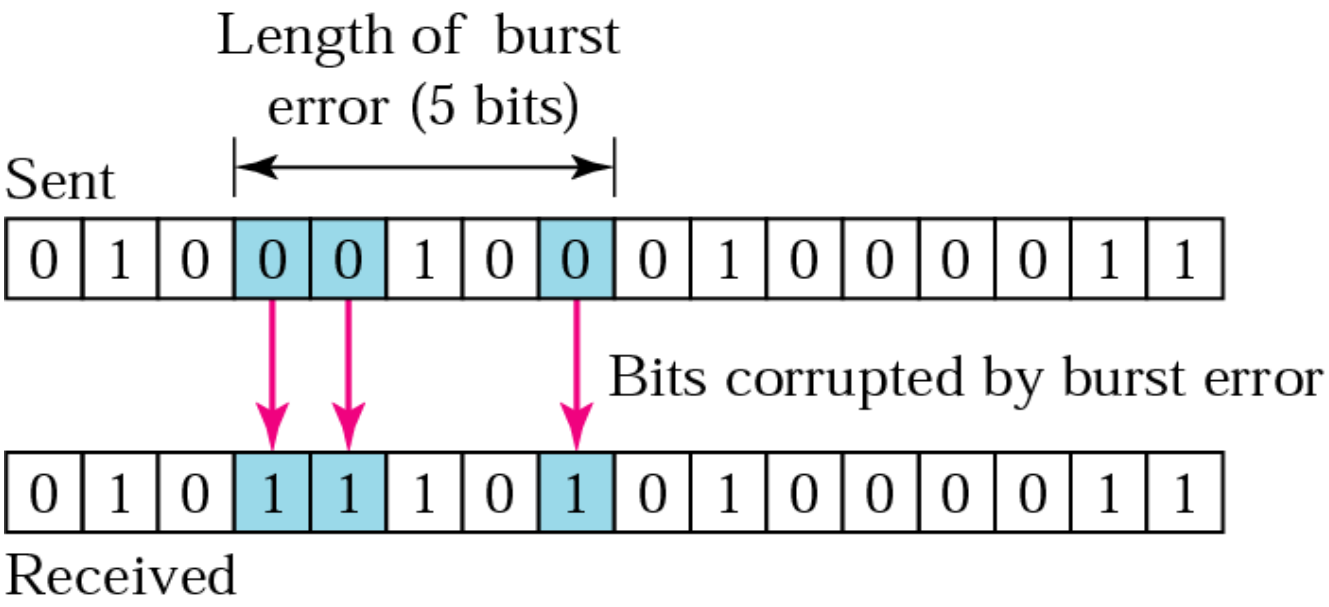
Single-bit error



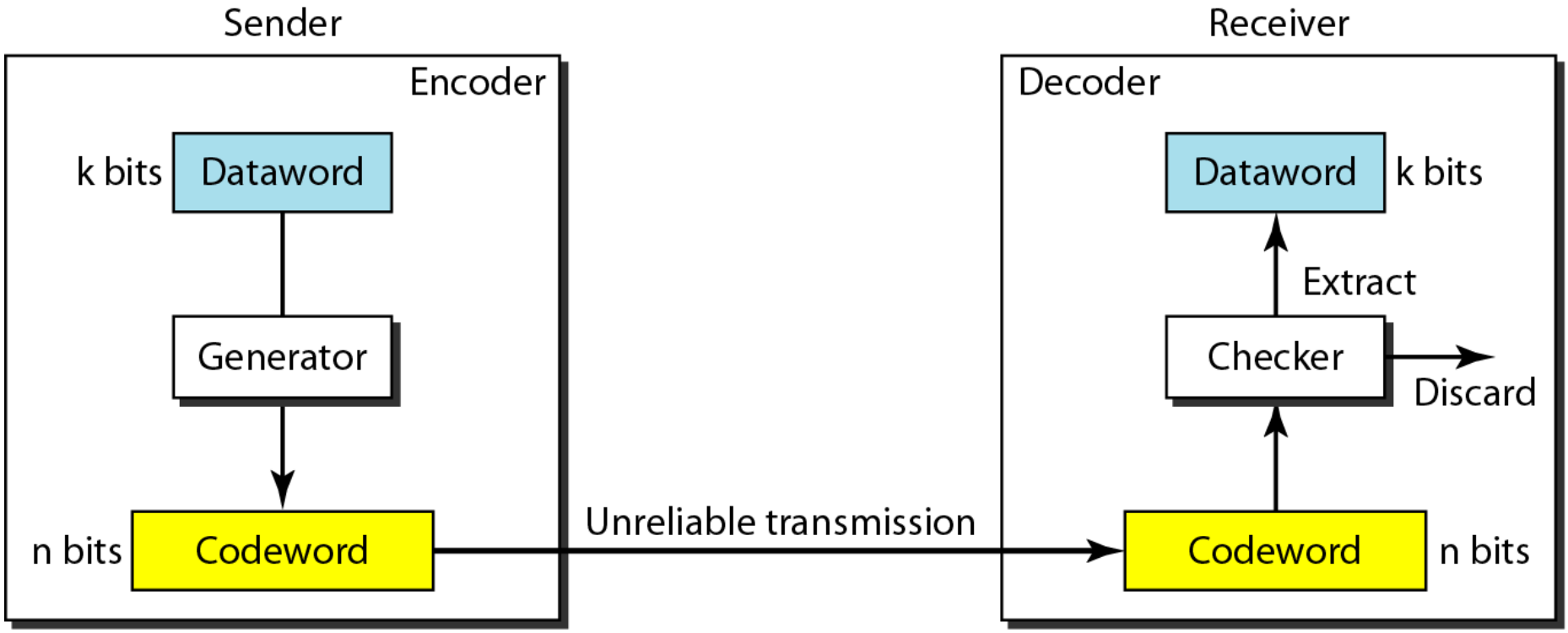
A burst error means that 2 or more bits in the data unit have changed.

The central concept in detecting or correcting errors is redundancy.

Burst error of length 5



The structure of encoder and decoder



To detect or correct errors, we need to send redundant bits

BLOCK CODING

In block coding, we divide our message into blocks, each of k bits, called datawords. We add r redundant bits to each block to make the length n = k + r. The resulting n-bit blocks are called codewords.

Topics discussed in this section:

Error Detection
Error Correction
Hamming Distance
Minimum Hamming Distance

A code for error detection

Dataword	*Codeword*
00	00000
01	01011
10	10101
11	11110

Codeword 01011 is received, no problem.
What if 01111 is received? Error detected.
What if 00000 is received? Error occurred, but not detected.

A code for error correction

Dataword	*Codeword*
00	00000
01	01011
10	10101
11	11110

Let's say we want to send 01. We then transmit 01011. What if an error occurs and we receive 01001. If we assume one bit was in error, we can correct.

Note

The Hamming distance between two words is the number of differences between corresponding bits.

Example

Let us find the Hamming distance between two pairs of words.

1. The Hamming distance d(000, 011) is 2 because

$000 \oplus 011$ is 011 (two 1s)

2. The Hamming distance d(10101, 11110) is 3 because

$10101 \oplus 11110$ is 01011 (three 1s)

Note

The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.

Example

Find the minimum Hamming distance of the coding scheme in Table

Solution

We first find all Hamming distances.

$d(000, 011) = 2$	$d(000, 101) = 2$	$d(000, 110) = 2$	$d(011, 101) = 2$
$d(011, 110) = 2$	$d(101, 110) = 2$		

The d_{min} in this case is 2.

Example

Find the minimum Hamming distance of the coding scheme in Table

Solution

We first find all the Hamming distances.

$d(00000, 01011) = 3$	$d(00000, 10101) = 3$	$d(00000, 11110) = 4$
$d(01011, 10101) = 4$	$d(01011, 11110) = 3$	$d(10101, 11110) = 3$

The d_{min} in this case is 3.

Note

To guarantee the *detection* of up to s errors in all cases, the minimum Hamming distance in a block code must be $d_{min} = s + 1$.

Example 10.7

The minimum Hamming distance for our first code scheme is 2. This code guarantees detection of only a single error. For example, if the third codeword (101) is sent and one error occurs, the received codeword does not match any valid codeword. If two errors occur, however, the received codeword may match a valid codeword and the errors are not detected.

Example 10.8

Our second block code scheme has $d_{min} = 3$. This code can detect up to two errors. Again, we see that when any of the valid codewords is sent, two errors create a codeword which is not in the table of valid codewords. The receiver cannot be fooled.

However, some combinations of three errors change a valid codeword to another valid codeword. The receiver accepts the received codeword and the errors are undetected.

Error Control

Error Control comprises of Error Detection and Error Correction

Error Detection Techniques

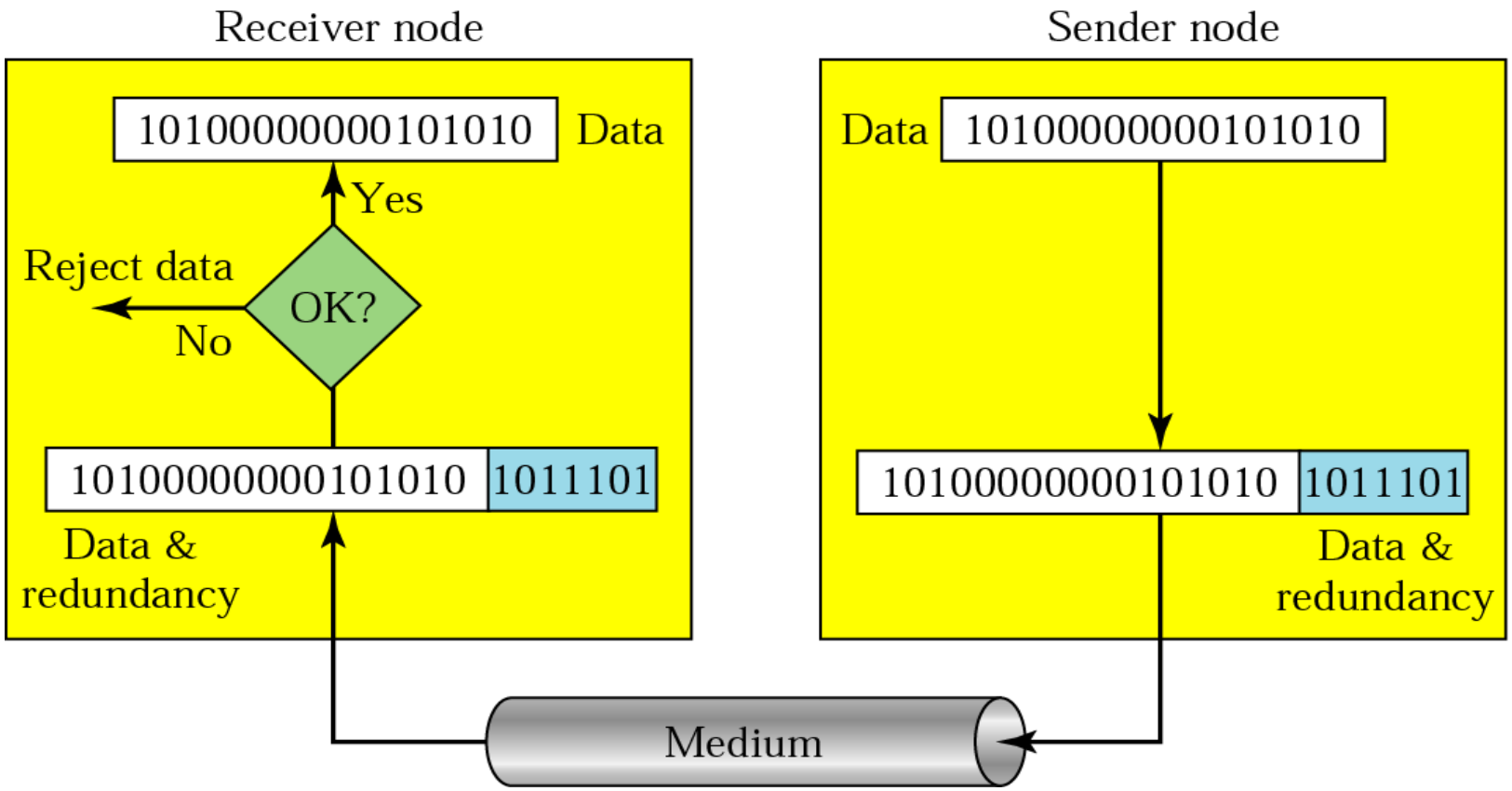
Redundancy

Parity Check

Cyclic Redundancy Check (CRC)

Checksum

Redundancy


Receiver node
Sender node
1010000000101010
Data
Yes
Reject data
OK?
No
1010000000101010
1011101
Data &
redundancy
Data
1010000000101010
1010000000101010
1011101
Data &
redundancy
Medium

LINEAR BLOCK CODES

Almost all block codes used today belong to a subset called linear block codes. A linear block code is a code in which is the exclusive OR (addition modulo-2) of two valid codewords creates another valid codeword.

Topics discussed in this section:

Minimum Distance for Linear Block Codes

Some Linear Block Codes

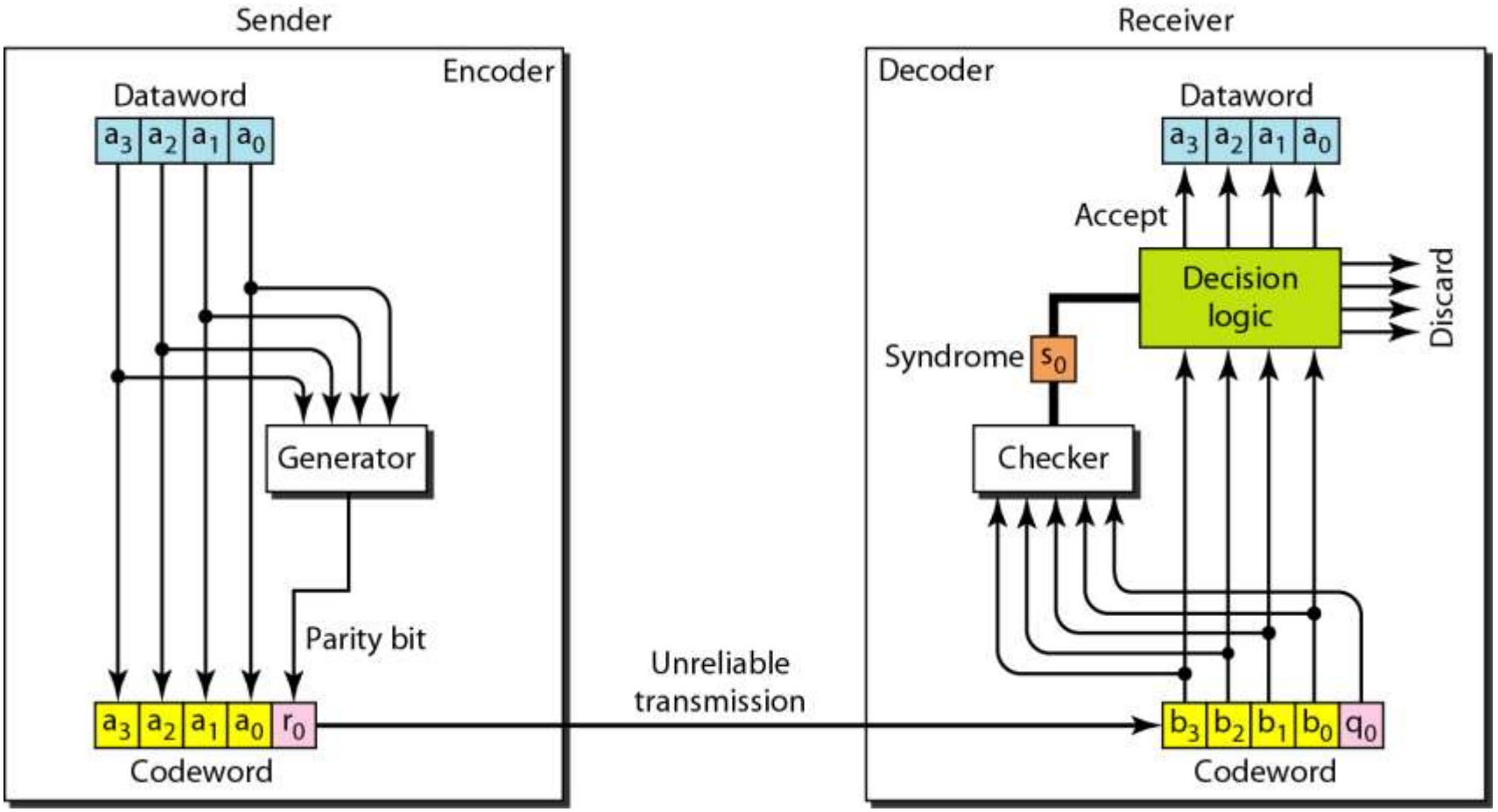
Note

A simple parity-check code is a single-bit error-detecting code in which $n = k + 1$ with $d_{min} = 2$.

Simple parity-check code C(5, 4)

Datawords	*Codewords*	*Datawords*	*Codewords*
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Encoder and decoder for simple parity-check code



Example 10.12

Let us look at some transmission scenarios. Assume the sender sends the dataword 1011. The codeword created from this dataword is 10111, which is sent to the receiver. We examine five cases:

1. No error occurs; the received codeword is 10111. The syndrome is 0. The dataword 1011 is created.
2. One single-bit error changes a_1 . The received codeword is 10011. The syndrome is 1. No dataword is created.
3. One single-bit error changes r_0 . The received codeword is 10110. The syndrome is 1. No dataword is created.

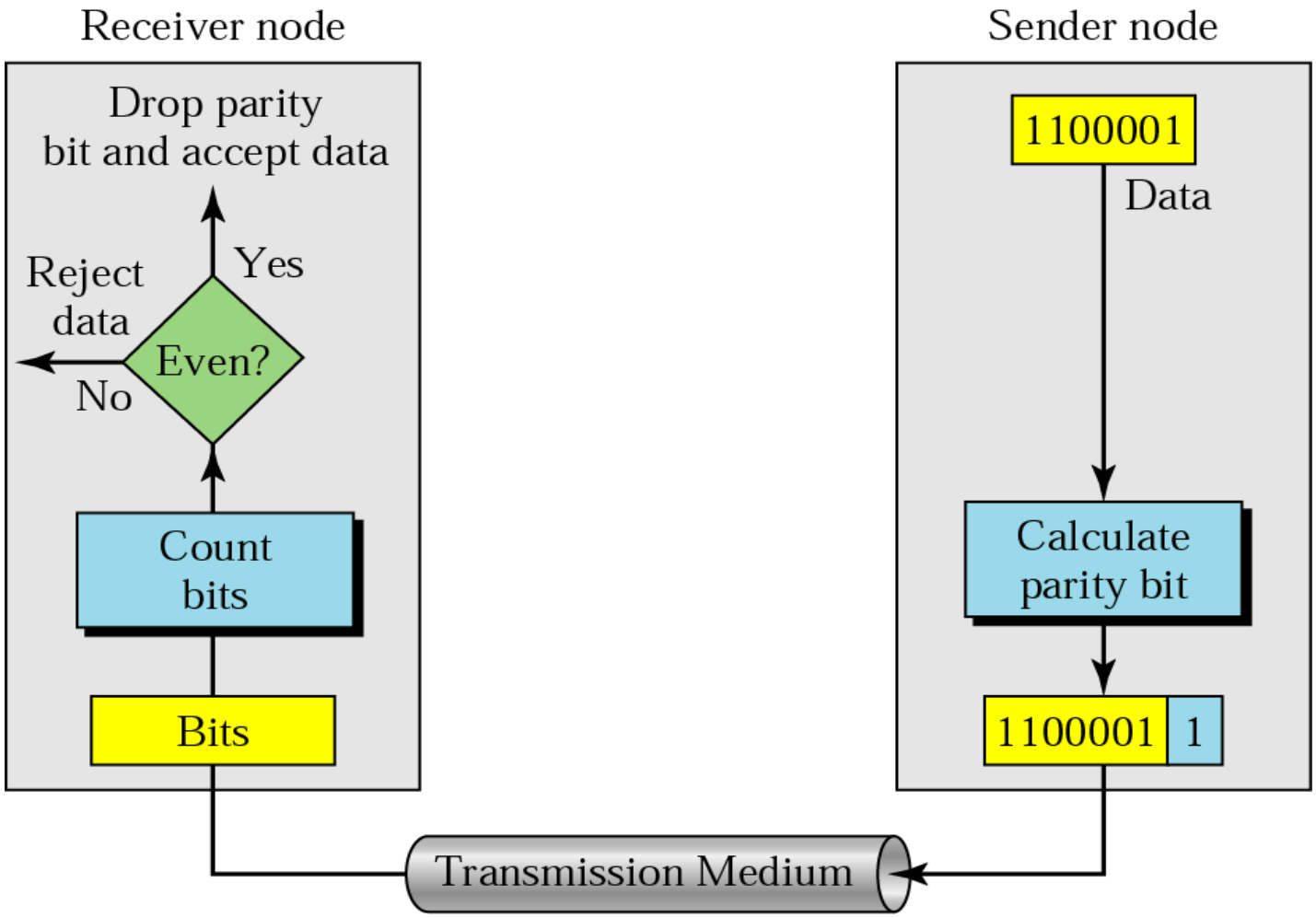
Example 10.12 (continued)

4. An error changes r_0 and a second error changes a_3 . The received codeword is 00110. The syndrome is 0. The dataword 0011 is created at the receiver. Note that here the dataword is wrongly created due to the syndrome value.
5. Three bits—a_3, a_2, and a_1—are changed by errors. The received codeword is 01011. The syndrome is 1. The dataword is not created. This shows that the simple parity check, guaranteed to detect one single error, can also find any odd number of errors.

Note

A simple parity-check code can detect an odd number of errors.

Even-parity concept



In parity check, a parity bit is added to every data unit so that the total number of 1s is even (or odd for odd-parity).

Example 1

Suppose the sender wants to send the word *world*. In ASCII the five characters are coded as

1110111 1101111 1110010 1101100 1100100

The following shows the actual bits sent

11101110 11011110 11100100 11011000 11001001

Example 2

Now suppose the word world in Example 1 is received by the receiver without being corrupted in transmission.

11101110 11011110 11100100 11011000 11001001

The receiver counts the 1s in each character and comes up with even numbers (6, 6, 4, 4, 4). The data are accepted.

Example 3

Now suppose the word world in Example 1 is corrupted during transmission.

11111110 11011110 11101100 11011000 11001001

The receiver counts the 1s in each character and comes up with even and odd numbers (7, 6, 5, 4, 4). The receiver knows that the data are corrupted, discards them, and asks for retransmission.

Simple parity check can detect all single-bit errors. It can detect burst errors only if the total number of errors in each data unit is odd.

Cyclic Redundancy Checksum

- Is a type of polynomial code, in which bit string is represented in the form of Polynomials, with coefficients of ‘0’ and ‘1’.
- Polynomial arithmetic uses a modulo-2 arithmetic. i.e. Addition and Subtraction are identical to “ E-XOR ”.
- For CRC code; sender and receiver should agree upon a ***generator polynomial*** G(x).
- A code-word can be generated for a given ***data word*** (message) ***Polynomial*** M(x), with the help of Long Division.
- This technique is more powerful than parity check or checksum error detection.

Cyclic Redundancy Checksum

The CRC error detection method treats the packet of data to be transmitted as a large polynomial.

The transmitter takes the message polynomial and using polynomial arithmetic, divides it by a given generating polynomial.

The quotient is discarded but the remainder is “attached” to the end of the message.

Cyclic Redundancy Checksum

The message (with the remainder) is transmitted to the receiver.

The receiver divides the message and remainder by the same generating polynomial.

If a remainder not equal to zero results, there was an error during transmission.

If a remainder of zero results, there was no error during transmission.

CRC Encoder and Decoder

- Encoder
  - Dataword has k bits
  - Codeword has n bits
  - Size of dataword is augmented by adding n-k 0's to RHS of the word
  - The n bit field is fed into generator
  - The generator uses a divisor of size n-k+1
  - The generator divided the augmented dataword by the divisor (modulo-2) division
  - Quotient of the division is discarded
  - The remainder r2,r1,r0 is appended to the data word to create the codeword
  - E.g Dataword is 1001 and divisor is 1011

CRC Encoder and Decoder

- Decoder
  - Copy of all bits is fed to checker (replica of generator)
  - The remainder produced by the checker is a syndrome of n-k
  - The result is fed to decision analyzer
  - If syndrome bits are all 0's, the 4 left-most bits of the codeword are accepted as dataword (no error); otherwise the four bits are discarded

ERROR DETECTION :

CYCLIC REDUNDANCY CHECK :

Is a type of polynomial code, in which bit string is represented in the form of Polynomials, with coefficients of ‘0’ and ‘1’.

Polynomial arithmetic uses a modulo-2 arithmetic. i.e. Addition and Subtraction are identical to “ E-XOR ”.

For CRC code; sender and receiver should agree upon a ***generator polynomial*** G(x).

A code-word can be generated for a given ***data word*** (message) ***Polynomial*** M(x), with the help of Long Division.

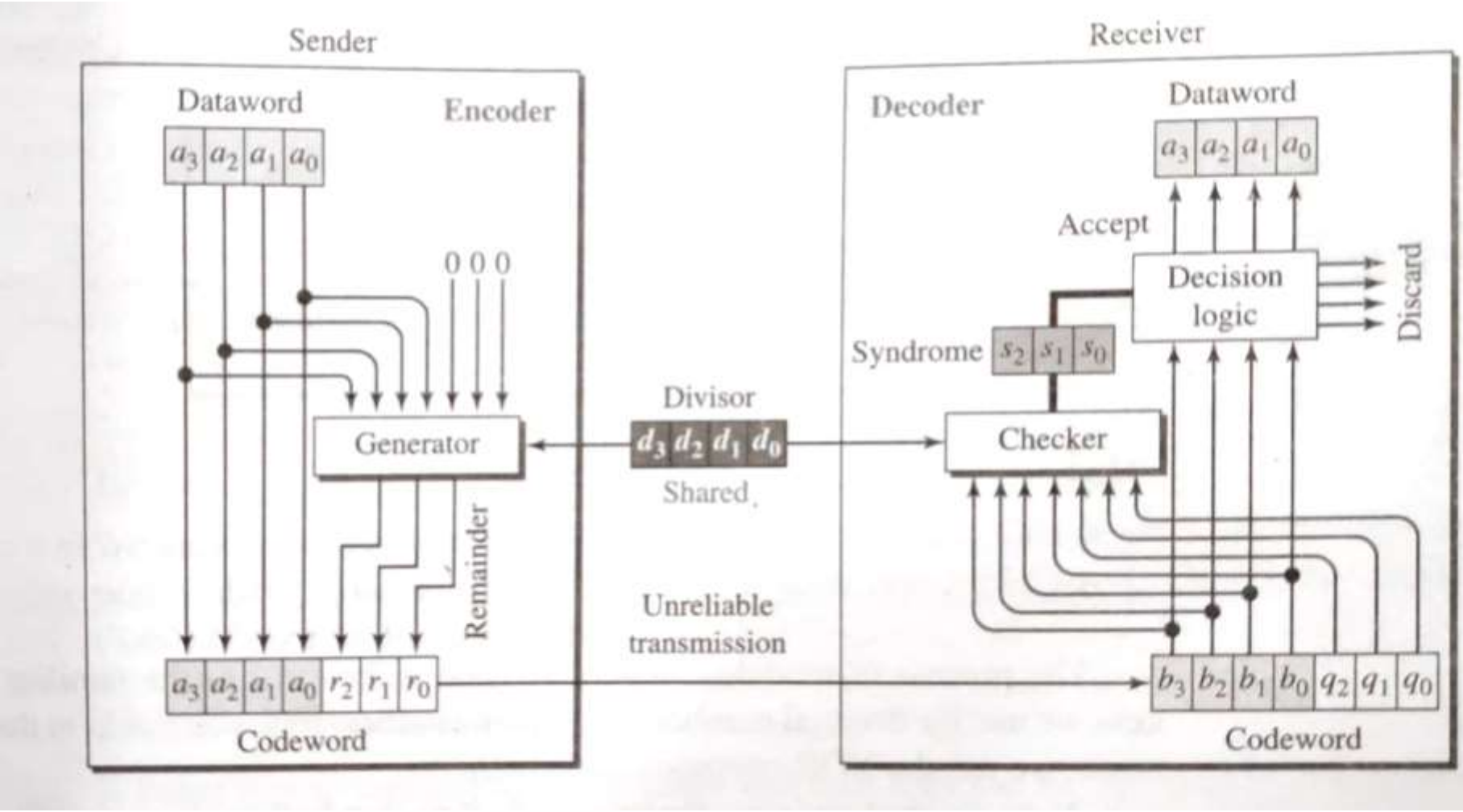
This technique is more powerful than parity check or checksum error detection.

CRC Generator :

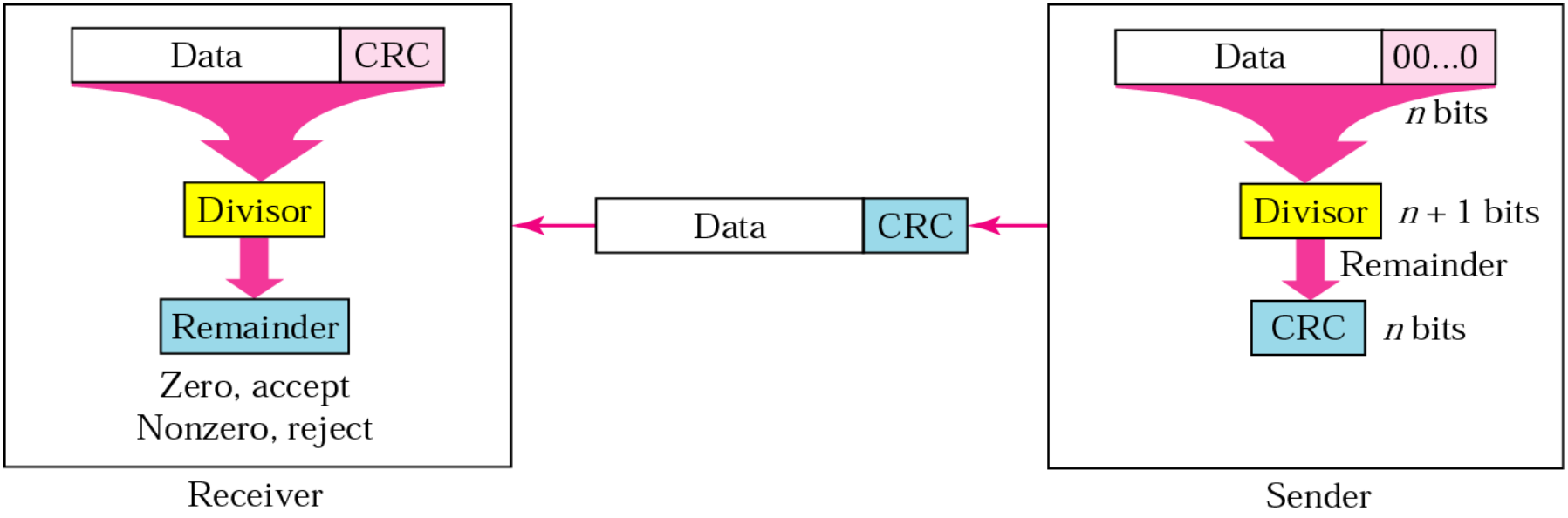
Step wise procedure:

1. Append a string of “n” 0’s to the data unit, where n is “1” less than the number of bits in the pre-decided divisor. (n+1 bits long).
2. Divide the newly generated data unit in step 1 by the divisor. This is binary division.
3. The remainder obtained after the division in step 2 is the n bit CRC.
4. The CRC will replace the “n” 0’s appended to the data unit in step 1, to get the code-word to be transmitted.

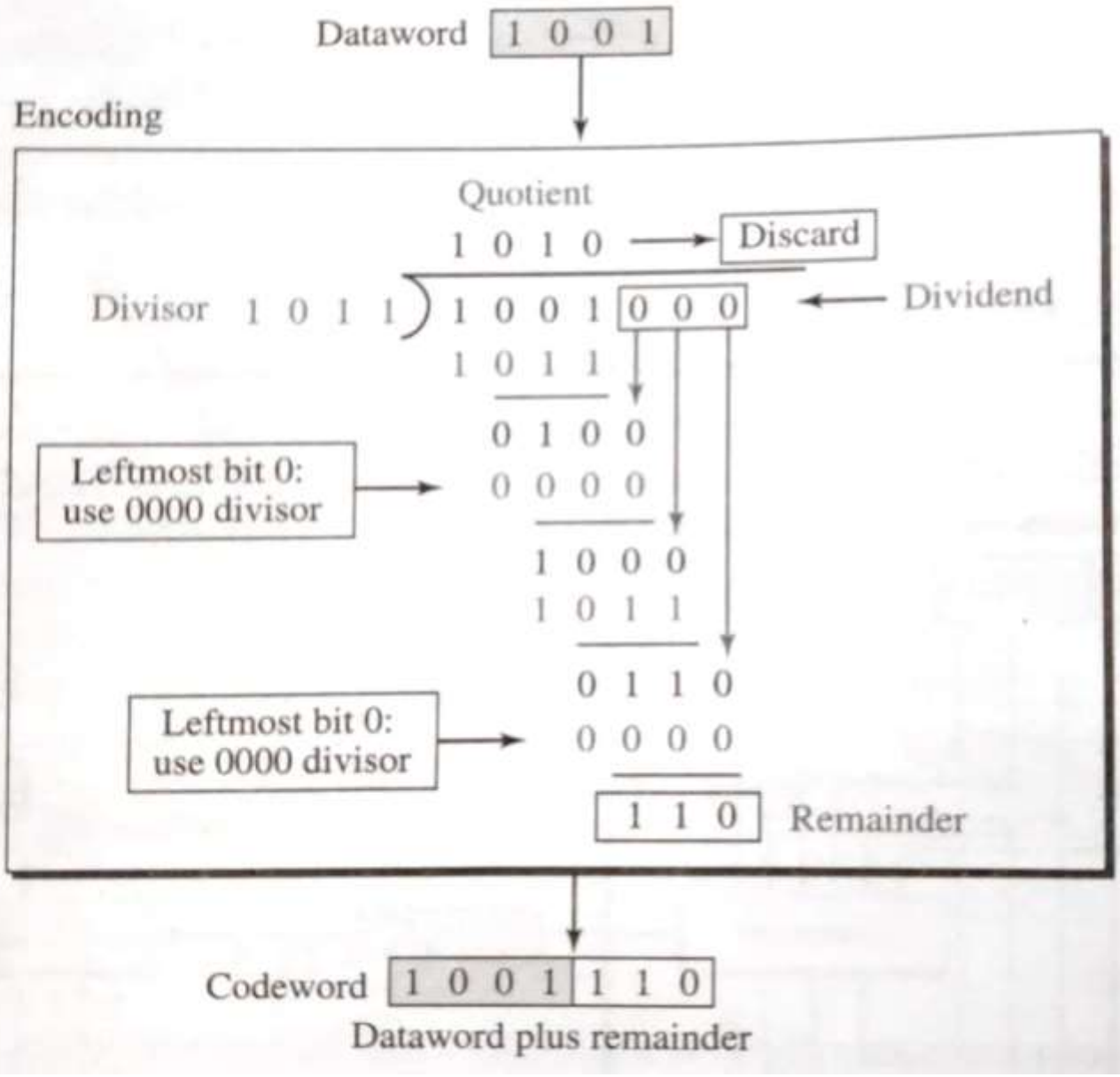
CRC Encoder and Decoder



CRC generator and checker



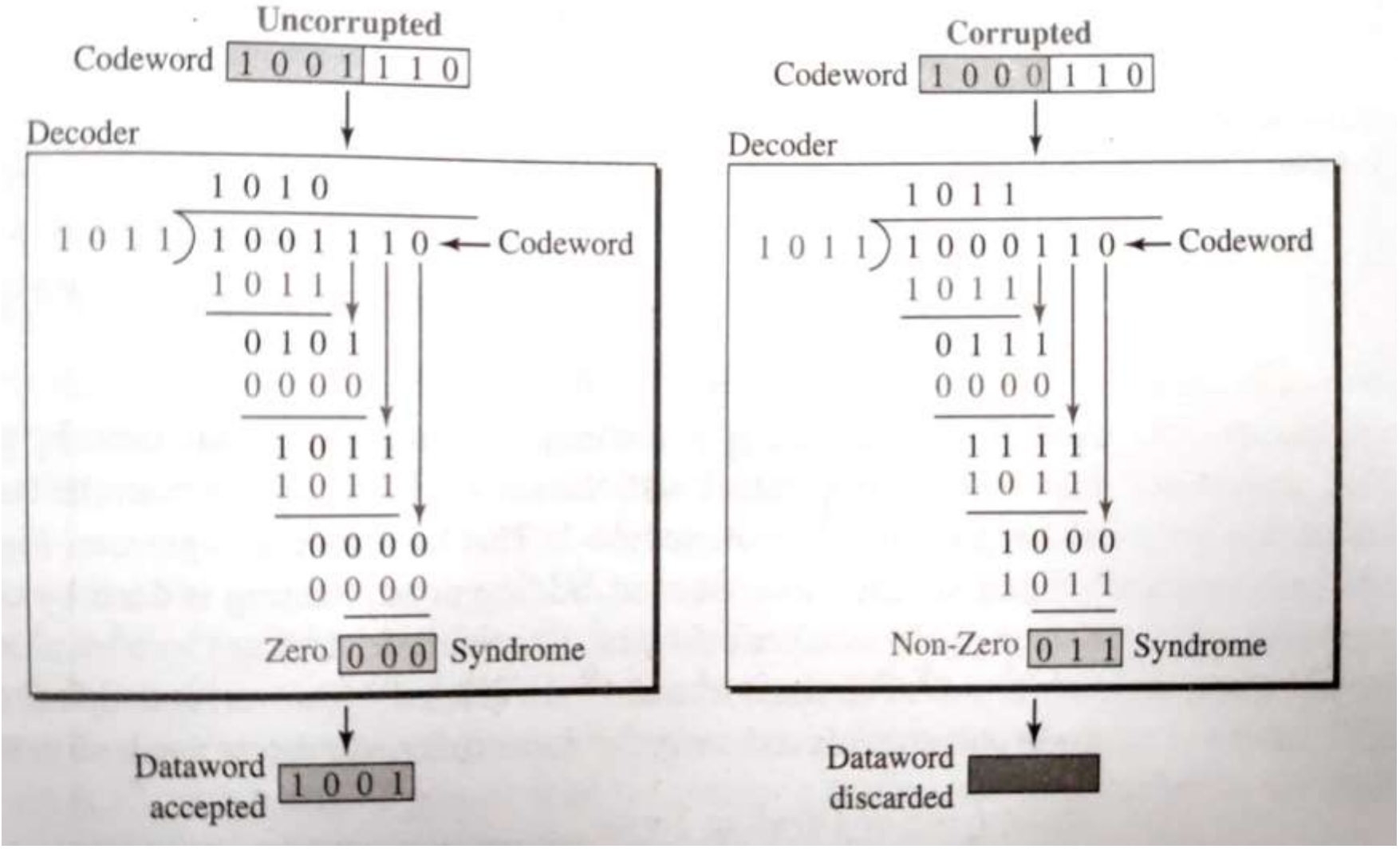
Binary division in a CRC generator



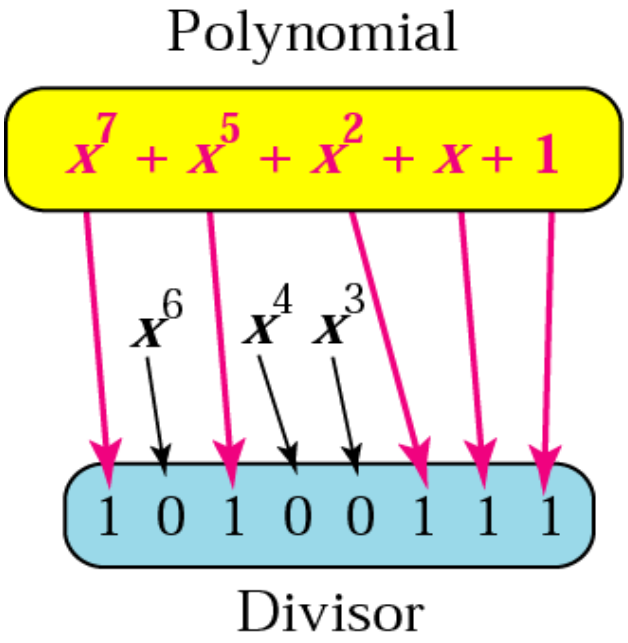
Note:

Multiply: AND
Subtract: XOR

Binary division in CRC checker



A polynomial representing a divisor



Standard polynomials

Name	Polynomial	Application
CRC-8	$x^8 + x^2 + x + 1$	**ATM header**
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	**ATM AAL**
ITU-16	$x^{16} + x^{12} + x^5 + 1$	**HDLC**
ITU-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	**LANs**

CHECKSUM

The last error detection method we discuss here is called the checksum, or arithmetic checksum. The checksum is used in the Internet by several protocols although not at the data link layer. However, we briefly discuss it here to complete our discussion on error checking

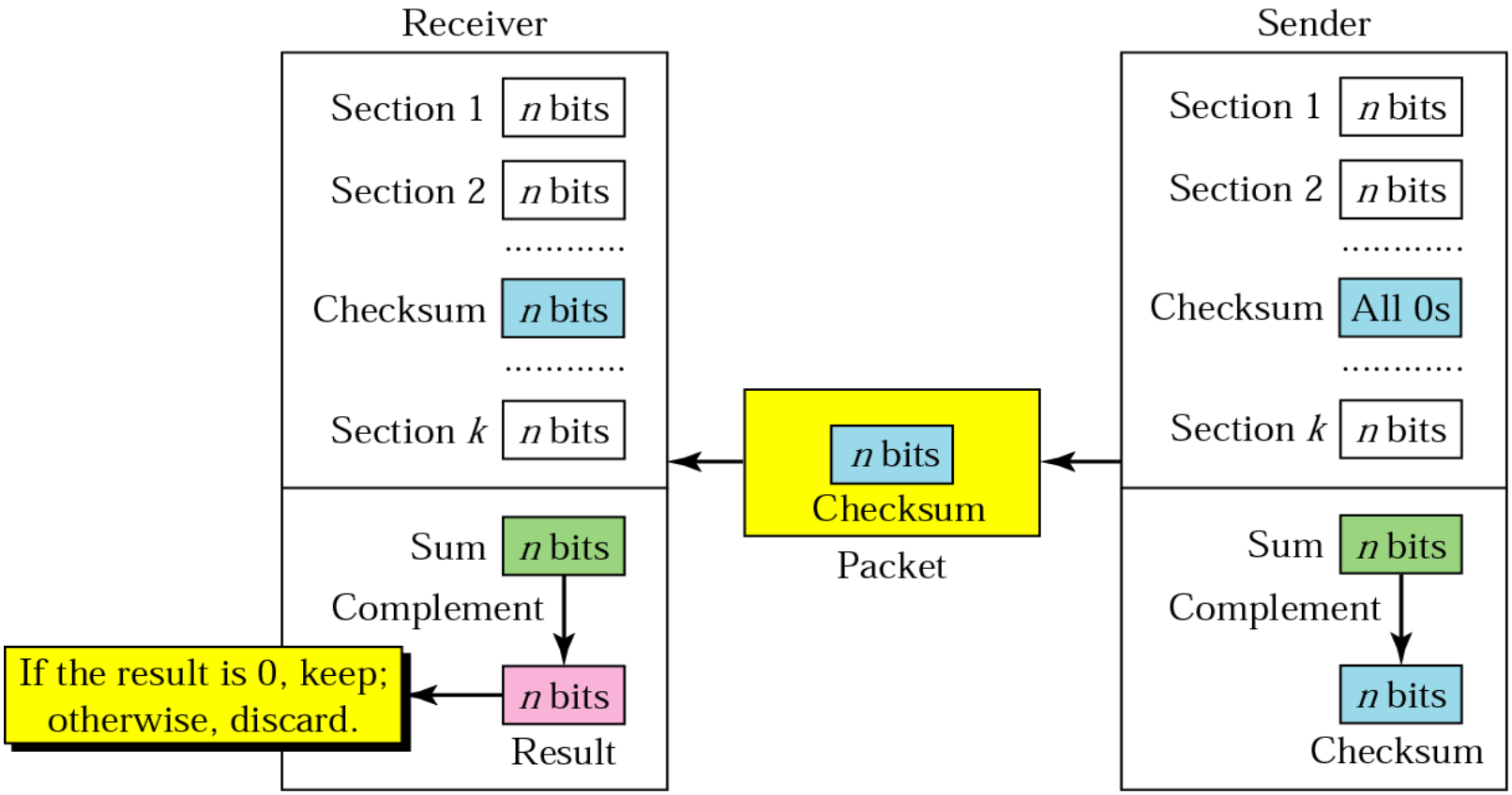
Example

Suppose our data is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers. For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum. If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum. Otherwise, there is an error somewhere and the data are not accepted.

Example

We can make the job of the receiver easier if we send the negative (complement) of the sum, called the checksum. In this case, we send (7, 11, 12, 0, 6, −36). The receiver can add all the numbers received (including the checksum). If the result is 0, it assumes no error; otherwise, there is an error.

Checksum


Receiver
Section 1
n bits
Section 2
n bits
Checksum
n bits
Section k
n bits
Sum
n bits
Complement
n bits
Result
If the result is 0, keep; otherwise, discard.
n bits
Checksum
Packet
Sender
Section 1
n bits
Section 2
n bits
Checksum
All 0s
Section k
n bits
Sum
n bits
Complement
n bits
Checksum

Note:

The sender follows these steps:

- *The unit is divided into k sections, each of n bits.*
- *All sections are added using one's complement to get the sum.*
- *The sum is complemented and becomes the checksum.*
- *The checksum is sent with the data.*

Note:

The receiver follows these steps:

- *The unit is divided into k sections, each of n bits.*
- *All sections are added using one's complement to get the sum.*
- *The sum is complemented.*
- *If the result is zero, the data are accepted: otherwise, rejected.*

Example 7

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits.

10101001 00111001

The numbers are added using one's complement

```
                10101001
                00111001
                ----------
Sum             11100010
```

Checksum 00011101

The pattern sent is 10101001 00111001 00011101

Example 8

Now suppose the receiver receives the pattern sent in Example 7 and there is no error.

10101001 00111001 00011101

When the receiver adds the three sections, it will get all 1s, which, after complementing, is all 0s and shows that there is no error.

	10101001
	00111001
	00011101
Sum	11111111
Complement	00000000 means that the pattern is OK.

Example 9

Now suppose there is a burst error of length 5 that affects 4 bits.

10101111 11111001 00011101

When the receiver adds the three sections, it gets

	10101111	
	11111001	
	00011101	
Partial Sum	1 11000101	
Carry	1	
Sum	11000110	
Complement	00111001	the pattern is corrupted.

Hamming Code

- Hamming code detects and corrects the errors that can occur when the data is moved from the sender to the receiver.
- It adds extra bits to the original data, allowing the system to detect and correct single-bit errors.
- Redundant bits are extra binary bits that are generated and added to the information-carrying bits of data transfer to ensure that no bits were lost during the data transfer. The number of redundant bits can be calculated using the following formula:

$$2^r \geq m + r + 1$$

m: no. of message bits , r : no. of redundant bits

- Suppose the number of data bits is 7, then the number of redundant bits can be calculated as:

  $2^4 \geq 7 + 4 + 1$ Thus, the number of redundant bits is 4.

Relationship between m and r

$$2^r \geq m + r + 1$$

Message bits :m	Redundant bits/parity bits : r	Total bits m+r
1	2	3
2	3	5
3	3	6
4	3	7
5	4	9
6	4	10
7	4	11

Types of Parity Bits

- A parity bit is a bit appended to a data of binary bits to ensure that the total number of 1’s in the data is even or odd. Parity bits are used for error detection. There are two types of parity bits:
- **Even Parity Bit:** In the case of even parity, for a given set of bits, the number of 1’s are counted. If that count is odd, the parity bit value is set to 1, making the total count of occurrences of 1’s an even number. If the total number of 1’s in a given set of bits is already even, the parity bit's value is 0.
- **Odd Parity Bit:** In the case of odd parity, for a given set of bits, the number of 1’s are counted. If that count is even, the parity bit value is set to 1, making the total count of occurrences of 1’s an odd number. If the total number of 1’s in a given set of bits is already odd, the parity bit's value is 0.

Algorithm of Hamming Code

Hamming Code is simply the use of extra parity bits to allow the identification of an error.

- Step 1: Write the bit positions starting from 1 in binary form (1, 10, 11, 100, etc).
- Step 2: All the bit positions that are a power of 2 are marked as parity bits (1, 2, 4, 8, etc).
- Step 3: All the other bit positions are marked as data bits.
- Step 4: Each data bit is included in a unique set of parity bits, as determined its bit position in binary form:
  - **Parity bit 1 covers all the bits positions whose binary representation includes a 1 in the least significant position (1, 3, 5, 7, 9, 11, etc).**
  - **Parity bit 2 covers all the bits positions whose binary representation includes a 1 in the second position from the least significant bit (2, 3, 6, 7, 10, 11, etc).**
  - **Parity bit 4 covers all the bits positions whose binary representation includes a 1 in the third position from the least significant bit (4–7, 12–15, 20–23, etc).**
  - **Parity bit 8 covers all the bits positions whose binary representation includes a 1 in the fourth position from the least significant bit bits (8–15, 24–31, 40–47, etc).**
  - **In general, each parity bit covers all bits where the bitwise AND of the parity position and the bit position is non-zero.**

Cntd..Algorithm of Hamming Code

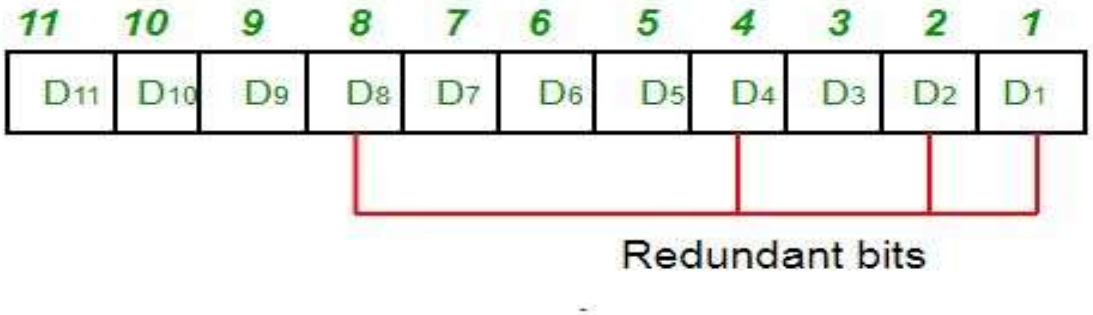
- Step 5: Since we check for even parity set a parity bit to 1 if the total number of ones in the positions it checks is odd. Set a parity bit to 0 if the total number of ones in the positions it checks is even.

Position	R8	R4	R2	R1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1

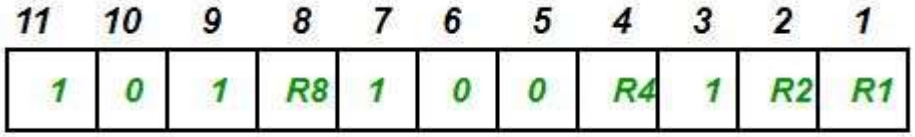
R1 -> 1,3,5,7,9,11
R2 -> 2,3,6,7,10,11
R3 -> 4,5,6,7
R4 -> 8,9,10,11

Hamming code…Determining The Position of Redundant Bits

- A redundancy bits are placed at positions that correspond to the power of 2. As in the above example:
- The number of data bits = 7 ,The number of redundant bits = 4 ,The total number of bits = 7+4=11
- The redundant bits are placed at positions corresponding to power of 2 that is 1, 2, 4, and 8



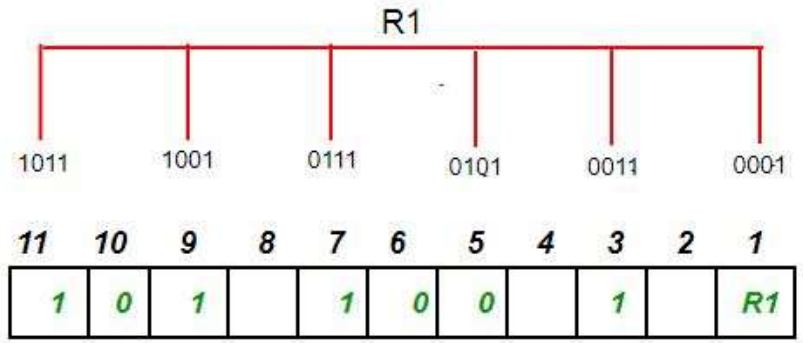
- If message bts are : 1011001

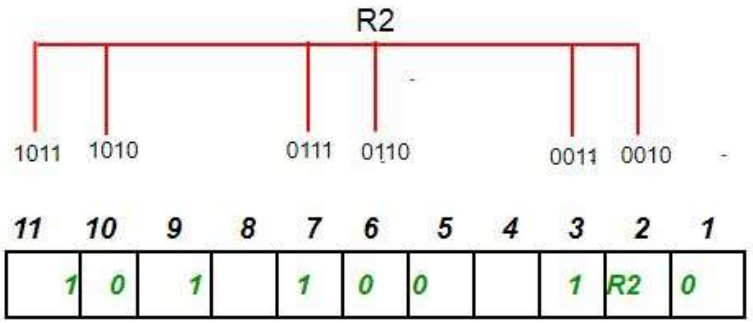


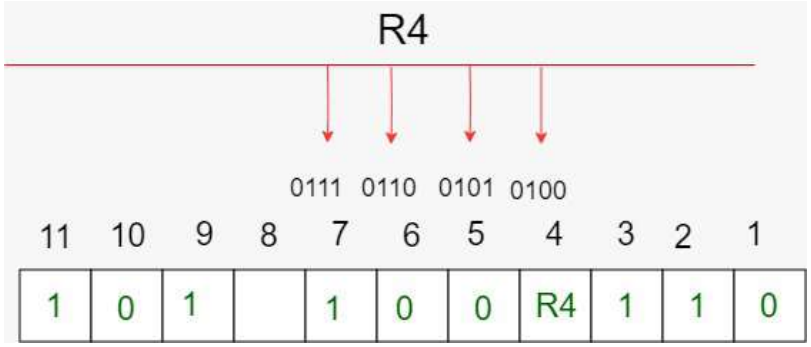
Determining The Parity Bits According to Even Parity

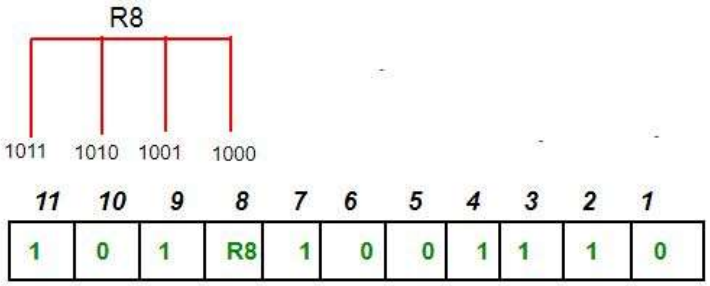
- R1: bits 1, 3, 5, 7, 9, 11 , R2: bits 2,3,6,7,10,11,

R4: bits 4,5,6,7 , R8 : bits8,9,10,11



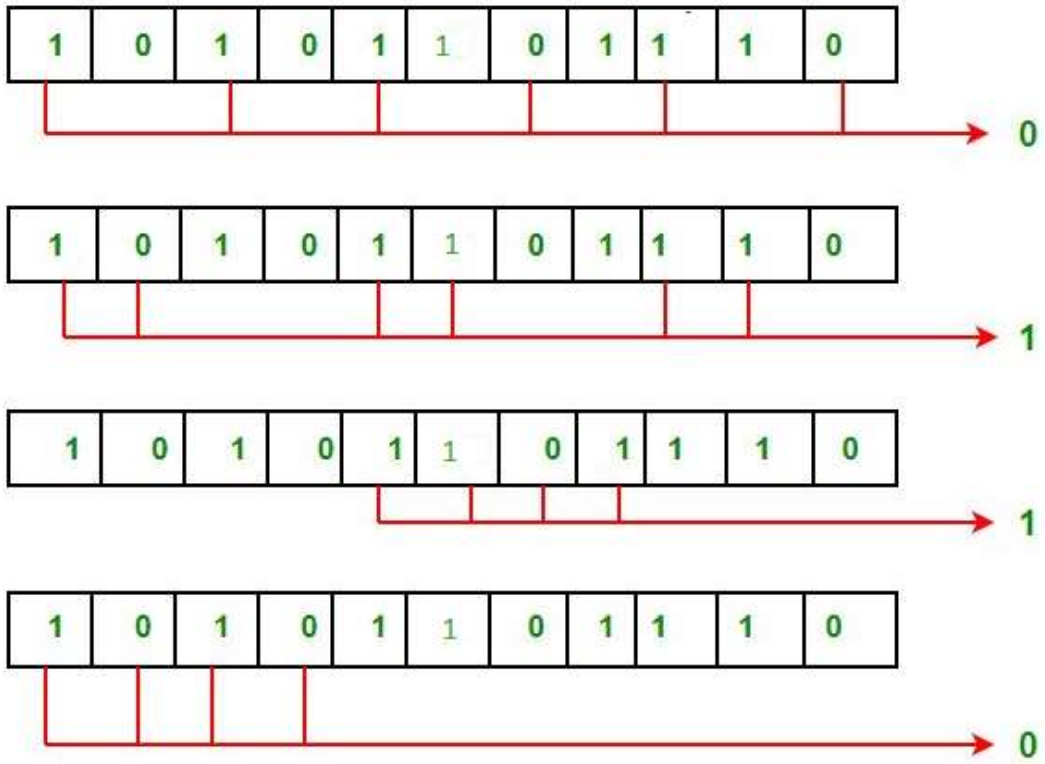






Hamming code: Error Detection and Correction

- Suppose in the above example the 6th bit is changed from 0 to 1 during data transmission, then it gives new parity values in the binary number:

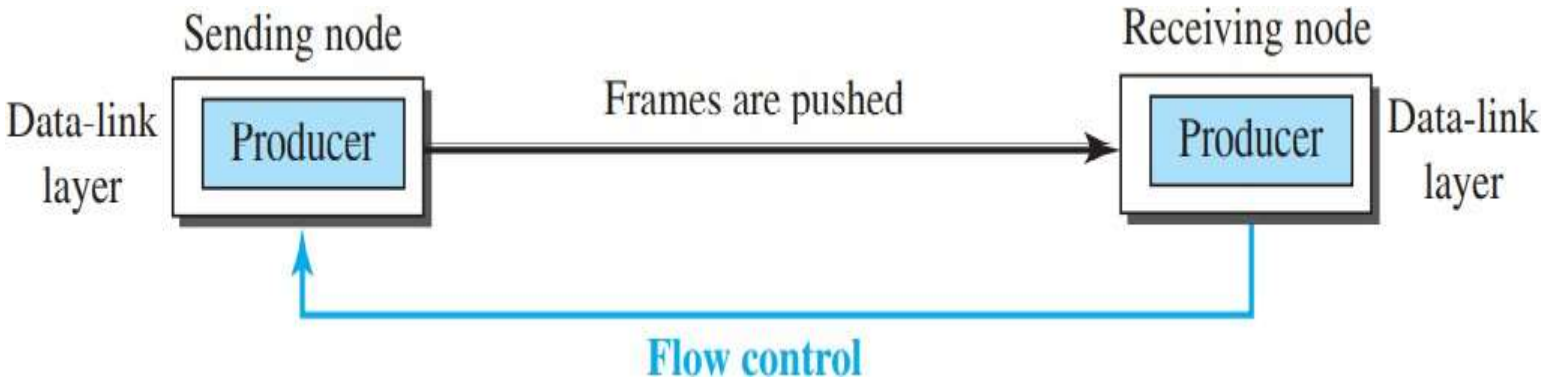


Hamming code: Error Detection and Correction

- For all the parity bits we will check the number of 1's in their respective bit positions.
- For R1: bits 1, 3, 5, 7, 9, 11. We can see that the number of 1's in these bit positions are 4 and that's even so we get a 0 for this.
- For R2: bits 2,3,6,7,10,11 . We can see that the number of 1's in these bit positions are 5 and that's odd so we get a 1 for this.
- For R4: bits 4, 5, 6, 7 . We can see that the number of 1's in these bit positions are 3 and that's odd so we get a 1 for this.
- For R8: bit 8,9,10,11 . We can see that the number of 1's in these bit positions are 2 and that's even so we get a 0 for this.
- The bits give the binary number 0110 whose decimal representation is 6. Thus, bit 6 contains an error. To correct the error the 6th bit is changed from 1 to 0.

Flow Control and Error Control

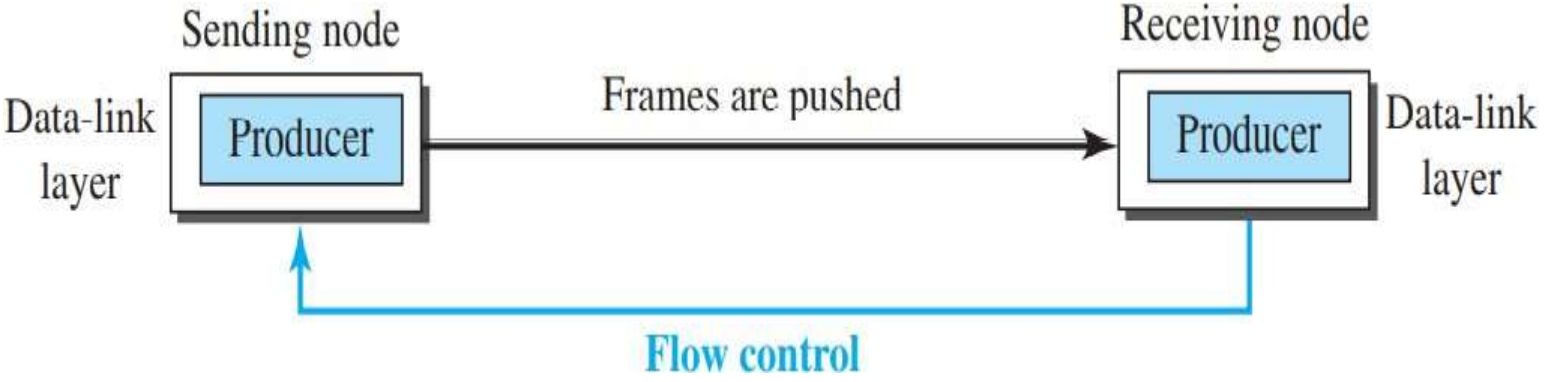
One of the responsibilities of the data link control sublayer is flow and error control at the data-link layer. Collectively, these functions are known as data link control. waiting for acknowledgment.



Flow Control and Error Control

Flow Control at Data Link Layer: Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before.

Error control in the data link layer is based on automatic repeat request(ARQ), which is the retransmission of data.



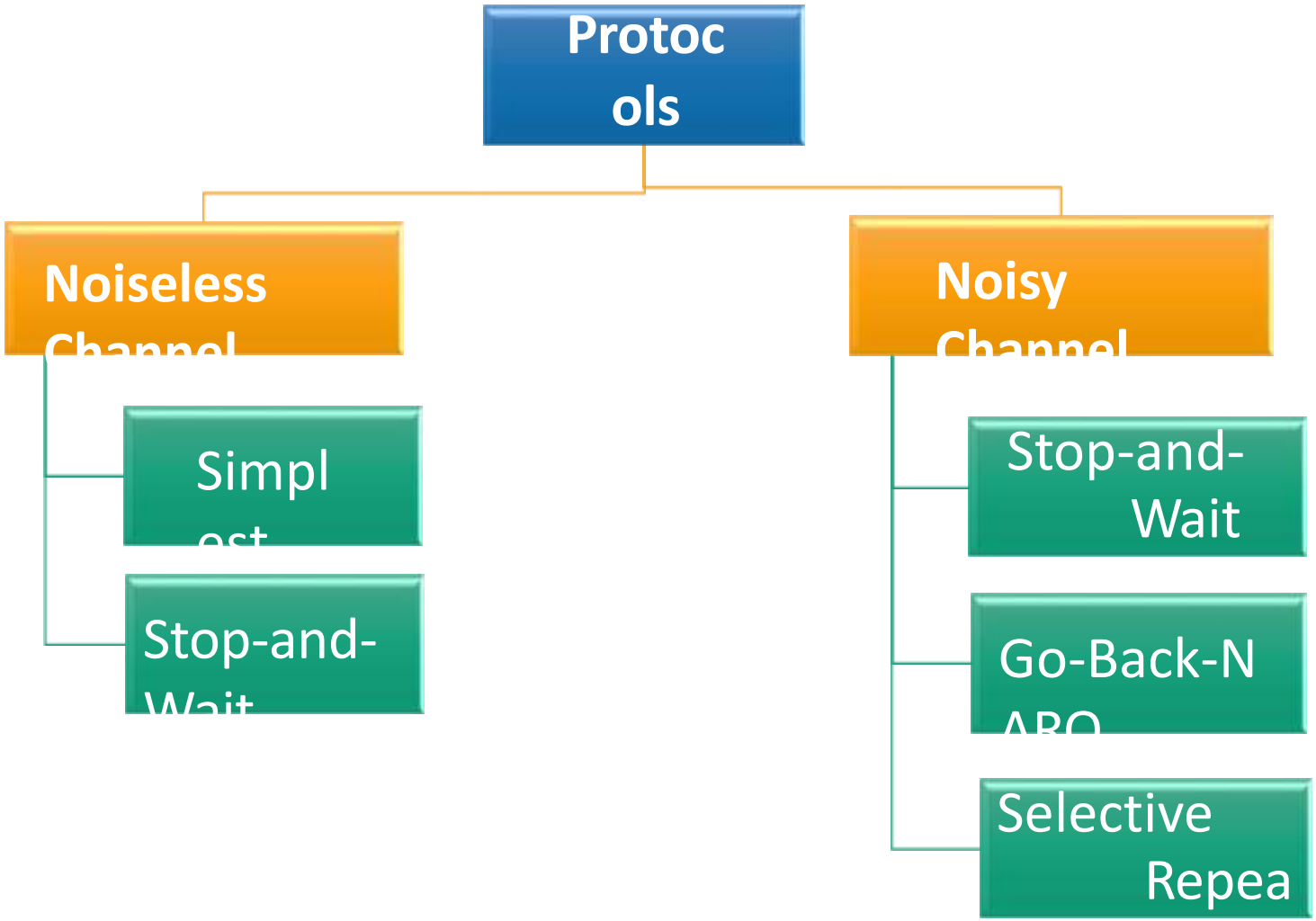
Flow Control and Error Control

- Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.

- Flow of data must not allow to overwhelm the receiver.

- Receiving device have limited speed at which it can process incoming data and limited amount of memory to store incoming data.

- Error control is both error detection and


Protoc
ols
Noiseless
Channel
Simpl
ost
Stop-and-
Wait
Noisy
Channel
Stop-and-
Wait
Go-Back-N
ARQ
Selective
Repea

Noiseless Channel

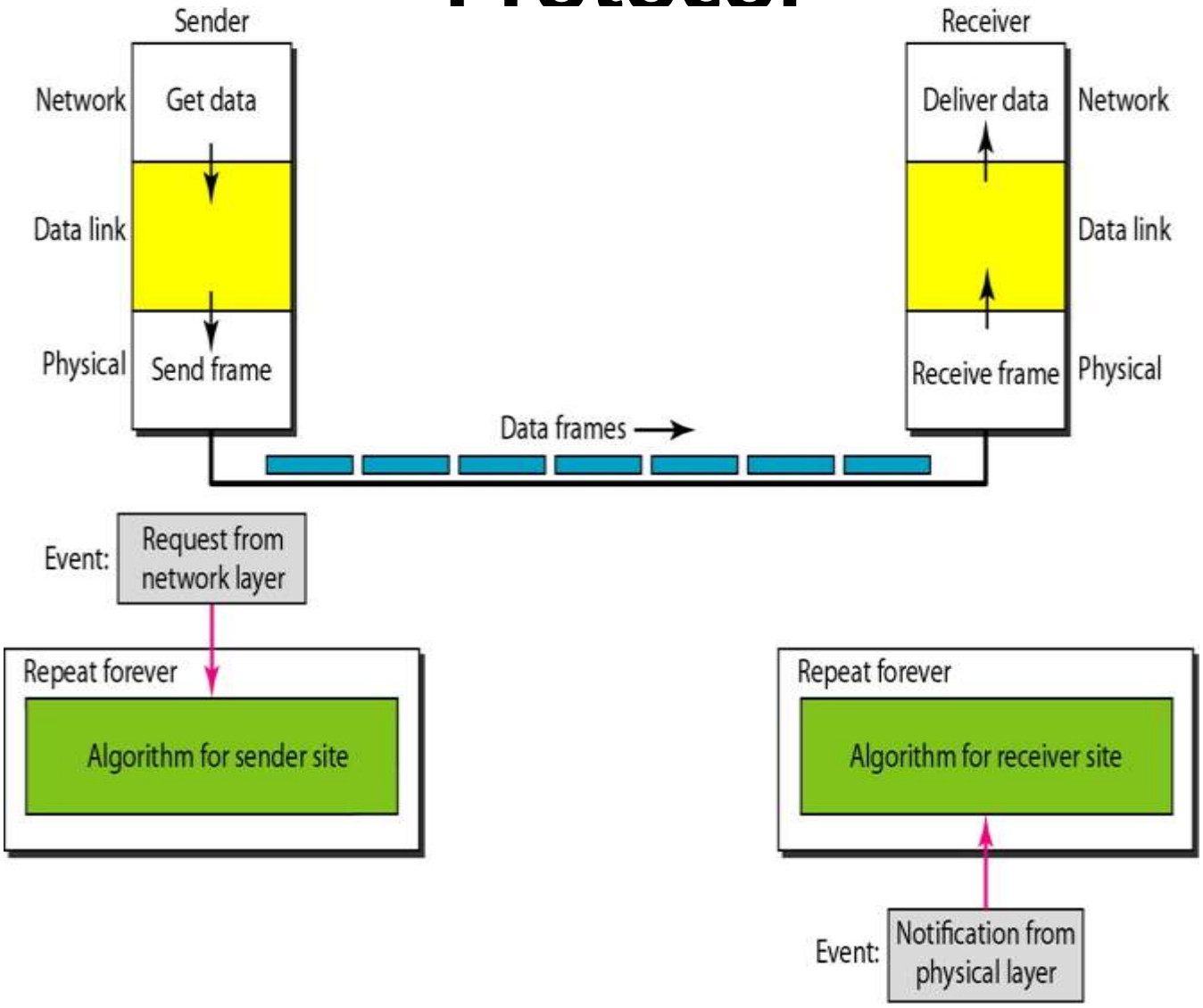
Let us first assume we have an ideal channel in which no frames are lost, duplicated, or corrupted. We introduce two protocols for this type of channel.

- Simplest Protocol
- Stop-and-Wait Protocol

Simplest Protocol

- Cannot be used in real life
- No flow and error control
- Receiver can handle any frame with negligibleprocessing time.
- Receiver can never be overwhelmed with incoming frames.

Simplest Protocol



Simplest Protocol *(Example)*

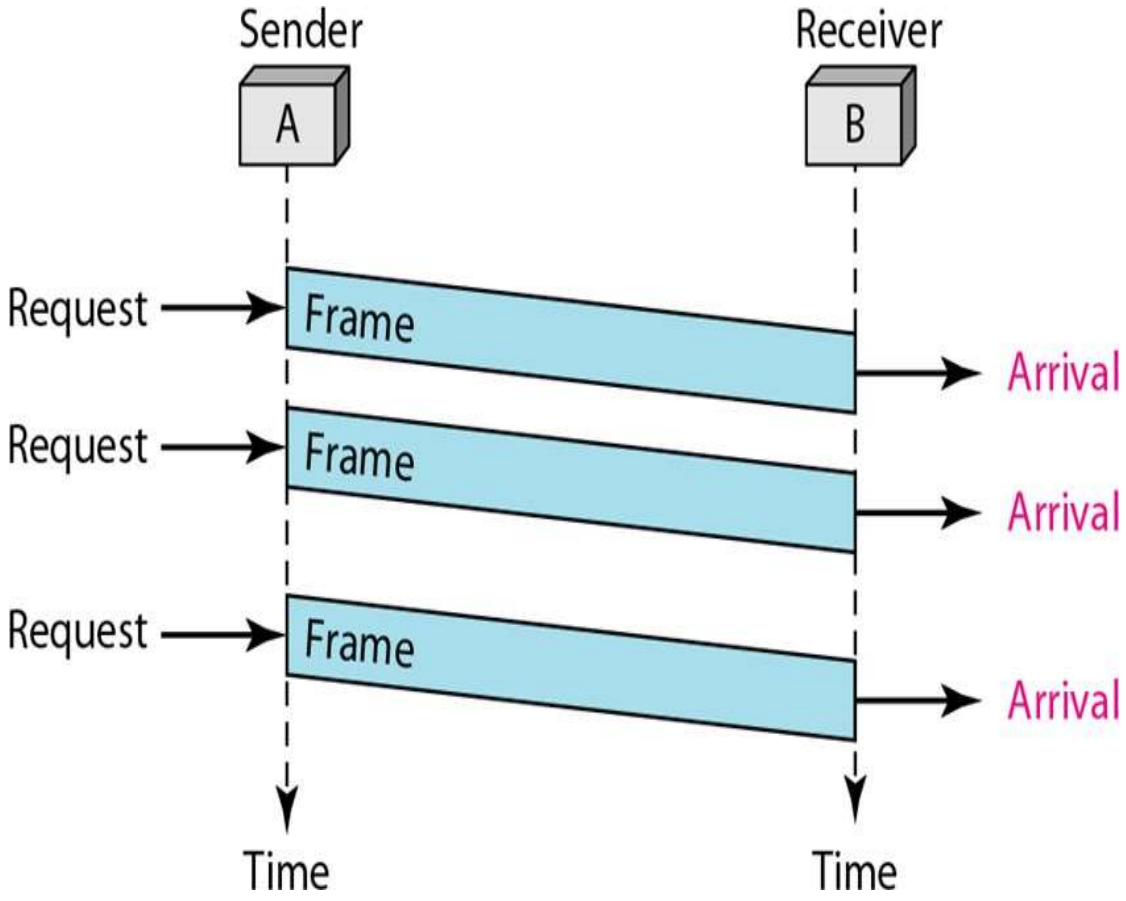
***Figure** shows an example of communication using this protocol.*

- *The sender sends a sequence of frames without even thinking about the receiver.*
- *To send three frames, three events occur at the sender site and three events at the receiver site.*

Note that the data frames are shown by tilted boxes; the height of the box defines the transmission time difference between the first bit and the last bit in the frame.

Simplest Protocol *(Example)*

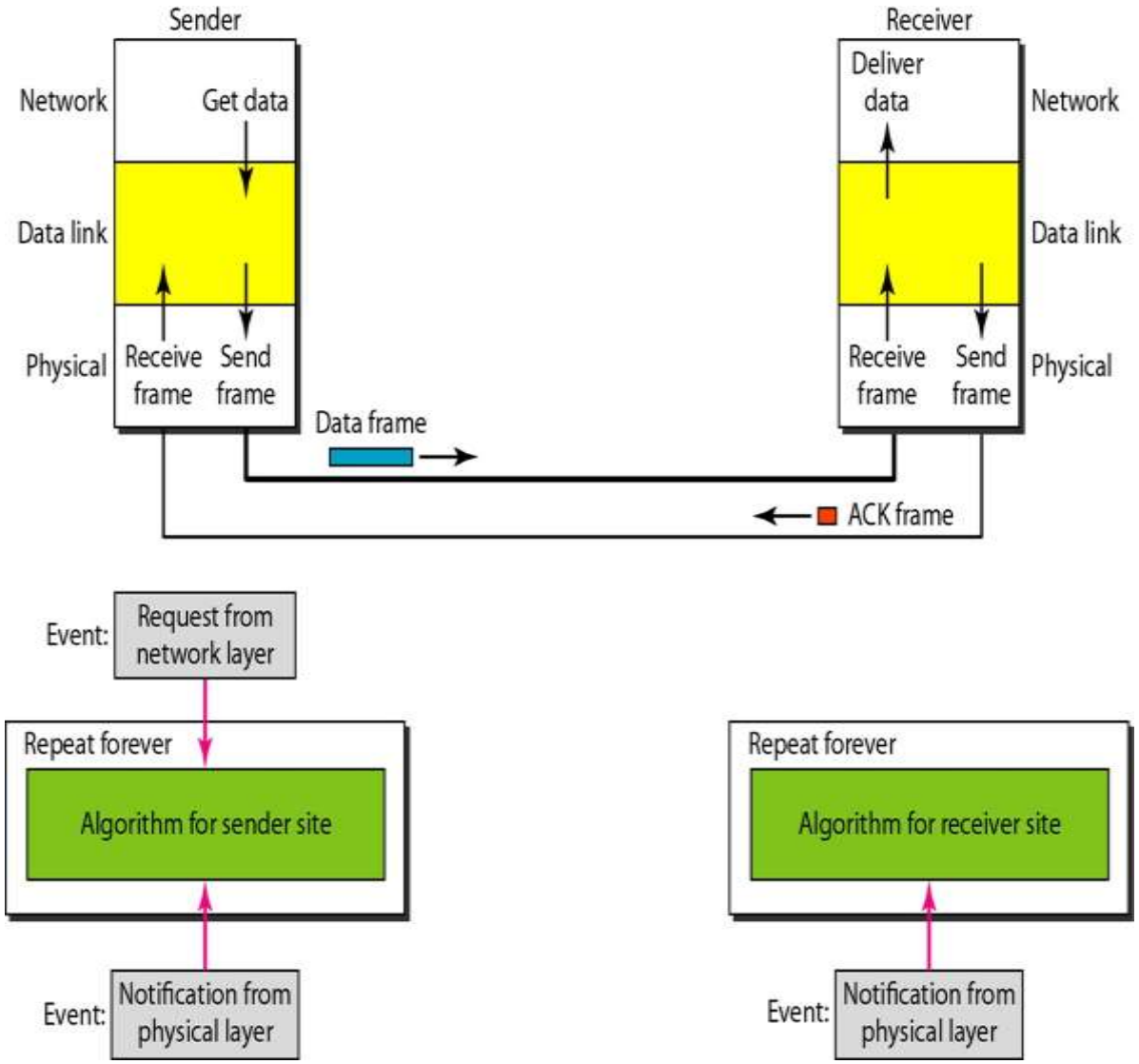
Figure: *Flow Diagram*



Stop and Wait Protocol

- If frames arrive faster than they can be processed, frames must be stored until their use.
- To prevent receiver from becoming overwhelmed, receiver needs to tell sender to slow down (ACK).
- Sender sends one frame, stops until it receives confirmation from receiver and then send next frame.

Stop and Wait Protocol



Stop and Wait Protocol

(Example)

Figure *shows an example of communication protocol. It is still very simple.*

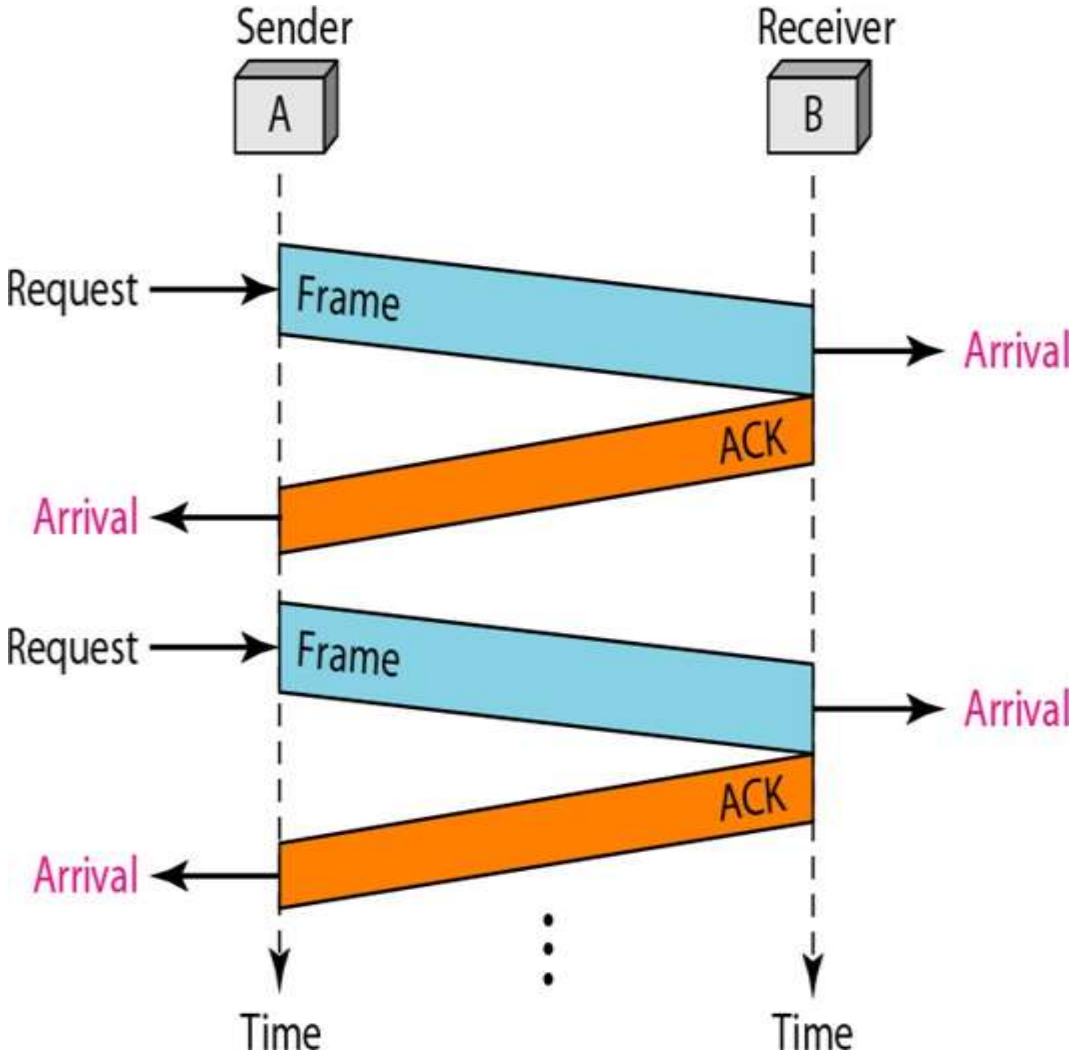
- *The sender sends one frame and waits for feedback from the receiver.*
- *When the ACK arrives, the sender sends the next frame.*

Note that sending two frames in the protocol involves the sender in four events and the receiver in two events.

using this

Stop and Wait Protocol *(Example)*

Figure: *Flow Diagram*



SOMAIYA VIDYAVIHAR UNIVERSITY

Noisy Channel

Although the Stop-and-Wait Protocol gives us an idea of how to add flow control to its predecessor, noiseless channels are nonexistent. We discuss three protocols in this section that use error control.

- Stop-and-Wait Automatic Repeat Request
- Go-Back-N Automatic Repeat Request
- Selective Repeat Automatic Repeat Request

Somaiya TRUST

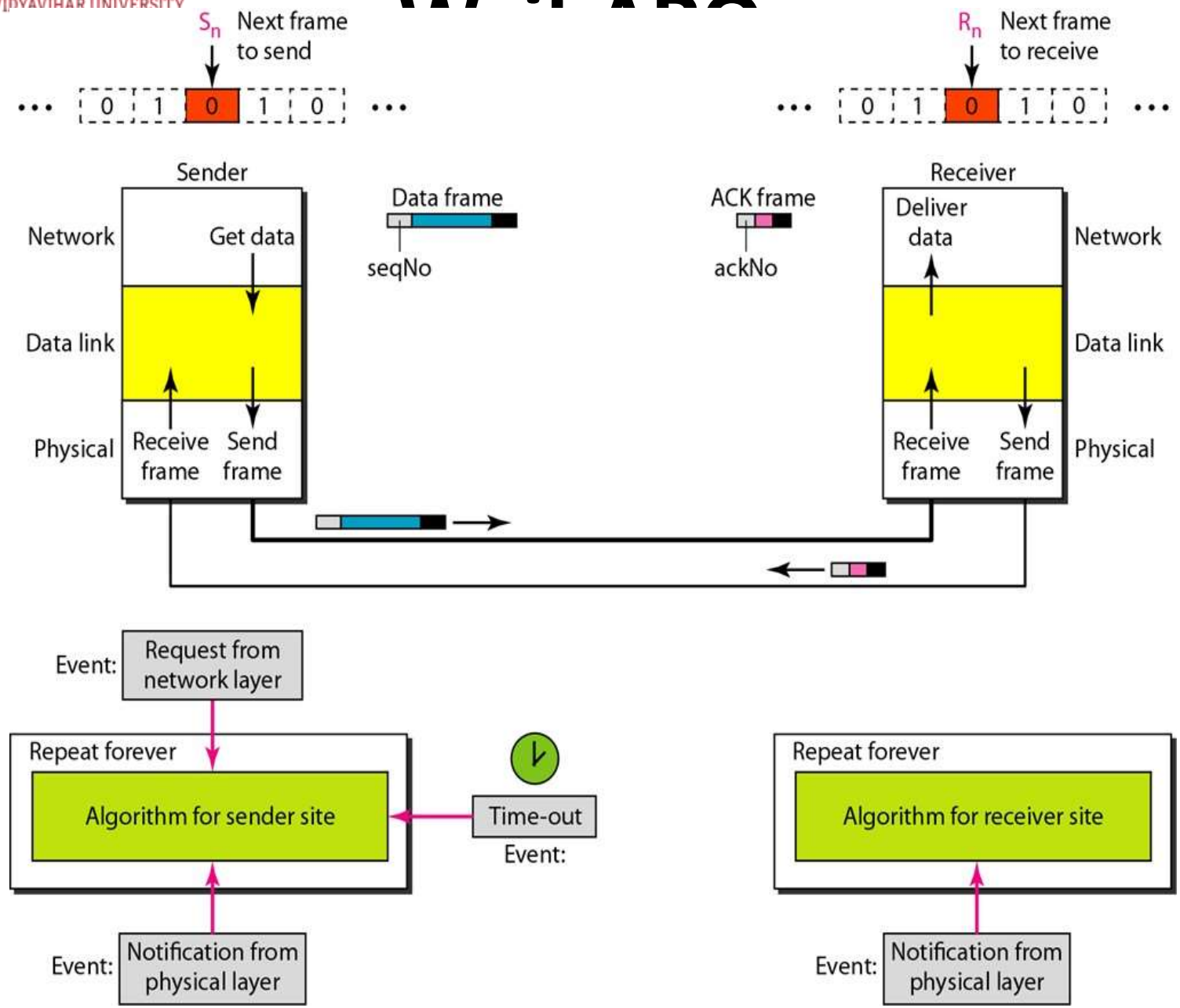
SOMAIYA VIDYAVIHAR UNIVERSITY

Stop And Wait ARQ

- Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.
- In Stop-and-Wait ARQ, we use sequence numbers to number the frames. The sequence numbers are based on modulo-2 arithmetic.
- In Stop-and-Wait ARQ, the acknowledgment number always announces in modulo-2 arithmetic the sequence number of the next frame expected.

Somaiya TRUST

Stop And


S_n Next frame to send
0 1 0 1 0
R_n Next frame to receive
0 1 0 1 0
Sender
Network
Get data
Data link
Physical
Receive frame
Send frame
Data frame
seqNo
ACK frame
ackNo
Receiver
Deliver data
Network
Data link
Physical
Receive frame
Send frame
Event:
Request from network layer
Repeat forever
Algorithm for sender site
Time-out
Event:
Event:
Notification from physical layer
Repeat forever
Algorithm for receiver site
Event:
Notification from physical layer

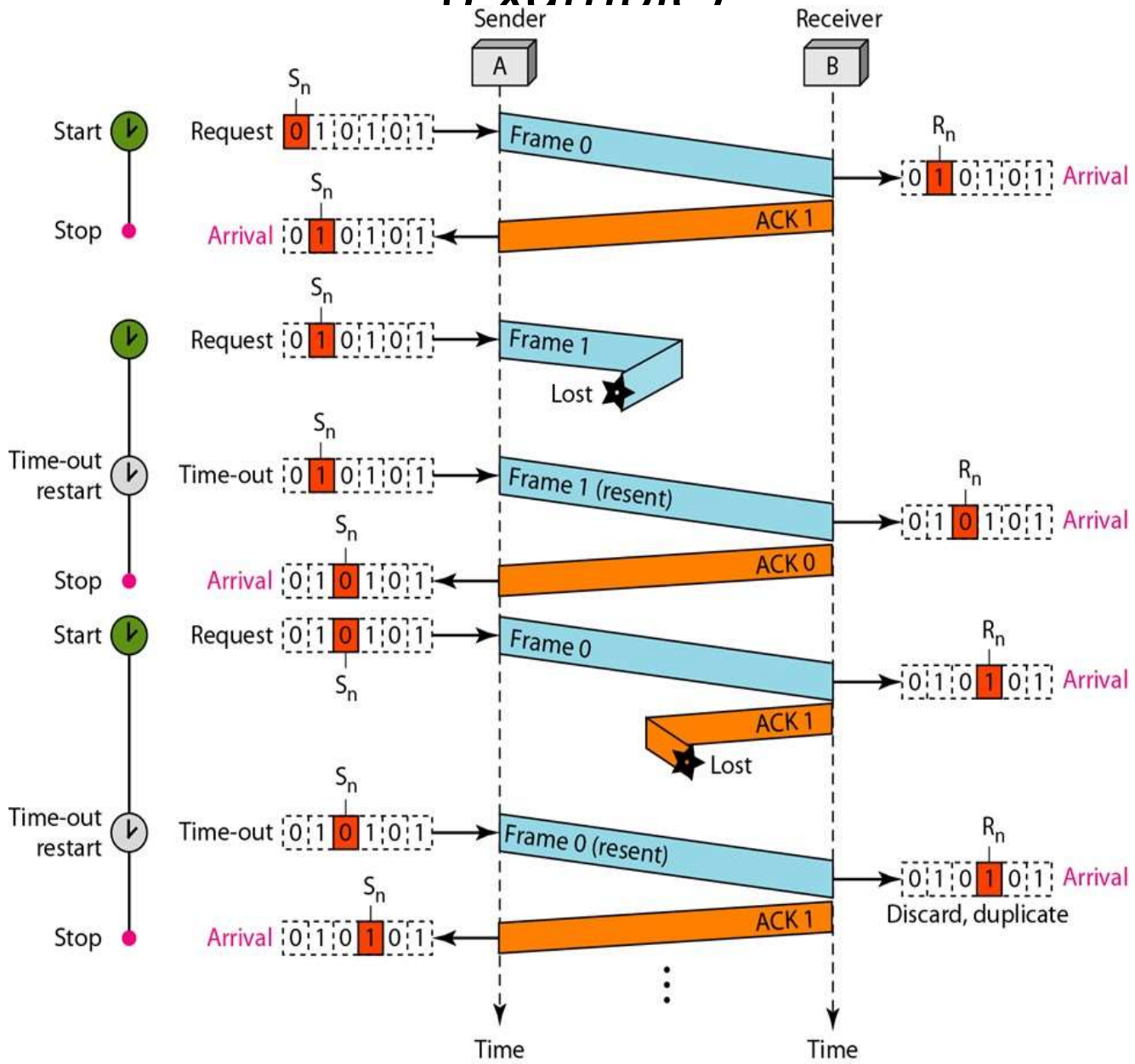
Stop And Wait ARQ
(Example)

***Figure** shows an example of Stop-and-Wait ARQ.*

- *Frame 0 is sent and acknowledged.*
- *Frame 1 is lost and resent after the time-out.*
- *The resent frame 1 is acknowledged and the timer stops.*
- *Frame 0 is sent and acknowledged, but the acknowledgment is lost. The sender has no idea if the frame or the acknowledgment is lost, so after the time-out, it resends frame 0, which is*

Stop And Wait ARQ

(Example)



Stop And Wait ARQ

Efficiency:

- *Very inefficient if channel is **thick** (large bandwidth)and **long** (round trip delay is long).*

- *The product of these two is called the **bandwidth-delay product** (volume of the pipe in bits).*

- *The **bandwidth-delay product** is a measure of the number of bits we can send out of our system while waiting for ACK from the receiver.*

Stop And Wait ARQ

Example 1:

Assume that, in a Stop-and-Wait ARQ system, the bandwidth of the line is 1 Mbps, and 1 bit takes 20 ms to make a round trip. What is the bandwidth-delay product? If the system data frames are 1000 bits in length, what is the utilization percentage of the link?

Solution:

The bandwidth-delay product is:

$$(1 \times 10^{6}) \times (20 \times 10^{-3}) = 20{,}000 \text{ bits}$$

Stop And Wait ARQ

The system can send 20,000 bits during the time it takes for the data to go from the sender to the receiver and then back again. However, the system sends only 1000 bits.

We can say that the ***link utilization*** *is only* ***1000/20,000****, or* ***5 percent****.*

For this reason, for a link with a high bandwidth or long delay, the use of Stop-and-Wait ARQ wastes the capacity of the link.

Stop And Wait ARQ

Pipelining:

- *No pipelining in Stop-And-Wait ARQ.*
- *We need to wait for a frame to reach the destination and be acknowledged before next frame can be sent.*
- *Pipelining is applicable to next two protocols, wherein several frames can be sent before receiving acknowledgement for previous frames.*
- *Pipelining improves efficiency.*

Somaiya TRUST

Go-Back-N ARQ

- To improve the efficiency of transmission (filling the pipe), multiple frames must be in transition while waiting for acknowledgment.

- This protocol can send several frames before receiving

  acknowledgments it keeps a copy of these frames until the acknowledgments arrive.

- In the Go-Back-N Protocol, the **sequence numbers** are modulo **2^m** i.e. the sequence numbers range from **0 to $2^m - 1$**, where m is the size of the sequence number

Go-Back-N ARQ

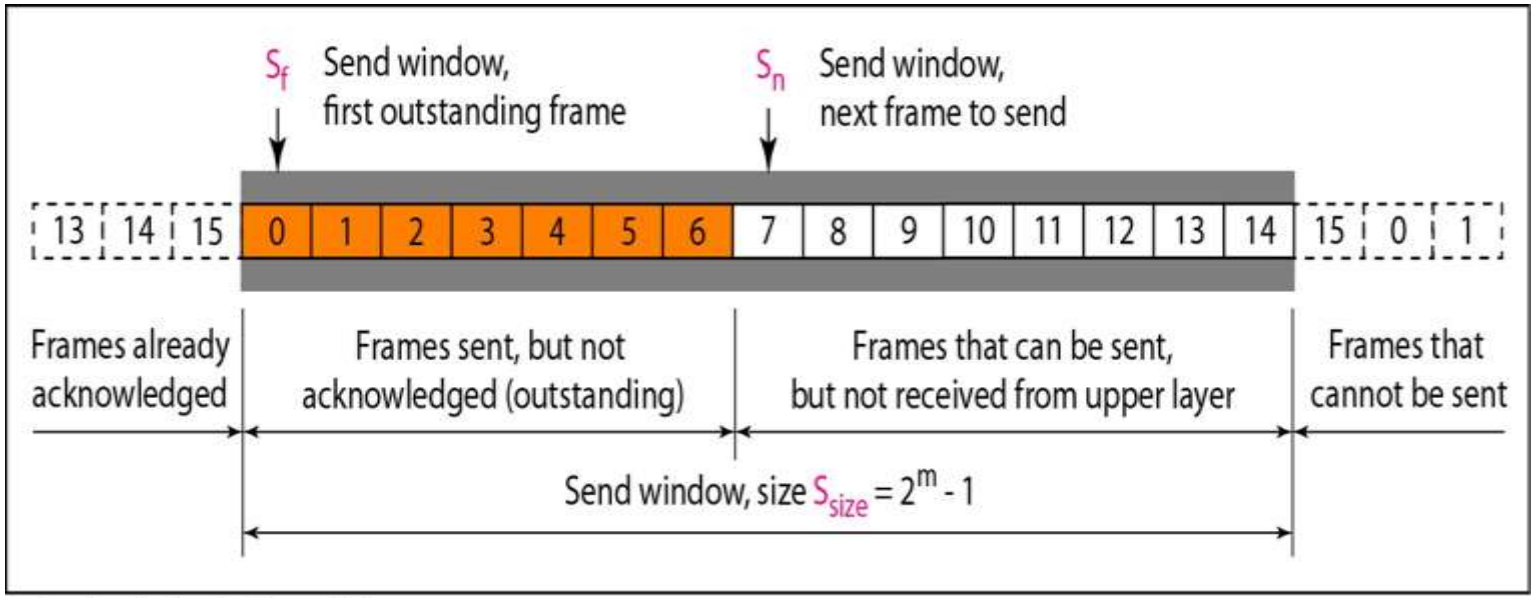
Sliding Window:

- Sliding window is an abstract concept that defines the range of sequence numbers.

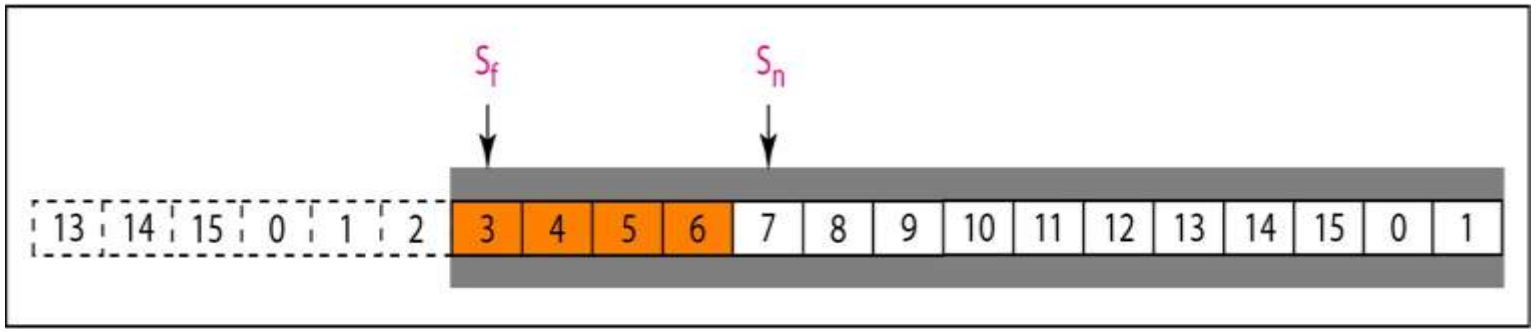
- The range which is the concern of the sender is called the send sliding window; the range that is the concern of the receiver is called the receive sliding window.

- The window at any time divides the possible sequence numbers into four regions.

Send Window for Go-Back-N ARQ:



a. Send window before sliding



b. Send window after sliding

Somaiya TRUST

Go-Back-N ARQ

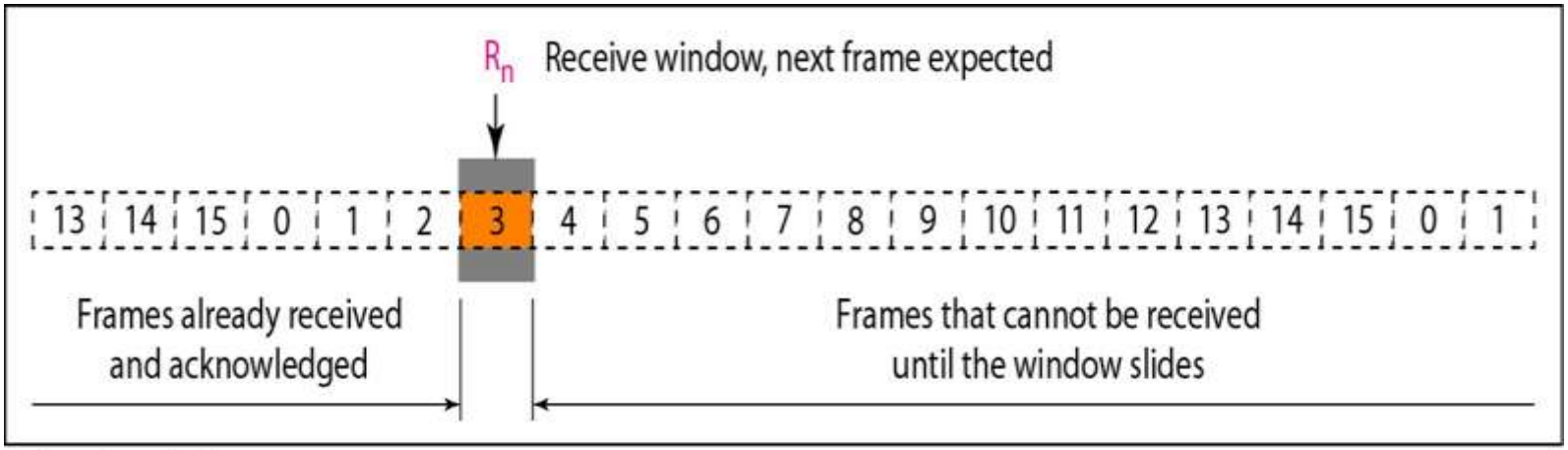
Send Window :

- Sliding window is an abstract concept that defines the range of sequence numbers.
- The send window can slide one or more slots when a valid acknowledgment arrives.

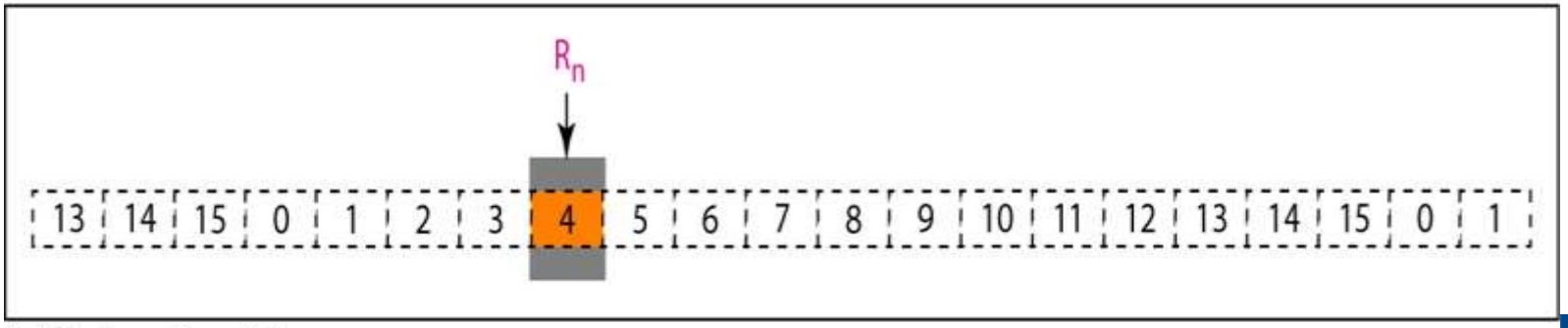
Receive Window :

- The receive window is an abstract concept defining an imaginary box of size 1 with one single variable Rn.
- The window slides when a correct frame has arrived; sliding occurs one slot at a time.

Receive Window for Go-Back-N ARQ:

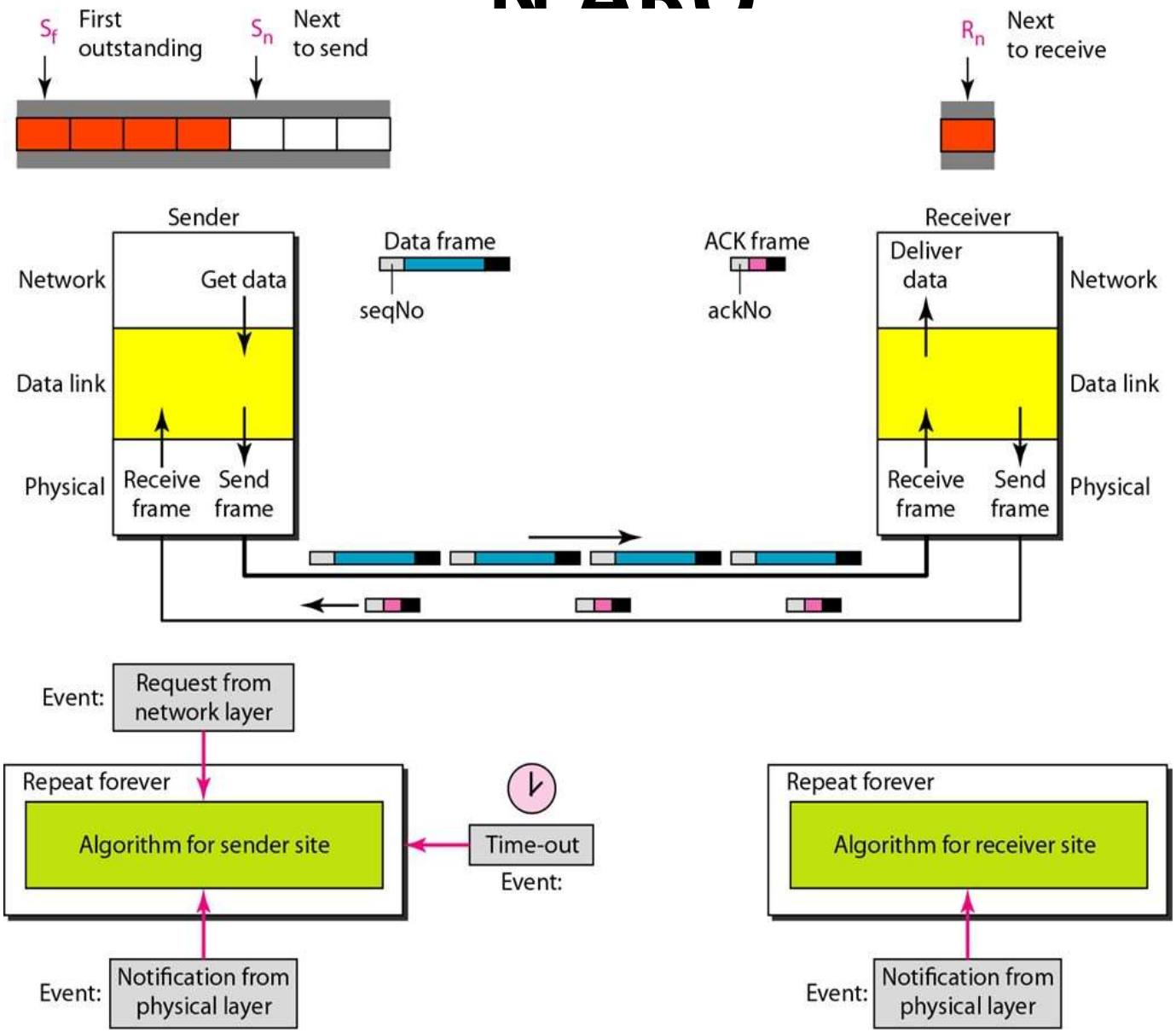


a. Receive window



b. Window after sliding

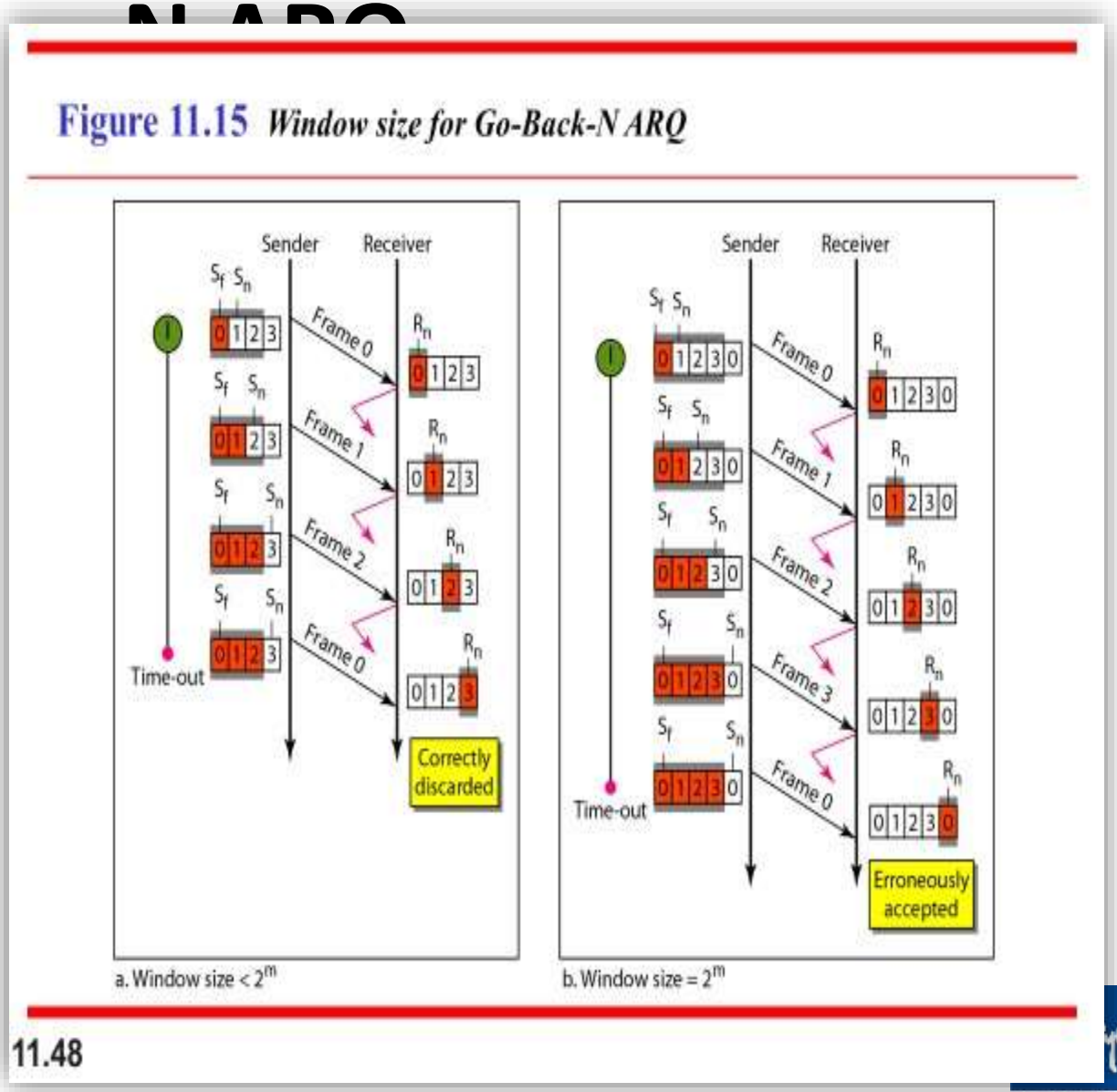
Go-Back-N ARQ


S_f First outstanding
S_n Next to send
R_n Next to receive
Sender
Receiver
Network
Get data
Data link
Physical
Receive frame
Send frame
Data frame
seqNo
ACK frame
ackNo
Deliver data
Network
Data link
Physical
Receive frame
Send frame
Event:
Request from network layer
Repeat forever
Algorithm for sender site
Time-out
Event:
Event:
Notification from physical layer
Repeat forever
Algorithm for receiver site
Event:
Notification from physical layer

Go-Back-N ARQ

NOTE:

- *In Go-Back-N ARQ, the size of the send window must be less than 2m;*
- *the size of the*



Go-Back-N ARQ

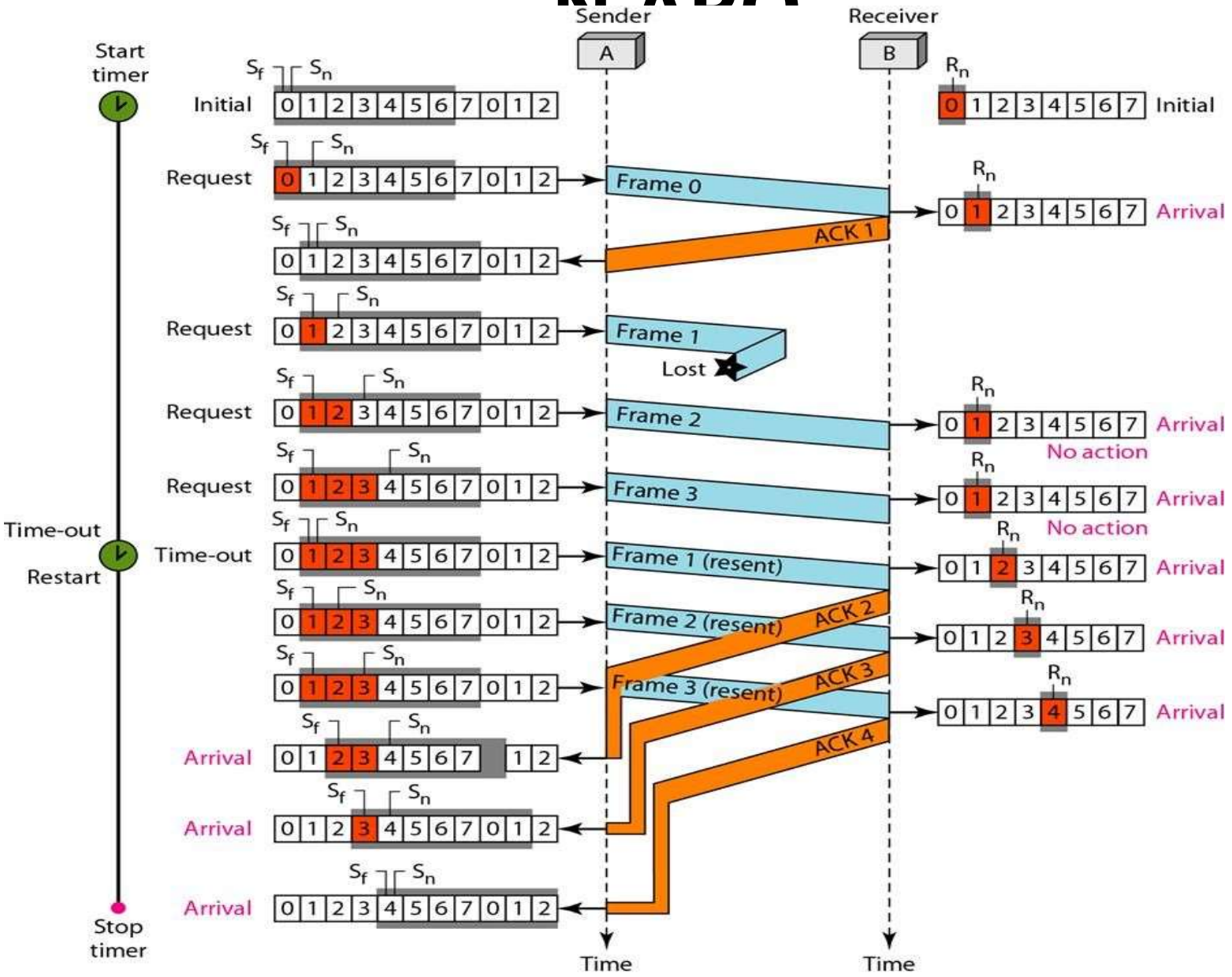
Example:

- *The physical layer must wait until this event is completed and the data link layer goes back to its sleeping state.*

- *We have shown a vertical line to indicate the delay. It is the same story with ACK 3; but when ACK 3 arrives, the sender is busy responding to ACK 2. It happens again when ACK 4 arrives.*

- *Note that before the second timer expires, all outstanding frames have been sent and the timer is stopped.*

Go-Back-N ARQ


Sender
Receiver
A
B
Start timer
Stop timer
Time-out
Restart
Initial
Request
Frame 0
ACK 1
Frame 1
Lost
Frame 2
Frame 3
Time-out
Frame 1 (resent)
Frame 2 (resent)
ACK 2
Frame 3 (resent)
ACK 3
ACK 4
Arrival
No action
Time
Time

Selective Repeat ARQ

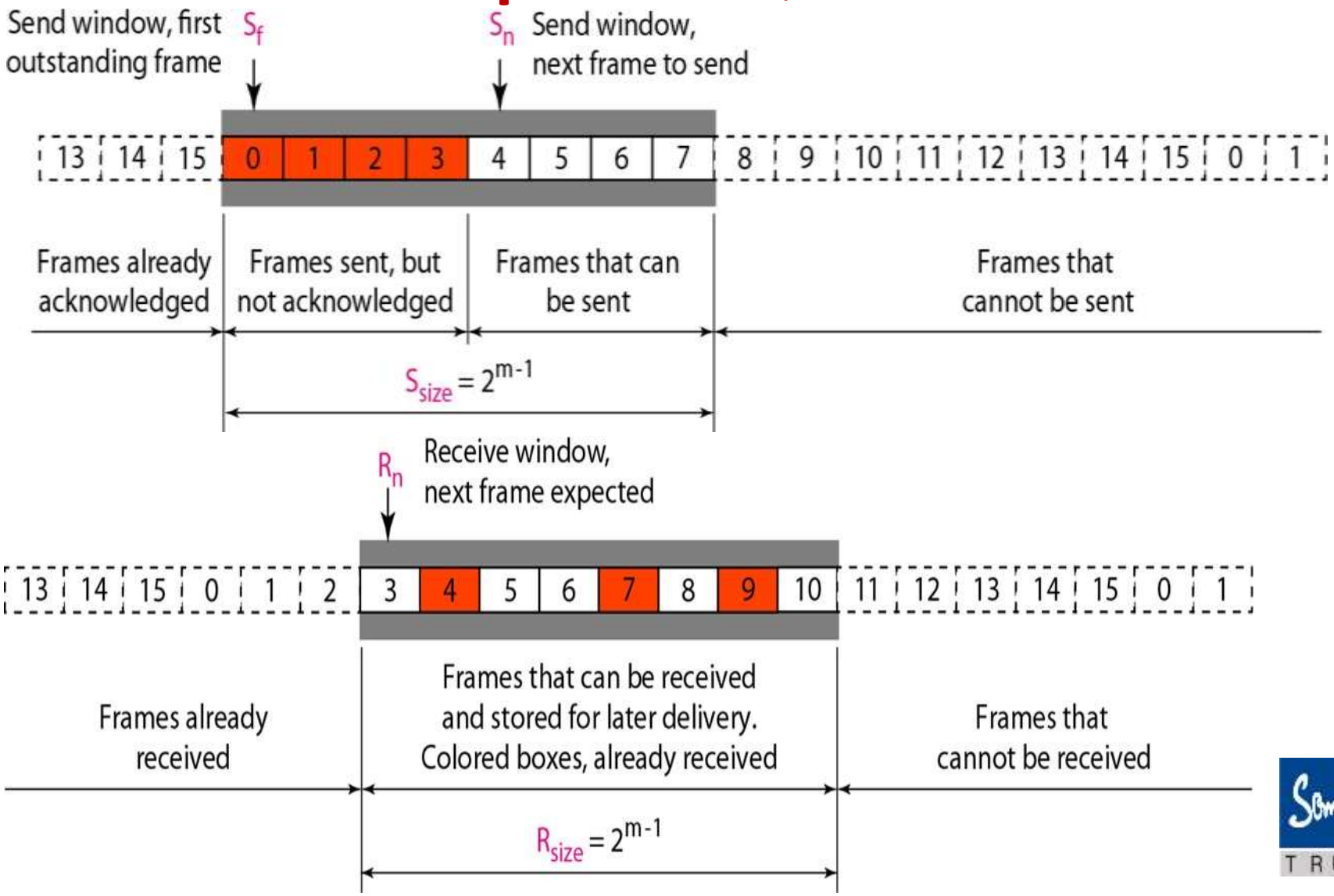
- Go-Back-N ARQ simplifies the process at the receiver site.
- The receiver keeps track of only one variable, and there is no need to buffer out-of-order frames; they are simply discarded.
-
- However, this protocol is very inefficient for a noisy link.
- In a noisy link a frame has a higher probability of damage, which means the resending of multiple frames. This resending uses up the

Selective Repeat ARQ

- Mechanism that does not resend N frames when just one frame is damaged; only the damaged frame is resent.
- More efficient for noisy links, but the processing at the receiver is more complex.
- More efficient for noisy links, but the processing at the receiver is more complex.
- the size of the **send window** is much smaller; it is 2^{m-1}.

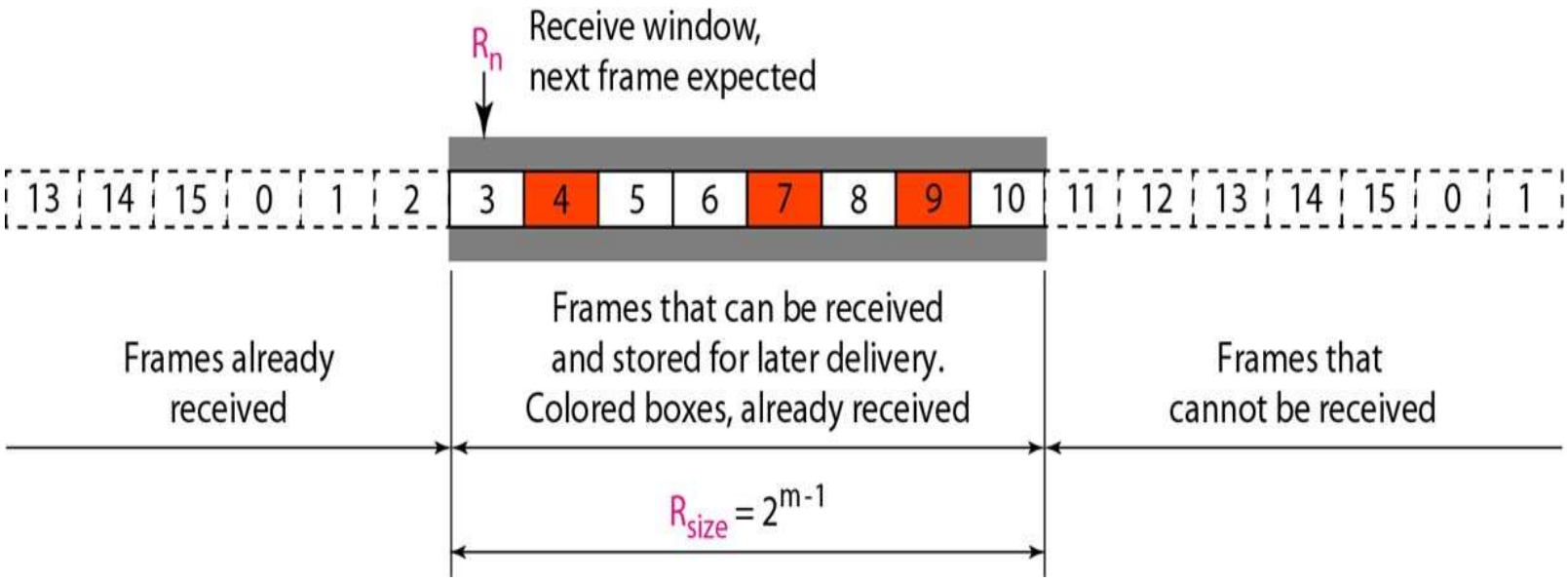
Selective Repeat ARQ

Send window for Selective Repeat ARQ

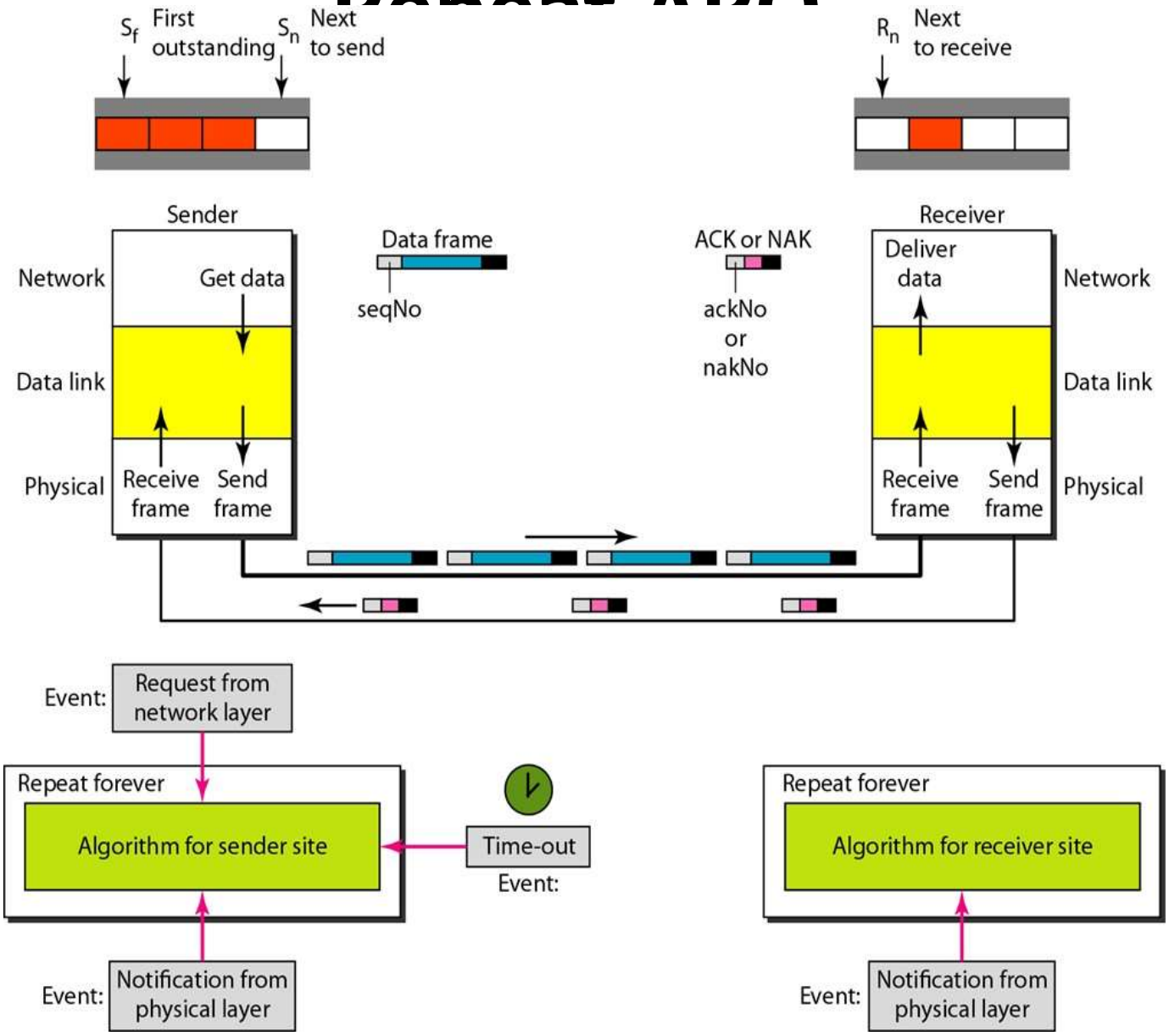


Selective Repeat ARQ

Receive window for Selective Repeat ARQ:

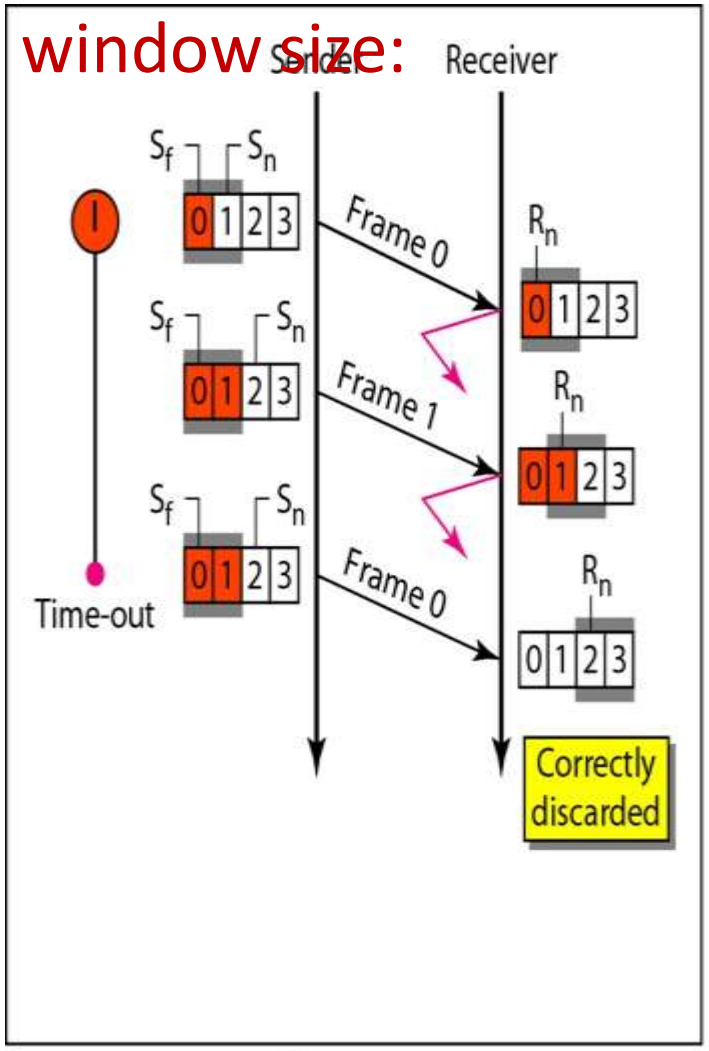


Selective Repeat ARQ

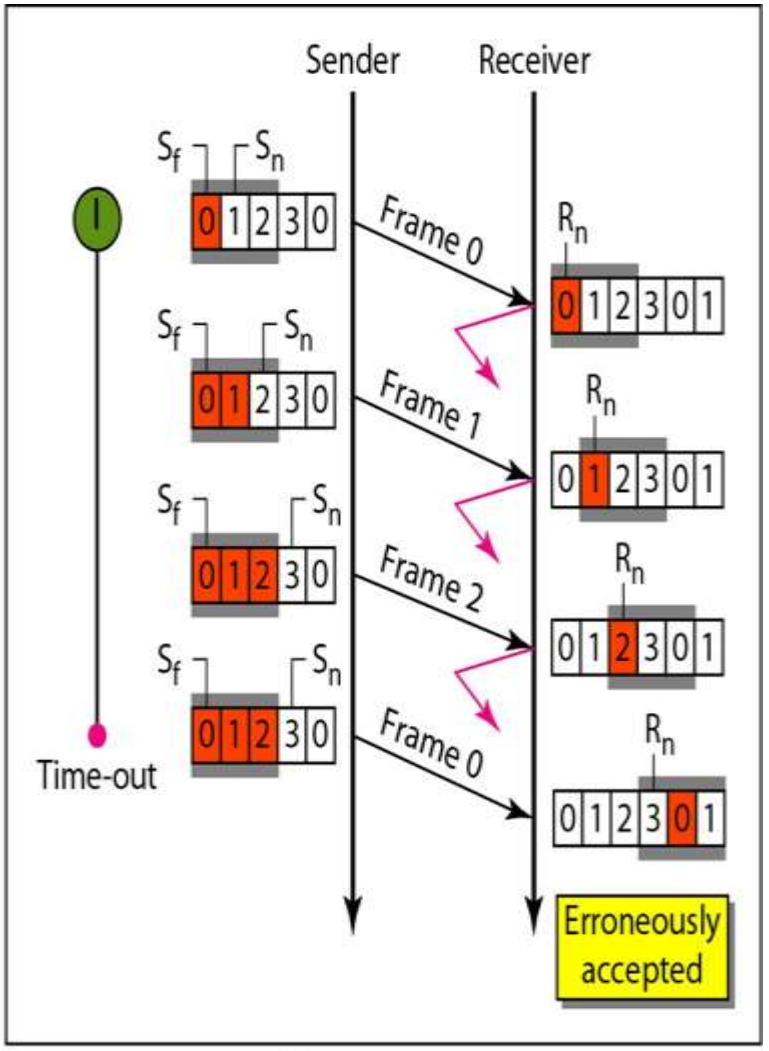


Selective Repeat ARQ

Selective Repeat ARQ, window size:



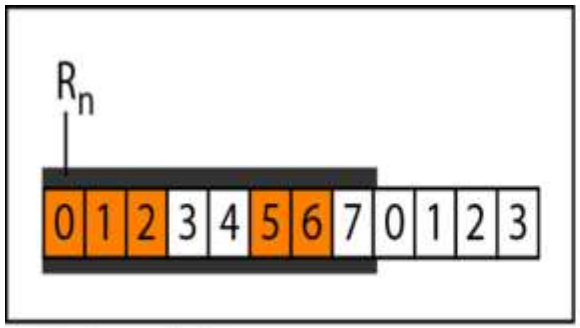
a. Window size = 2^{m-1}



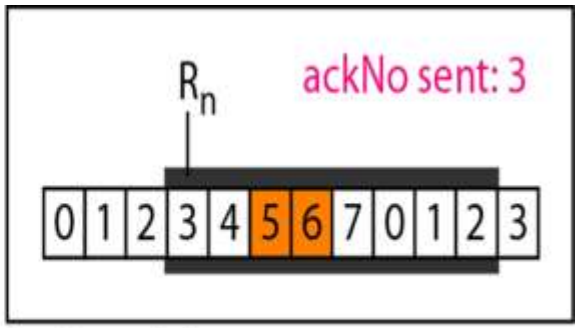
b. Window size > 2^{m-1}

Selective Repeat ARQ

Delivery of data in Selective Repeat ARQ:



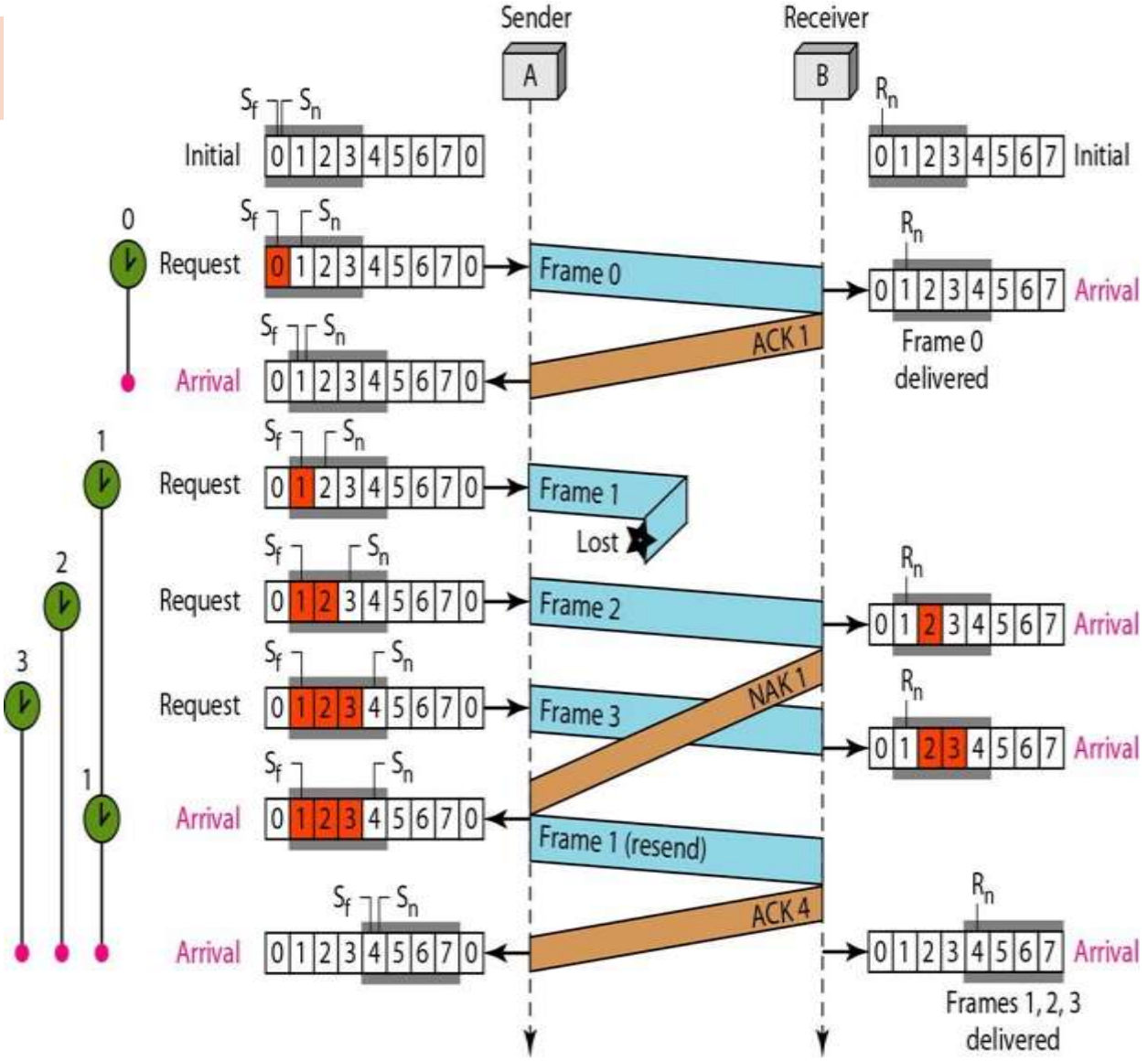
a. Before delivery



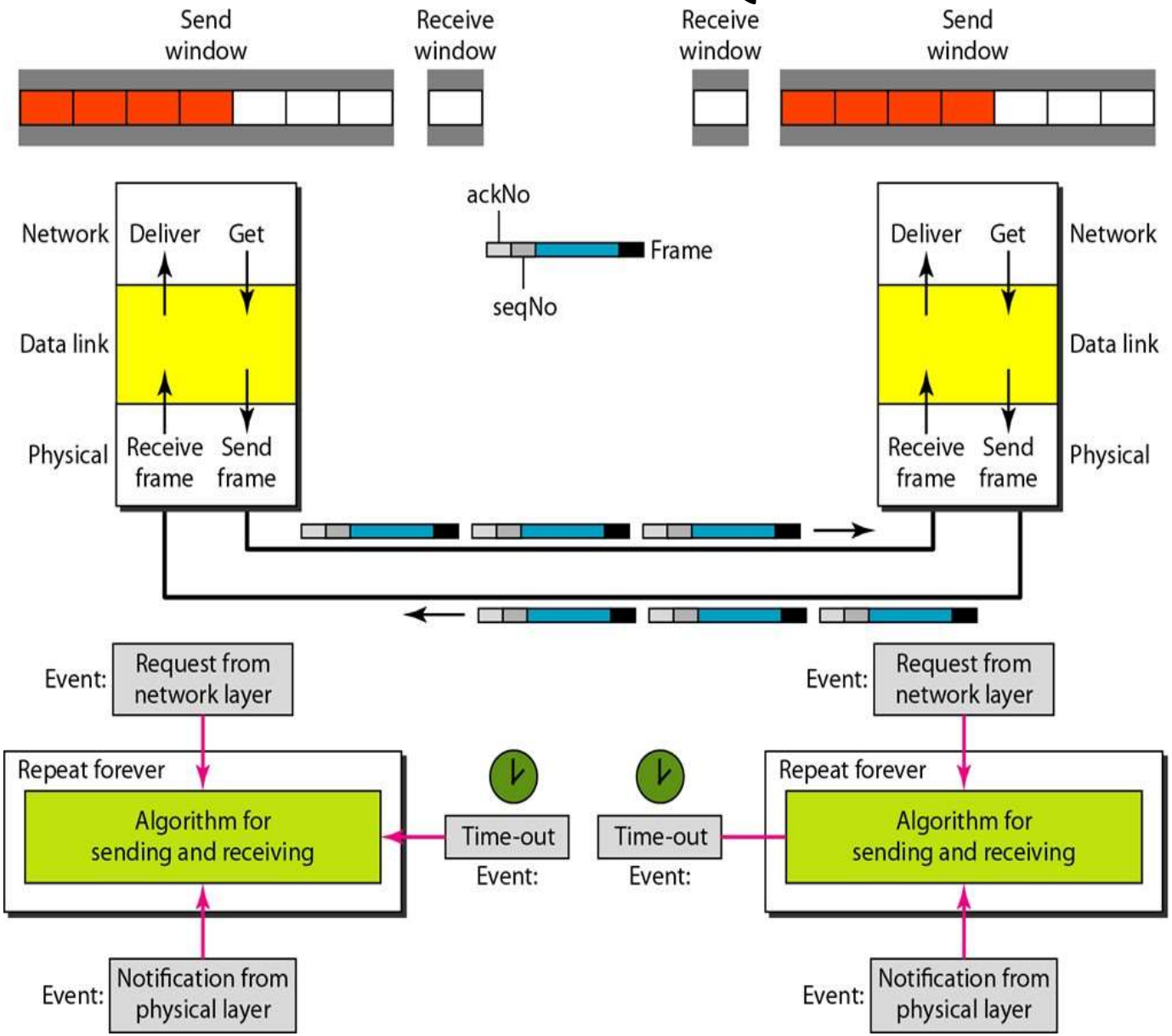
b. After delivery

Selective Repeat ARQ

Flow Diagram:

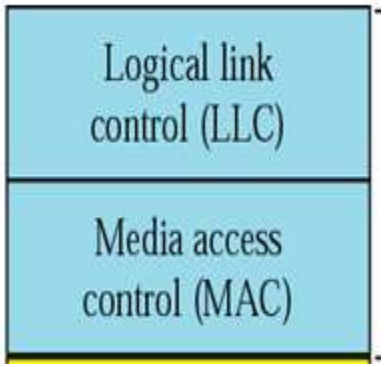


Piggybacking in Go Back N ARQ



MULTIPLE ACCESS Protocols:-

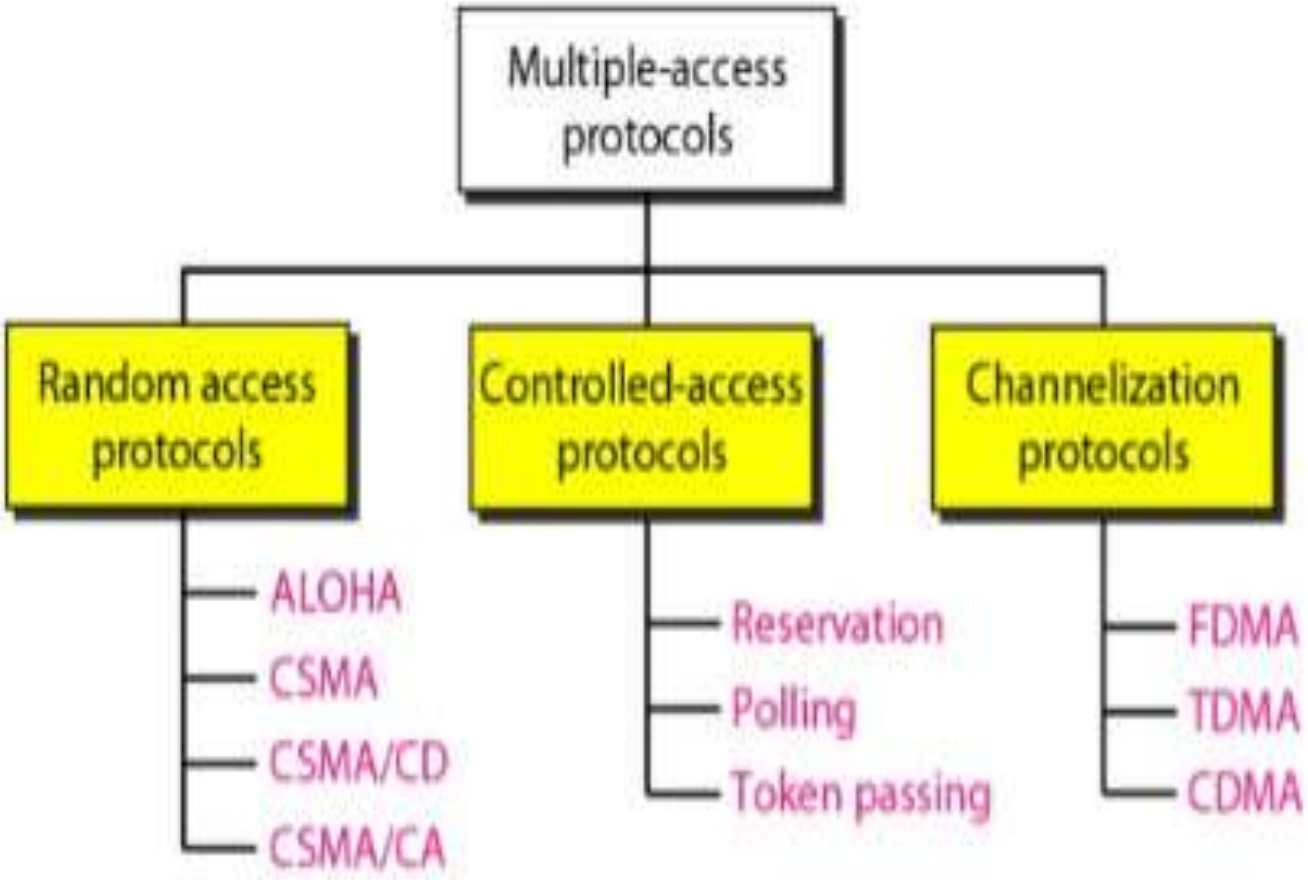
- DLL has two sub layers.



Upper Sub Layer (LLC)- it is responsible for Data Link Control.
Lower Sub Layer (MAC)- it is responsible for resolving Access to shared media. (if the channel is dedicated; we do not need lower sub-Layer).

- Responsibility of Data Link Control → for Flow and Error control. (LLC).
- Responsibility of Access resolution → Multiple Access resolution. (called MAC layer).

- Nodes or stations are connected, and use a common link, called a “ **Multi-point** “ or “ **Broadcast Link** “.
- So we need multiple access protocols to co-ordinate access to the link.
- Controlling the access to the medium is analogous to the rules of speaking in an assembly:
  - Everyone has right to speak
  - two people cannot speak at same time
  - do not interrupt each other
  - do not monopolize the discussion
- Many protocols are used to handle the access to the shared medium/link .All these protocols belong to the MAC sublayer in DLL.


Multiple-access protocols
Random access protocols
Controlled-access protocols
Channelization protocols
ALOHA
CSMA
CSMA/CD
CSMA/CA
Reservation
Polling
Token passing
FDMA
TDMA
CDMA

CHANNELIZATION

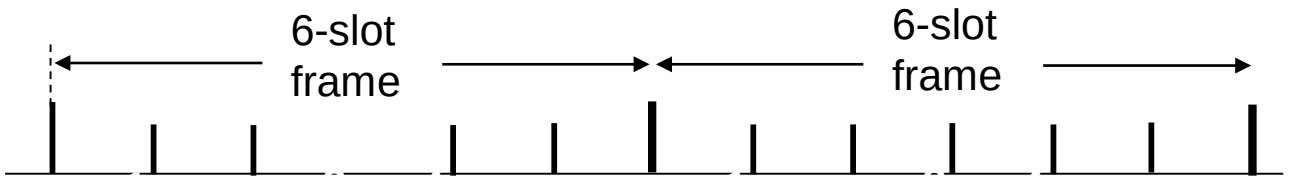
Channelization allows the total usable bandwidth in a shared channel to be shared across multiple stations based on their time, distance and codes. It can access all the stations at the same time to send the data frames to the channel.

Frequency-Division Multiple Access (FDMA)
Time-Division Multiple Access (TDMA)
Code-Division Multiple Access (CDMA)

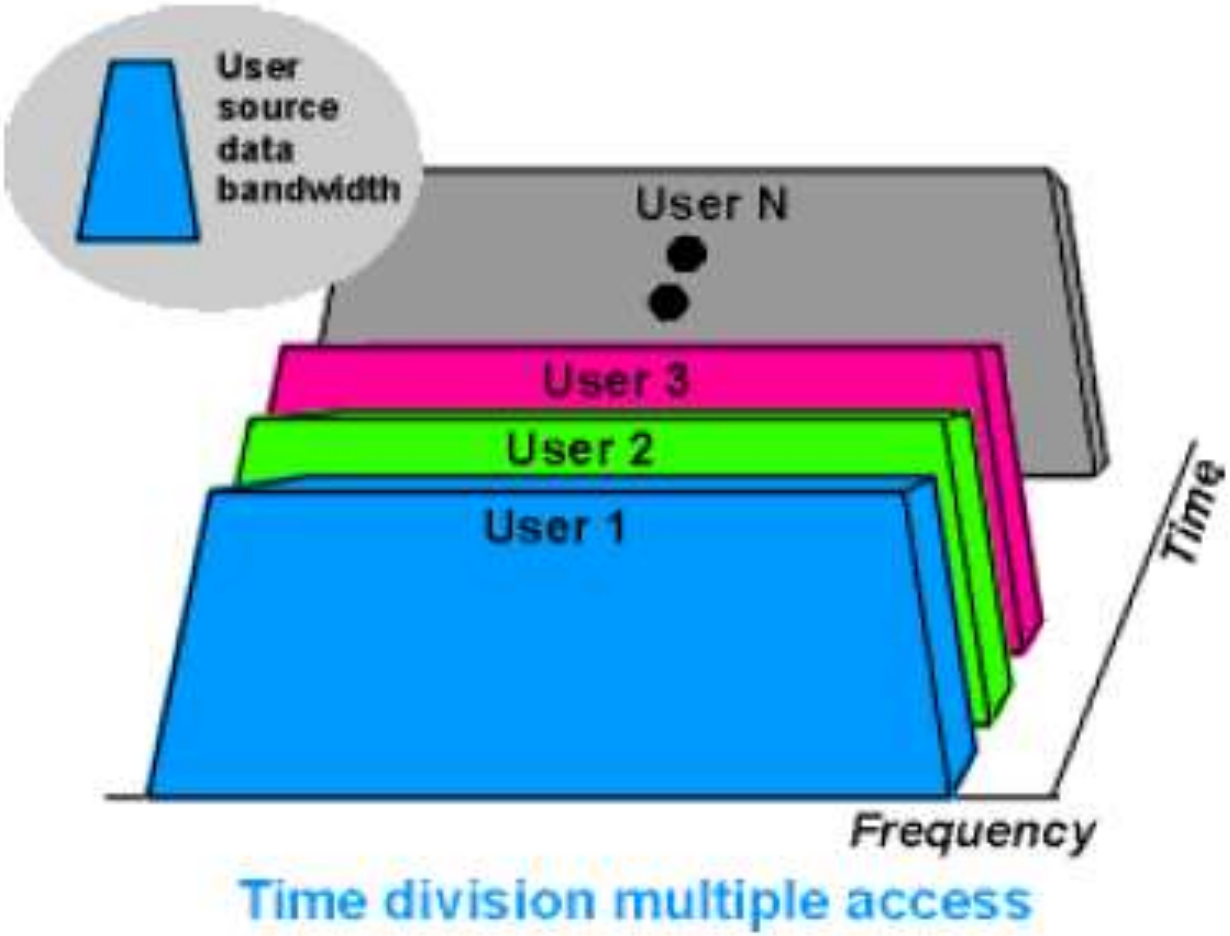
Channel partitioning MAC protocols: TDMA

TDMA: time division multiple access

- access to channel in "rounds"
- each station gets fixed length slot (length = pkt trans time) in each round
- unused slots go idle
- example: 6-station LAN, 1,3,4 have pkt, slots 2,5,6 idle

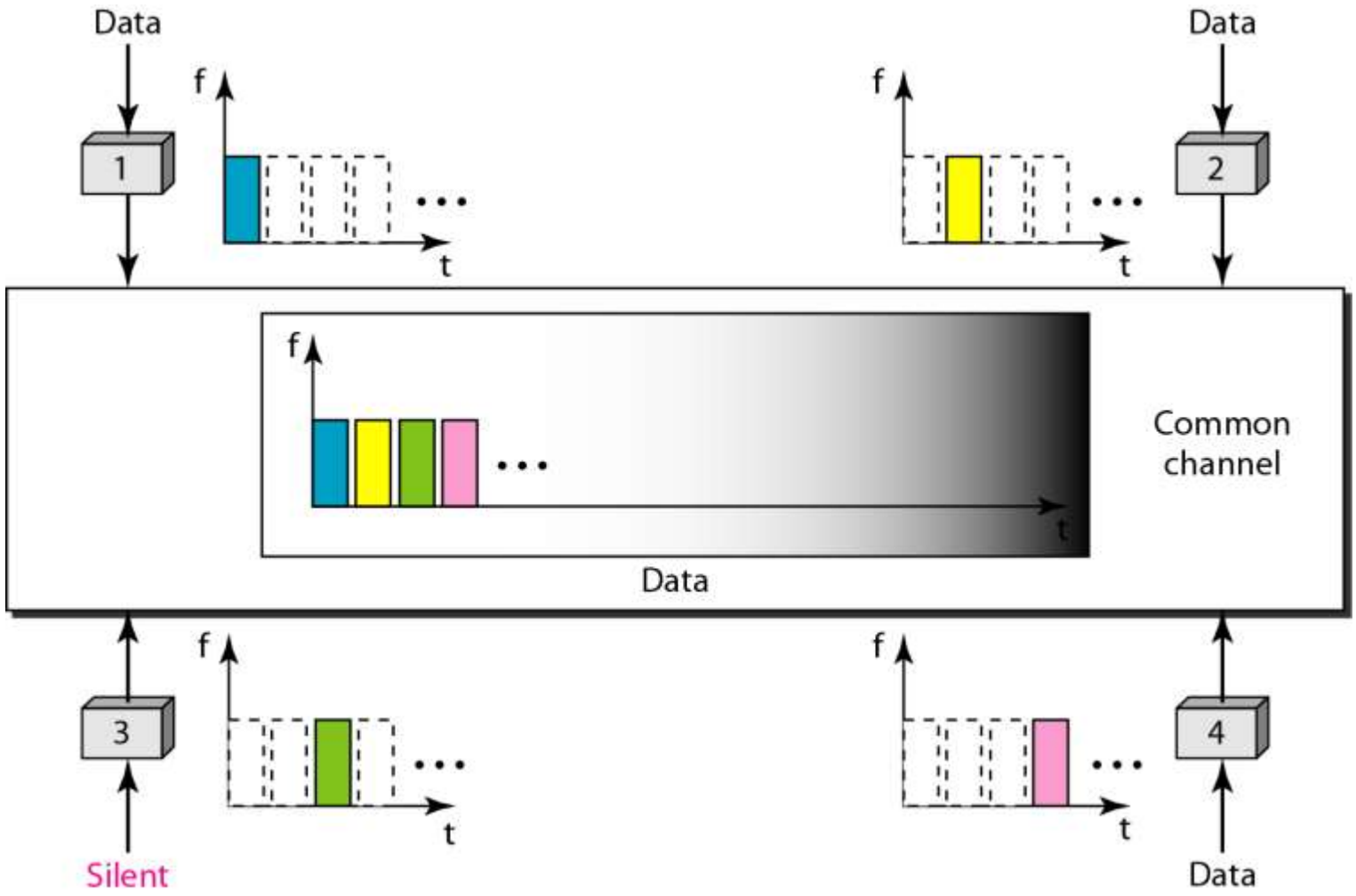


Time Division Multiple Access (TDMA)

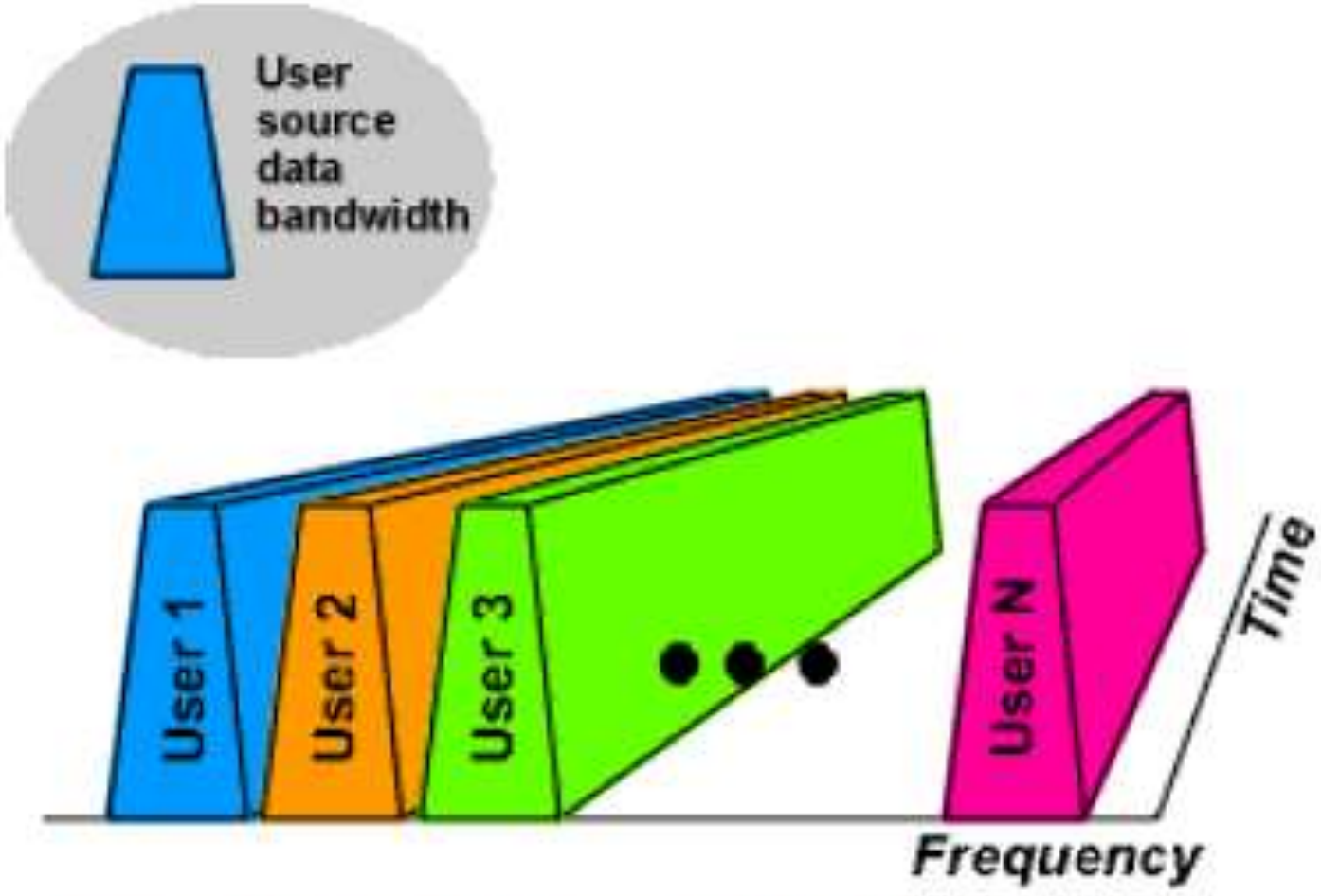


Time division multiple access

Time-division multiple access (TDMA)

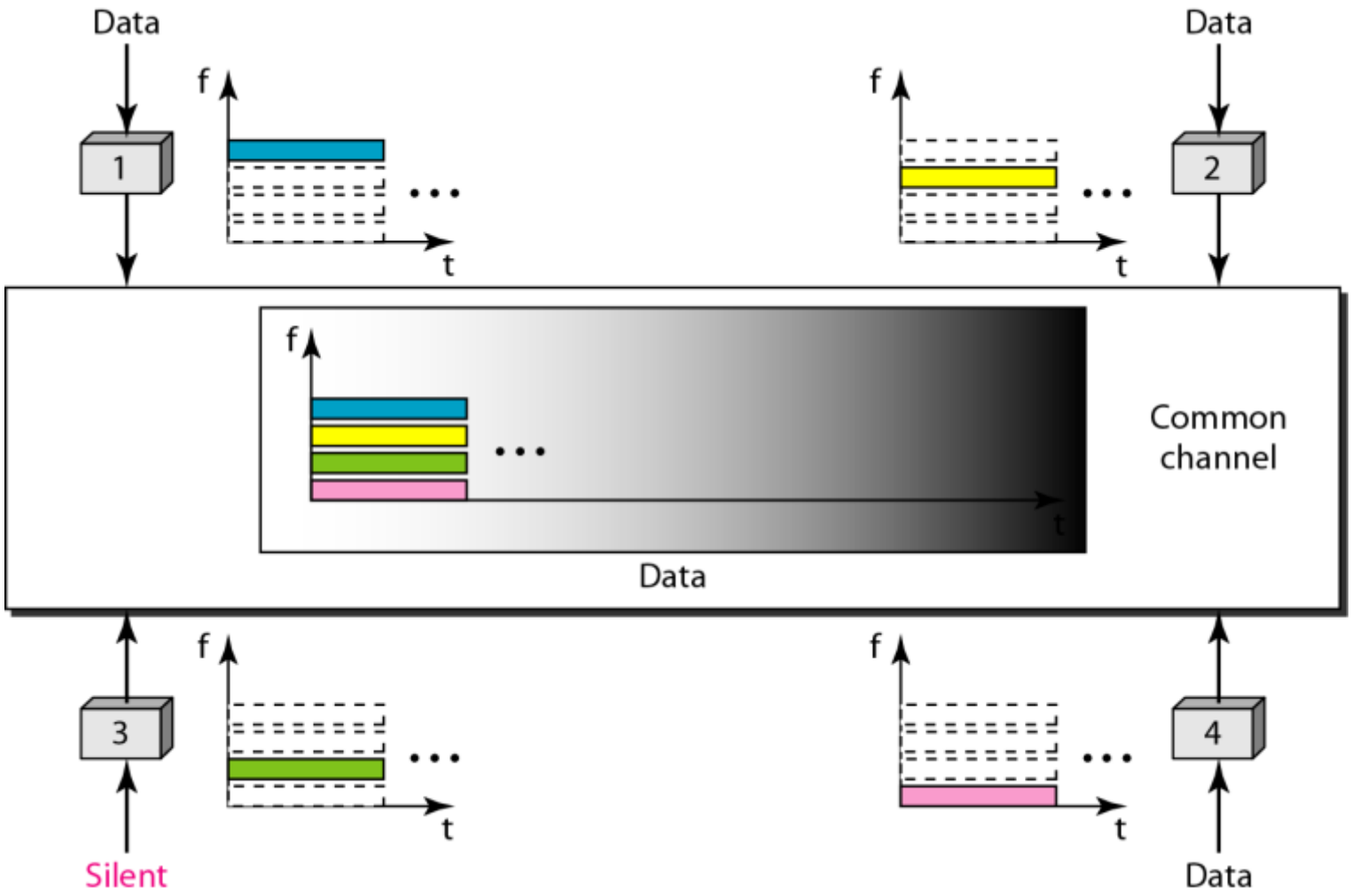


Frequency Division Multiple Access (FDMA)

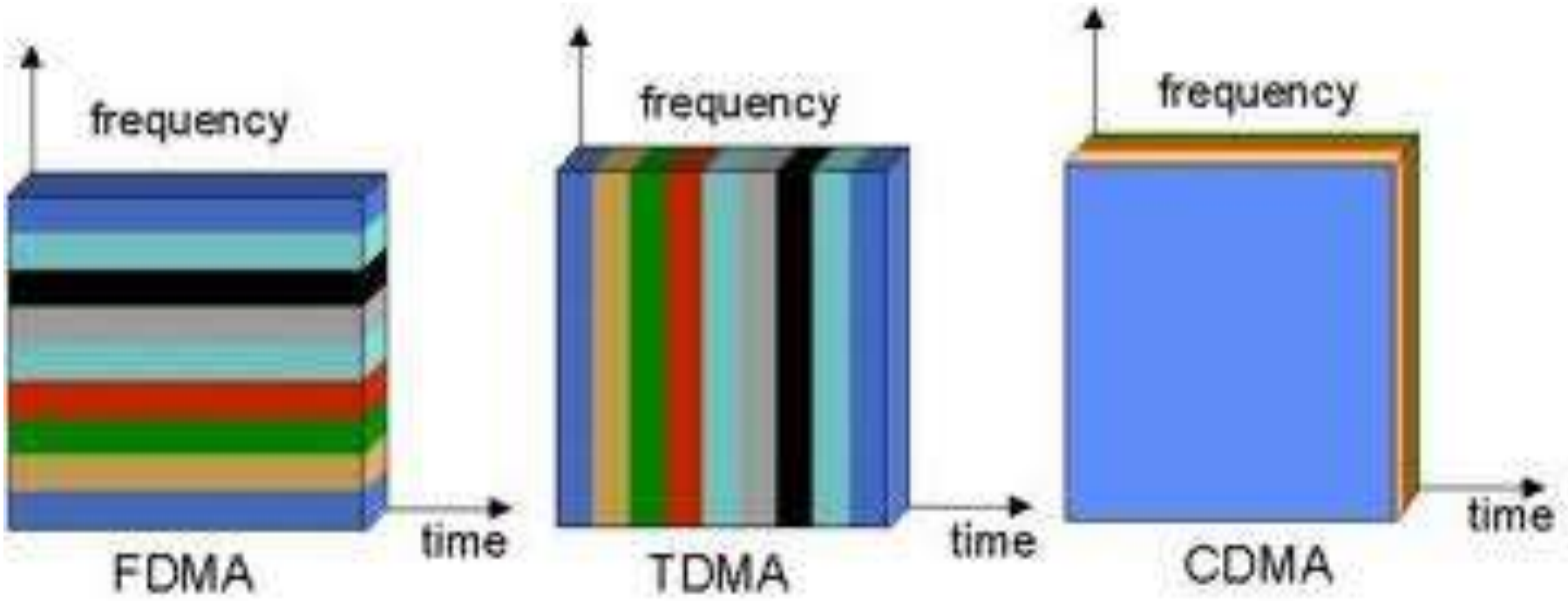


Frequency division multiple access

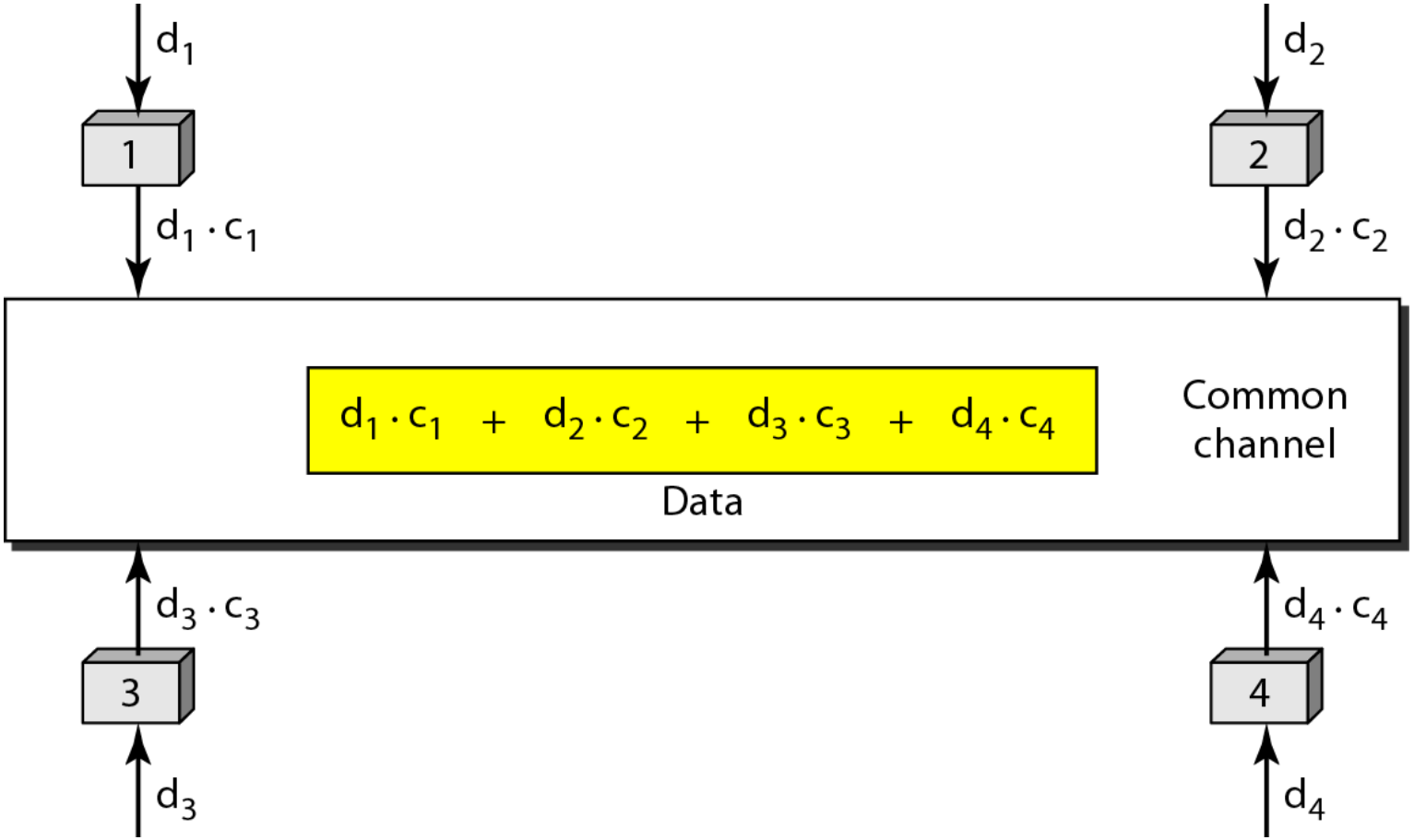
Frequency-division multiple access (FDMA)



Code Division Multiple Access(CDMA)



Simple idea of communication with code



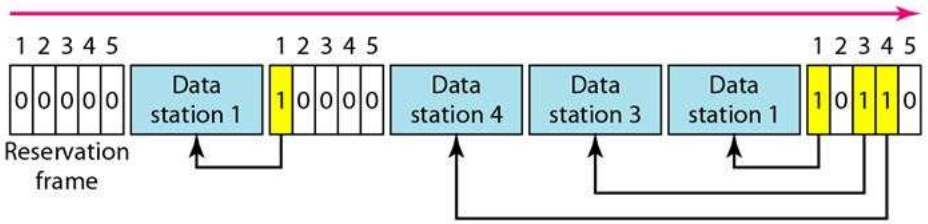
Controlled Access Protocols:

In Controlled access , the stations consult one another to find which station has the right to send . A station can not send unless it has been authorized by other stations.

The three Controlled Access protocols are :

Reservation , Polling and Token Passing

1. **Reservation**



In the reservation method, a station needs to make a reservation before sending data.

The time line has two kinds of periods:

i)Reservation interval of fixed time length

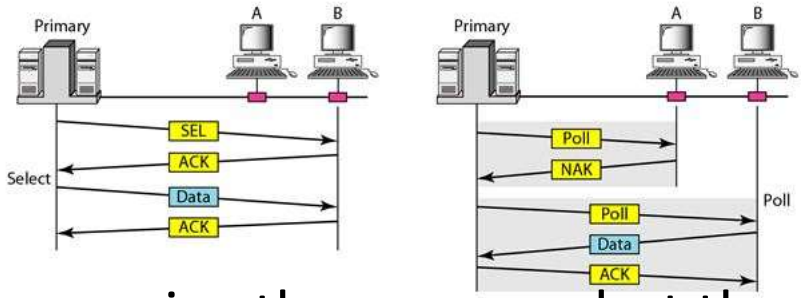
ii)Data transmission period of variable frames.

Controlled Access Protocols ..cntd..

- If there are M stations, the reservation interval is divided into M slots, and each station has one slot.
- Suppose if station 1 has a frame to send, it transmits 1 bit during the slot 1. No other station is allowed to transmit during this slot.
- In general, i th station may announce that it has a frame to send by inserting a 1 bit into i th slot. After all N slots have been checked, each station knows which stations wish to transmit.
- The stations which have reserved their slots transfer their frames in that order.
- After data transmission period, next reservation interval begins.
- Since everyone agrees on who goes next, there will never be any collisions.

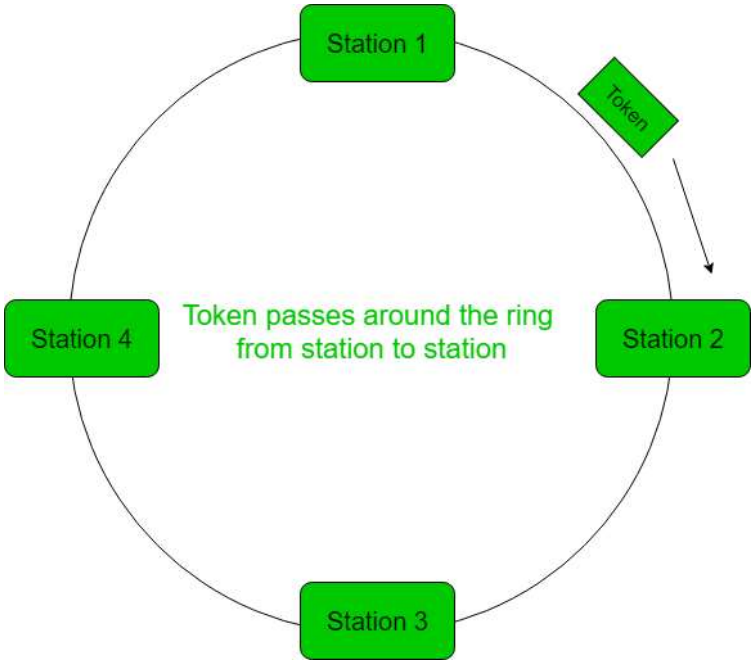
Polling

In this, one station acts as a primary station(controller) and the others are secondary stations.
All data exchanges must be made through the controller.
The message sent by the controller contains the address of the node being selected for granting access.



Although all nodes receive the message but the addressed one responds to it and sends data, if any. If there is no data, usually a “poll reject”(NAK) message is sent back.
Problems include high overhead of the polling messages and high dependence on the reliability of the controller.

Token Passing



- A token is passed from one station to another adjacent station in the ring .
- A token represents permission to send. If a station has a frame queued for transmission when it receives the token, it can send that frame before it passes the token to the next station.
- If it has no queued frame, it passes the token simply.
- After sending a frame, each station must wait for all N stations (including itself) to send the token to their neighbours and the other N – 1 stations to send a frame, if they have one.

Random Access MAC Protocols

- ALOHA
- CSMA
- CSMA/CD
- CSMA/CA

- In Random Access techniques , No station is superior to another station and none is assigned the control over another.
- At each instance, a station that has to send data, uses the procedures defined by protocol, and makes a decision to send or not to send the data.
- This decision depends on the state of the medium. (idle / busy)

  Two features gives this method its name.

  1. There is no scheduled time, for a station to transmit. Transmission is random among the station. (i.e. these methods called Random Access).
  2. No rule specify which station should send next.

  Stations compete with one another to access the medium. (i.e. why they are also called, " Contention methods ".

- If more than one station try to send; there arises an access conflict – collision. (Frames will be destroyed or changed / modified).
- In Random Access protocols , Following questions needs to be addressed

- When can station Access the medium ?
- What station can do; if medium is busy ?
- How stations come to know about successful transfer or collision ?
- What station should do in case of Collision ?

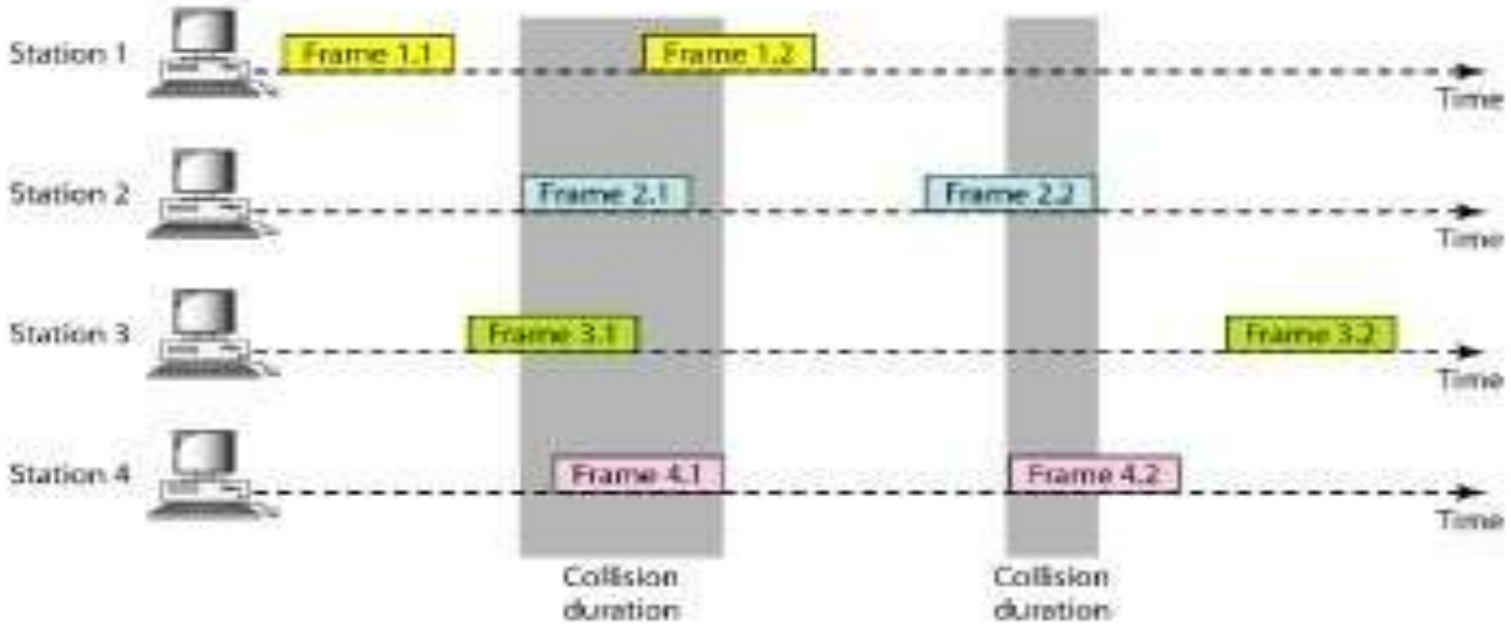
ALOHA :-

- It was the earliest Random Access method, developed at University of Hawaii, in early 1970.
- It was designed for Radio (wireless) LAN, but can be used on any shared medium.
- The medium is shared between stations. When station sends data, the other stations may attempt to send data. In this case it collides and the data frame gets damaged.

  (in the diagram ,Shaded regions shows collision)
- *Original ALOHA protocol is called PURE ALOHA.*

(*ALOHA originally stood for* ***Additive Links On-line Hawaii Area***. Also ALOHA is an Hawaiian word, meaning "hello")

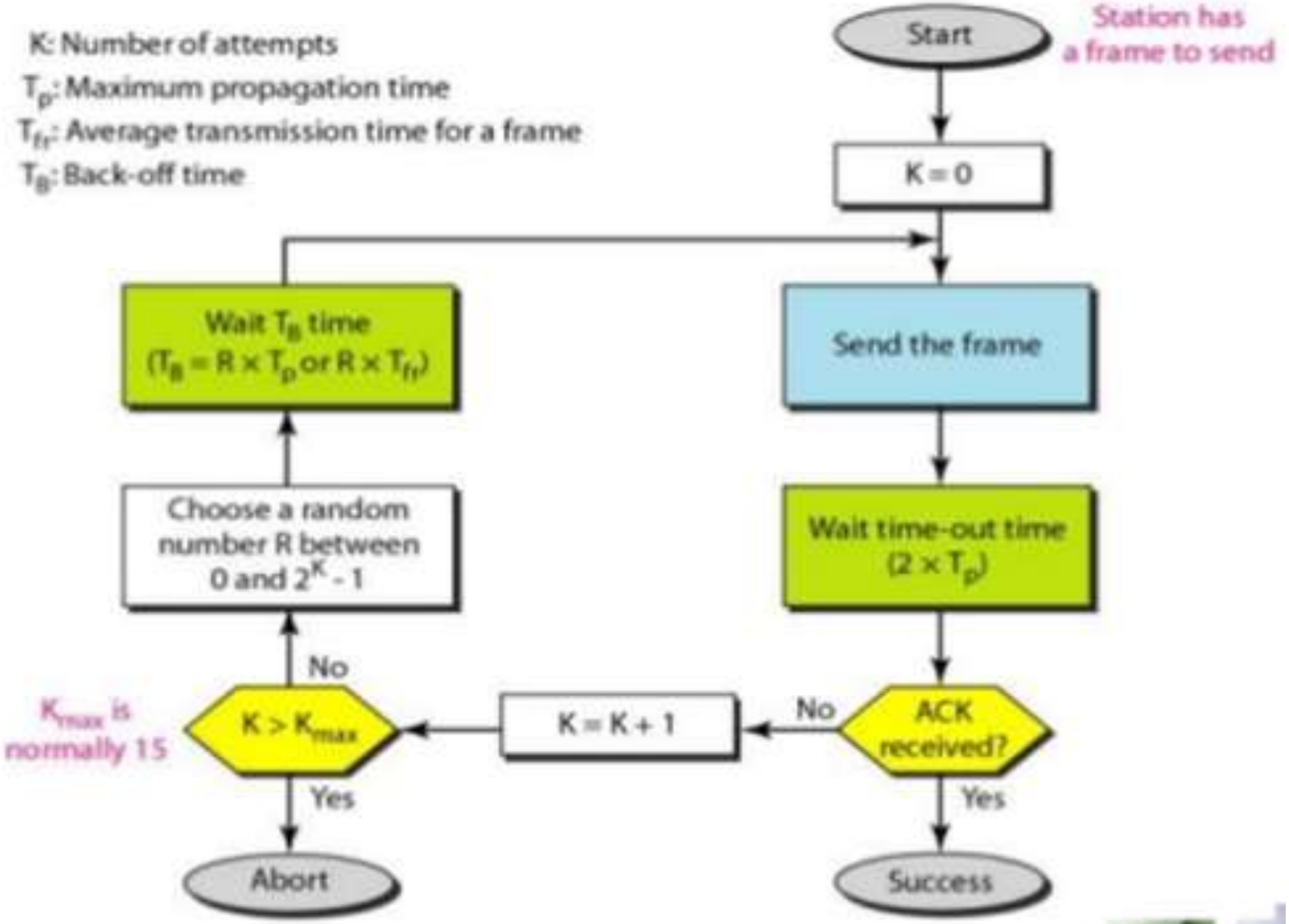
Figure 12.3 *Frames in a pure ALOHA network*



- When there is collision, the frames has to be resent.
- Pure ALOHA, relies on **Acknowledgement** sent from the Receiver.
- When a station sends a frame, it expects the receiver to send an acknowledgement.
- If acknowledgement does not arrive within time-out period, the station assumes that the frames has been destroyed; and so it resends the frame.
- A collision occurs when two or more stations attempt to send simultaneously.
- If all stations try to resend their frames after the timeout, the frames will collide again.

-Pure ALOHA dictates that, when the time out period passes, each station **waits for a random amount of time**, before resending its frame. This randomness will avoid more collisions. This time is called " ***back-off time*** " (T_B). After a number of retransmission attempts $\mathbf{k_{max}}$, a station must give up, and try later.

Procedure for pure ALOHA protocol

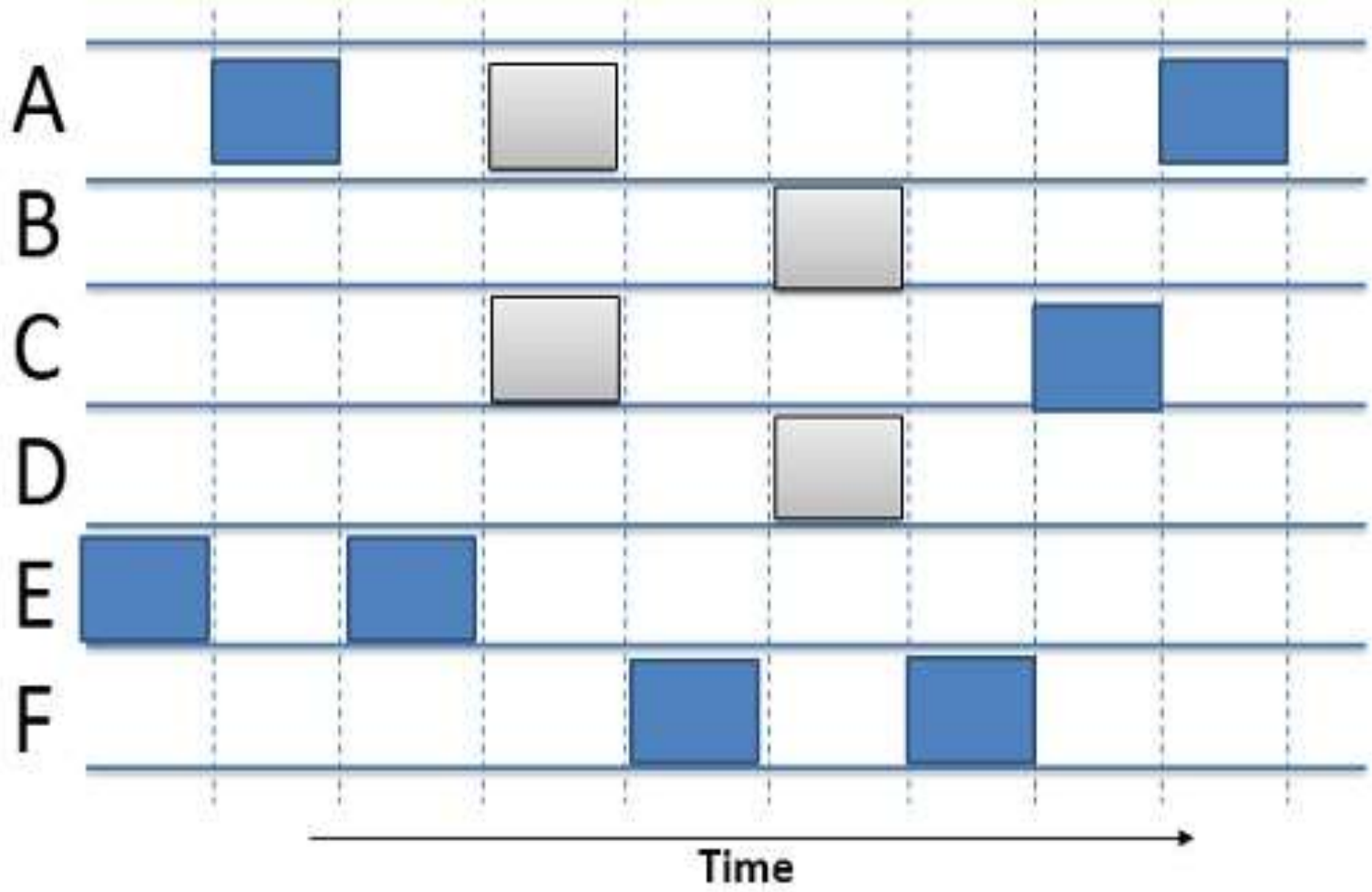


SLOTTED ALOHA :--

- To improve the efficiency of PURE ALOHA, the time is divided into slots of T_f, (average transmission time for a slot).
- A station is allowed to send only at the beginning of the synchronized time slot.
- If a station miss this moment, it must wait until the beginning of the next time slot.
- The station which has started at the beginning of the slot, has already finished sending its frame.

Of course; there is still collision; if two stations try to send at the beginning of the same time slot.

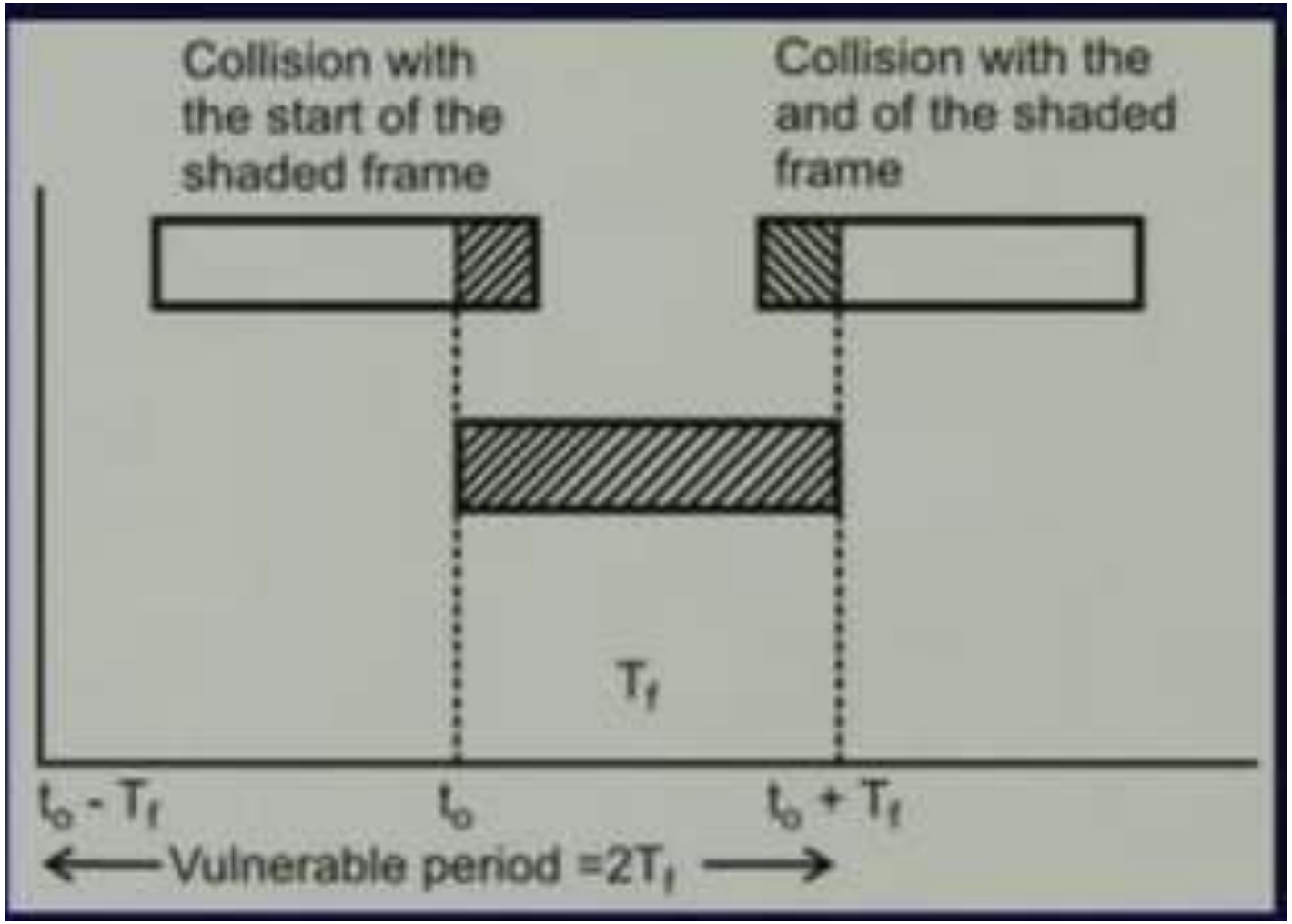
(Shaded region shows collosion)



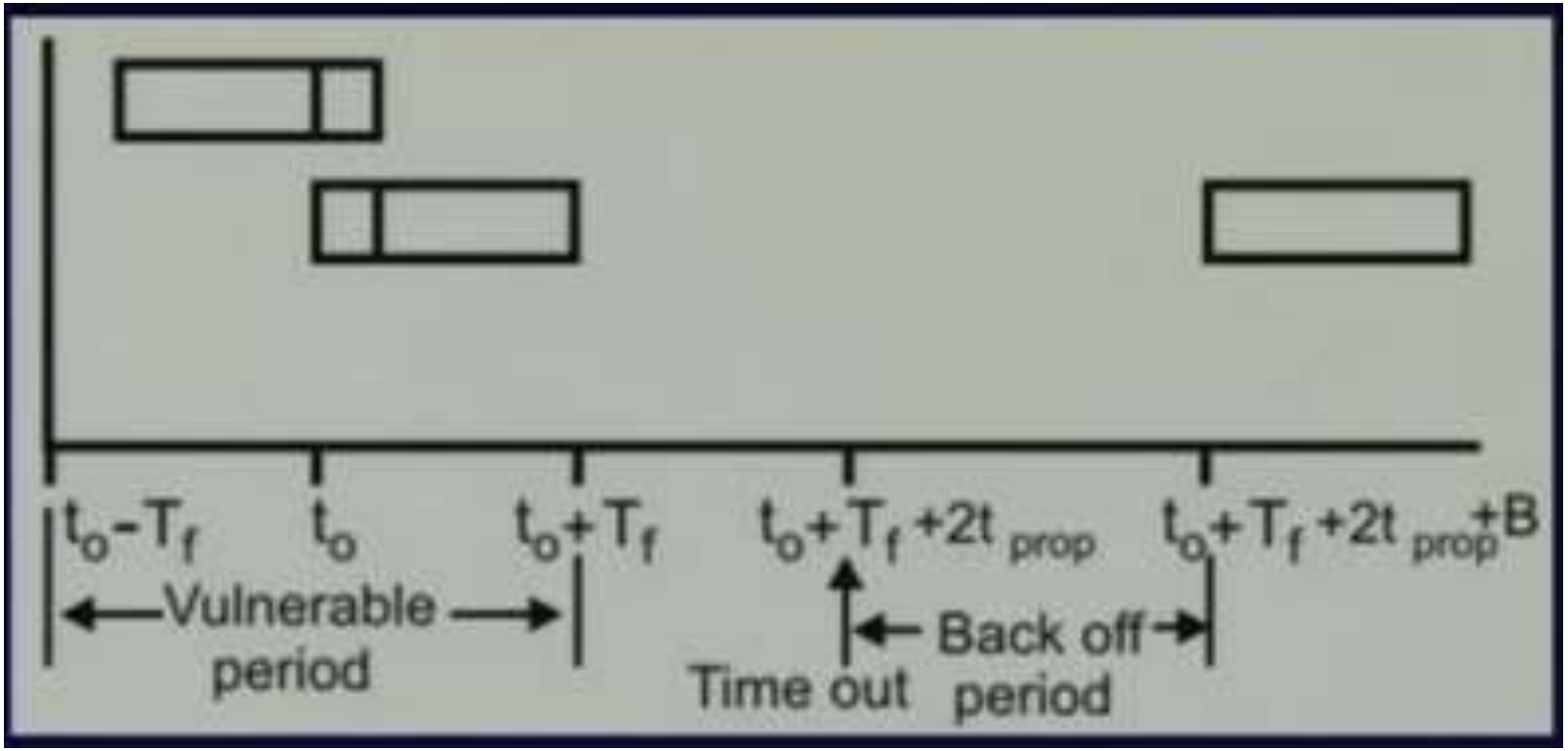
Slotted ALOHA

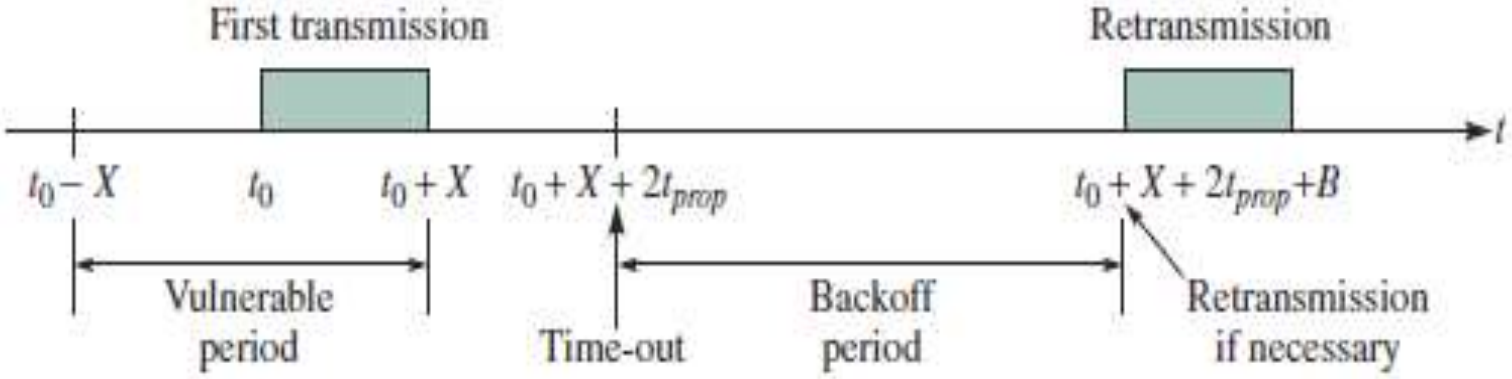
Performance of ALOHA

Vulnerable period



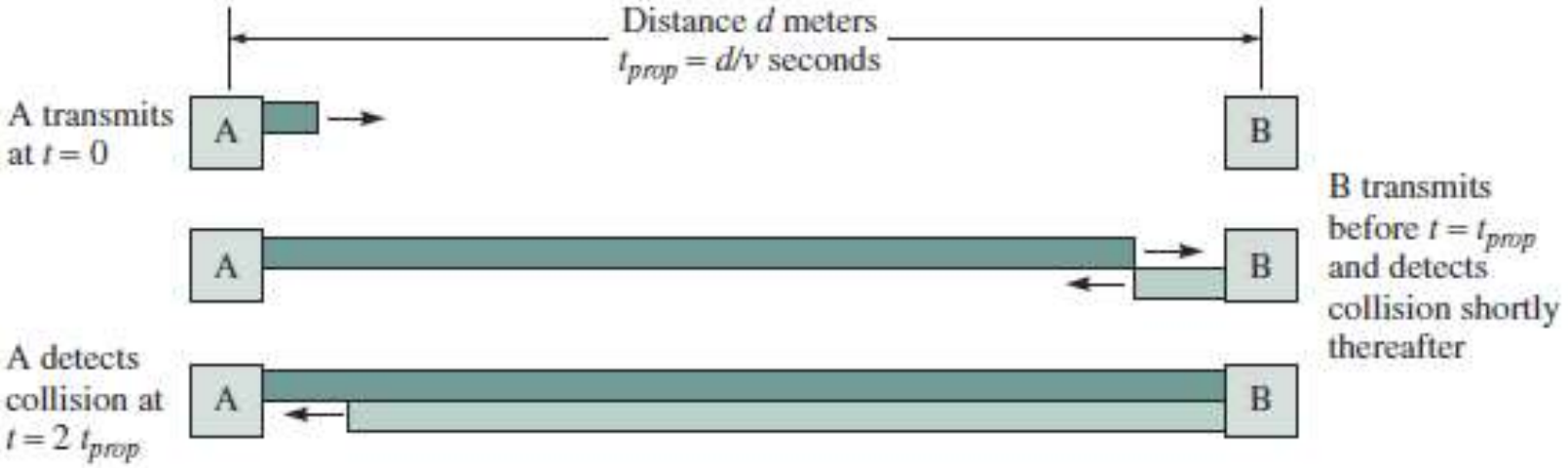
ALOHA


$t_o - T_f$
t_o
$t_o + T_f$
$t_o + T_f + 2t_{prop}$
$t_o + T_f + 2t_{prop} + B$
Vulnerable period
Time out
Back off period


First transmission
Retransmission
t
$t_0 - X$
t_0
$t_0 + X$
$t_0 + X + 2t_{prop}$
$t_0 + X + 2t_{prop} + B$
Vulnerable period
Time-out
Backoff period
Retransmission if necessary

Maximum Propagation Delay

- **Maximum propagation delay**(t_{prop}): time it takes for a bit of a frame to travel between the **two most widely** separated stations.



If **'X'** is the Frame time (i.e sending station takes X Sec to transmit a frame)

'L' is the No. of bits in a frame

'R' is the Rae of transmission of sending station in bits/Sec

Then , X = L/R sec

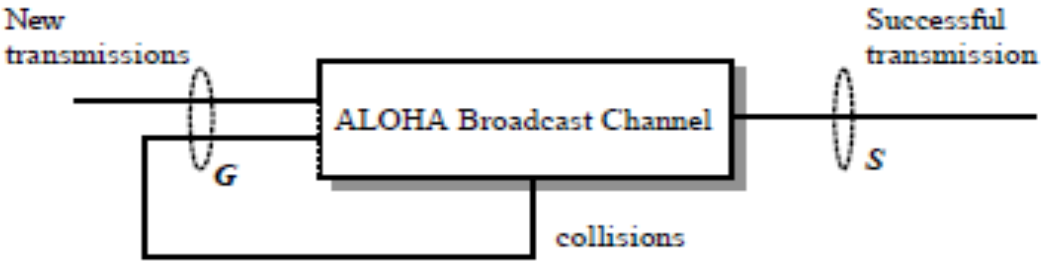
Pure ALOHA

- The outcome of the transmission is obtained after the reaction time i.e. $2t_{prop}$
- The probability of successful transmission is when there is no transmission during the vulnerable time t_0-X to t_0+X
- S= arrival rate of new packets in the system in the unit of frames/X seconds, hence it represents throughput of the system
- G= total arrival rate(new frames+retransmited frames) in unit of frames/ X seconds

Throughput of ALOHA

- Relation between throughput and offered load:

$$S = G * Prob[frame\ suffers\ no\ collision]$$



Throughput: Actual rate at which information is sent over the channel .It is measured in bits/Sec or Frames /sec

Pure ALOHA

- Assuming that the aggregate arrival process that results from new packets and retransmitted packets has a Poisson distribution with an average number of arrivals of 2G arrivals/2X seconds

$$P(\text{ k events in interval}) = \frac{\lambda^k e^{-\lambda}}{k!}$$

where λ is the average number of events per interval

$$P[k \text{ transmissions in } 2X \text{ seconds}] = \frac{(2G)^k}{k!} e^{-2G}, k = 0, 1, 2, \ldots$$

Pure ALOHA

The throughput S is equal to the total arrival rate G times the probability of a successful transmission, that is:

$$S = GP[\text{no collision}] = GP[0 \text{ transmissions in } 2X \text{ seconds}]$$

$$= G\frac{(2G)^0}{0!}e^{-2G}$$

$$= Ge^{-2G}$$

Results: Maximum Achievable Throughput

- Take derivative and set $\frac{\partial S}{\partial G} = 0$
- Maximum is attained at $G = 0.5$
- We obtain: $s_{\max} = 0.5 \times e^{-1} = \frac{1}{2e} = 0.184$
- Note: That is 18% of channel capacity!

Note

The throughput (S) for pure ALOHA is

$$S = G \times e^{-2G} .$$

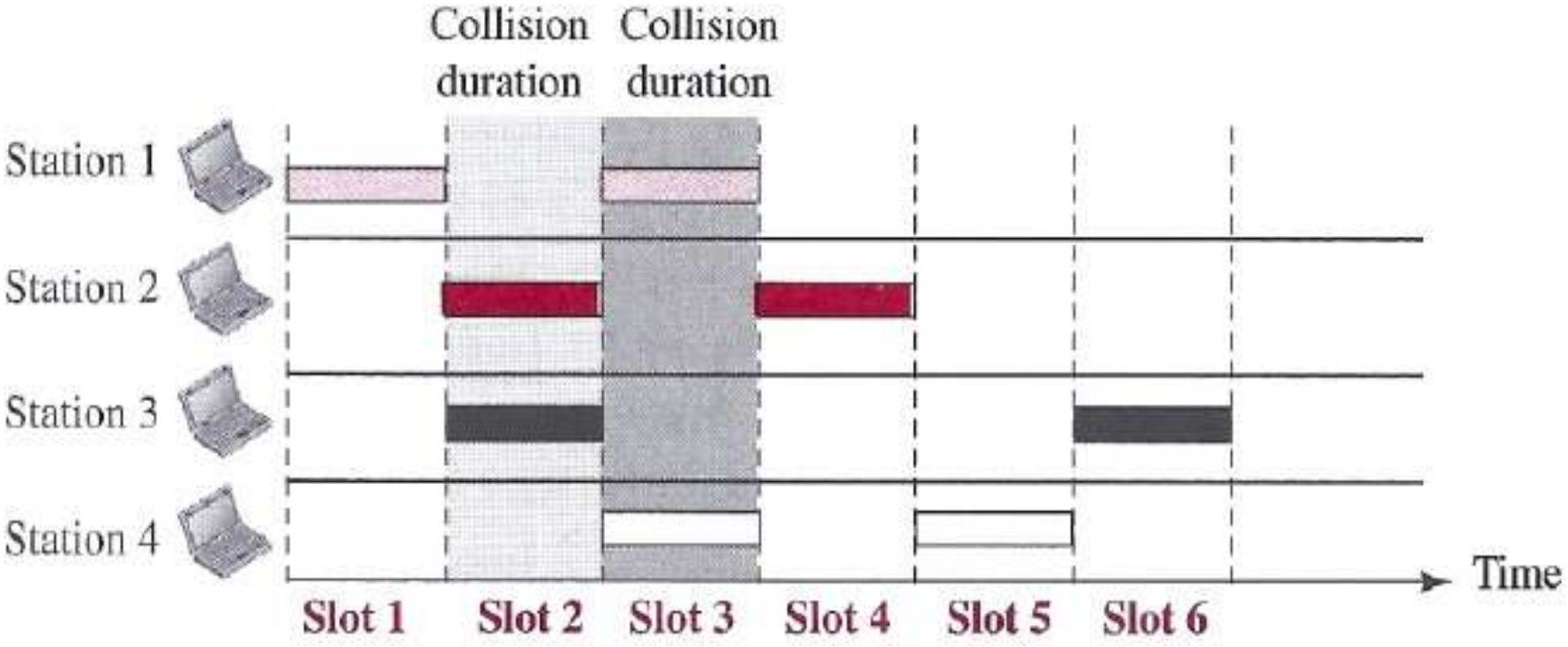
The maximum throughput

$$S_{max} = 0.184 \text{ when } G = (1/2).$$

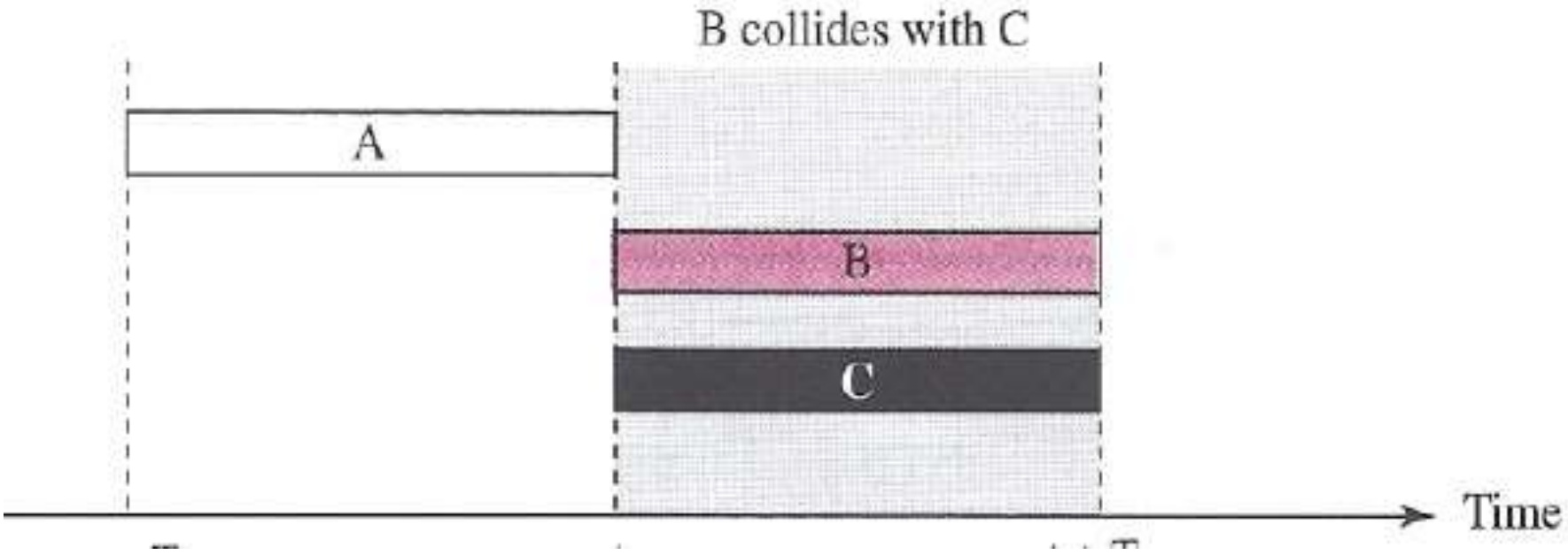
G = Average number of frames generated by the system (all stations) during one frame transmission time

Slotted ALOHA

- Transmits frame in fixed time slots
- Vulnerable period reduces to T_f from $2T_f$ of pure ALOHA



Vulnerable period



Performance of Slotted ALOHA

- Derivation is analogous to (pure) ALOHA:

$$S = G * Prob[frame\ suffers\ no\ collision]$$

- Prob[frame suffers no collision]
  = Prob[no other frame is generated during a vulnerable period]
  = Prob[no frame is generated during **1 frame period**] =

$$\frac{(1G)^0}{0!} \times e^{-1G} = e^{-1G}$$

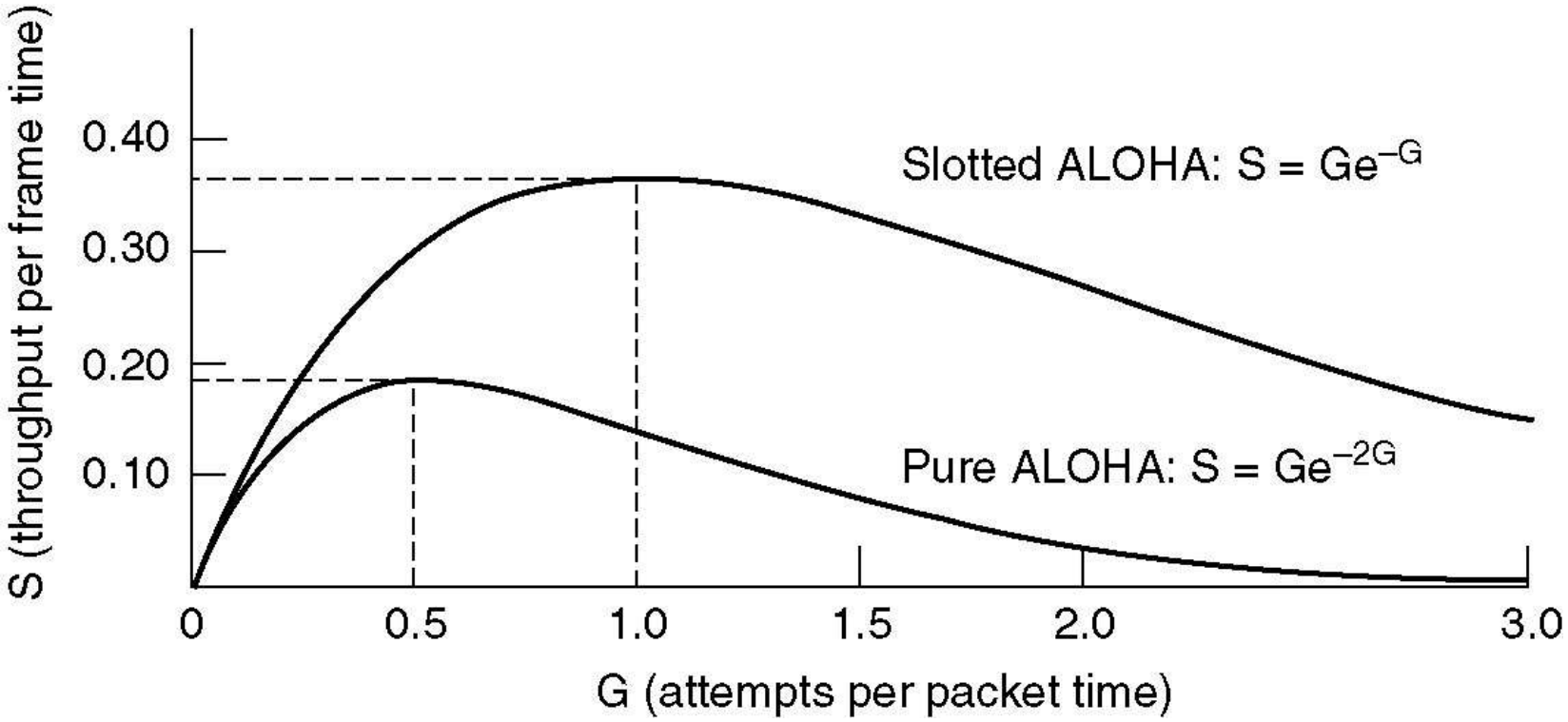
- **Total throughput in Slotted ALOHA:** $S = G \times e^{-G}$
- **Achievable Throughput:** $S_{max} = e^{-1} = \frac{1}{e} = 0.37$

Note

The throughput for slotted ALOHA is $S = G \times e^{-G}$.

The maximum throughput $S_{max} = 0.368$ when G = 1.

Efficiency



- Throughput versus offered traffic for ALOHA systems.

Pure ALOHA vulnerable time = $2 \times T_{fr}$

Example 12.2

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the requirement to make this frame collision-free?

Solution

Average frame transmission time T_{fr} is 200 bits/200 kbps or 1 ms. The vulnerable time is 2×1 ms = 2 ms. This means no station should send later than 1 ms before this station starts transmission and no station should start sending during the period (1 ms) that this station is sending.

Throughput

Let us call G the average number of frames generated by the system during one frame transmission time. Then it can be proven that the average number of successfully transmitted frames for pure ALOHA is $S = G \times e^{-2G}$. The maximum throughput S_{max} is 0.184, for $G = 1/2$. (We can find it by setting the derivative of S with respect to G to 0; see Exercises.) In other words, if one-half a frame is generated during one frame transmission time (one frame during two frame transmission times), then 18.4 percent of these frames reach their destination successfully. We expect $G = 1/2$ to produce the maximum throughput because the vulnerable time is 2 times the frame transmission time. Therefore, if a station generates only one frame in this vulnerable time (and no other stations generate a frame during this time), the frame will reach its destination successfully.

The throughput for pure ALOHA is $S = G \times e^{-2G}$.
The maximum throughput $S_{max} = 1/(2e) = 0.184$ when $G = (1/2)$.

Example 12.3

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the throughput if the system (all stations together) produces

a. 1000 frames per second?

b. 500 frames per second?

c. 250 frames per second?

Solution

The frame transmission time is 200/200 kbps or 1 ms.

a. If the system creates 1000 frames per second, or 1 frame per millisecond, then $G = 1$. In this case $S = G \times e^{-2G} = 0.135$ (13.5 percent). This means that the throughput is $1000 \times 0.135 = 135$ frames. Only 135 frames out of 1000 will probably survive.

b. If the system creates 500 frames per second, or 1/2 frames per millisecond, then $G = 1/2$. In this case $S = G \times e^{-2G} = 0.184$ (18.4 percent). This means that the throughput is $500 \times 0.184 = 92$ and that only 92 frames out of 500 will probably survive. Note that this is the *maximum* throughput case, percentagewise.

Slotted ALOHA vulnerable time = T_{fr}

Throughput

It can be proven that the average number of successful transmissions for slotted ALOHA is $S = G \times e^{-G}$. The maximum throughput S_{max} is 0.368, when $G = 1$. In other words, if one frame is generated during one frame transmission time, then 36.8 percent of these frames reach their destination successfully. We expect $G = 1$ to produce maximum throughput because the vulnerable time is equal to the frame transmission time. Therefore, if a station generates only one frame in this vulnerable time (and no other station generates a frame during this time), the frame will reach its destination successfully.

The throughput for slotted ALOHA is $S = G \times e^{-G}$.
The maximum throughput $S_{max} = 0.368$ when $G = 1$.

Example 12.4

A slotted ALOHA network transmits 200-bit frames using a shared channel with a 200-kbps bandwidth. Find the throughput if the system (all stations together) produces

a. 1000 frames per second.

b. 500 frames per second.

c. 250 frames per second.

Solution

This situation is similar to the previous exercise except that the network is using slotted ALOHA instead of pure ALOHA. The frame transmission time is 200/200 kbps or 1 ms.

a. In this case G is 1. So $S = G \times e^{-G} = 0.368$ (36.8 percent). This means that the throughput is $1000 \times 0.0368 = 368$ frames. Only 368 out of 1000 frames will probably survive. Note that this is the maximum throughput case, percentagewise.

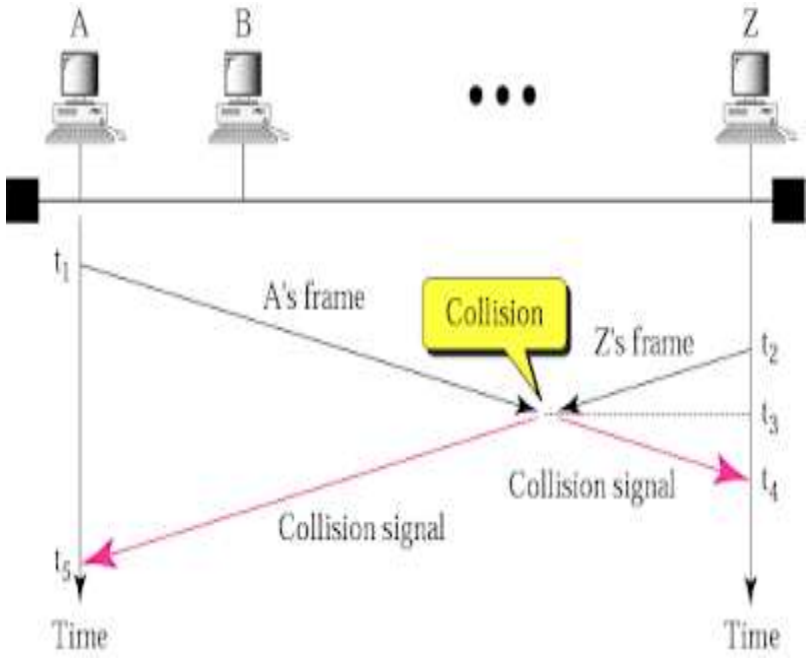
b. Here G is 1/2. In this case $S = G \times e^{-G} = 0.303$ (30.3 percent). This means that the throughput is $500 \times 0.0303 = 151$. Only 151 frames out of 500 will probably survive.

c. Now G is 1/4. In this case $S = G \times e^{-G} = 0.195$ (19.5 percent). This means that the throughput is $250 \times 0.195 = 49$. Only 49 frames out of 250 will probably survive.

CSMA :- [CARRIER SENSE MULTIPLE ACCESS] :-

- To minimize collision and to increase the performance, the CSMA method was developed.
- The chance of collision can be reduced; if station senses the medium, before trying to use it.
- CSMA requires; that each station first checks the medium, before sending.
- CSMA is based on the principle “ Sense before transmission “ or “listen before talk “.
- CSMA can reduce the possibility of collision; but it cannot eliminate.

Collision in CSMA



- At $\mathbf{t_1}$, " A " senses, the medium to be idle. So it sends frame.
- At t_2, (t2>t1) "Z" senses the medium to be idle , as propagation from station A has not reached station Z and so it sends a frame.
- The two signals collide at time t3(>t2>t1).
- The garbled signal reaches at stn Z at time t4 and at stn A at time t5.
- Hence loss of frame. " ***Collision exists because of Propagation Delay*** ". i.e. it takes some time to reach from one station to another.

Persistence Method :-

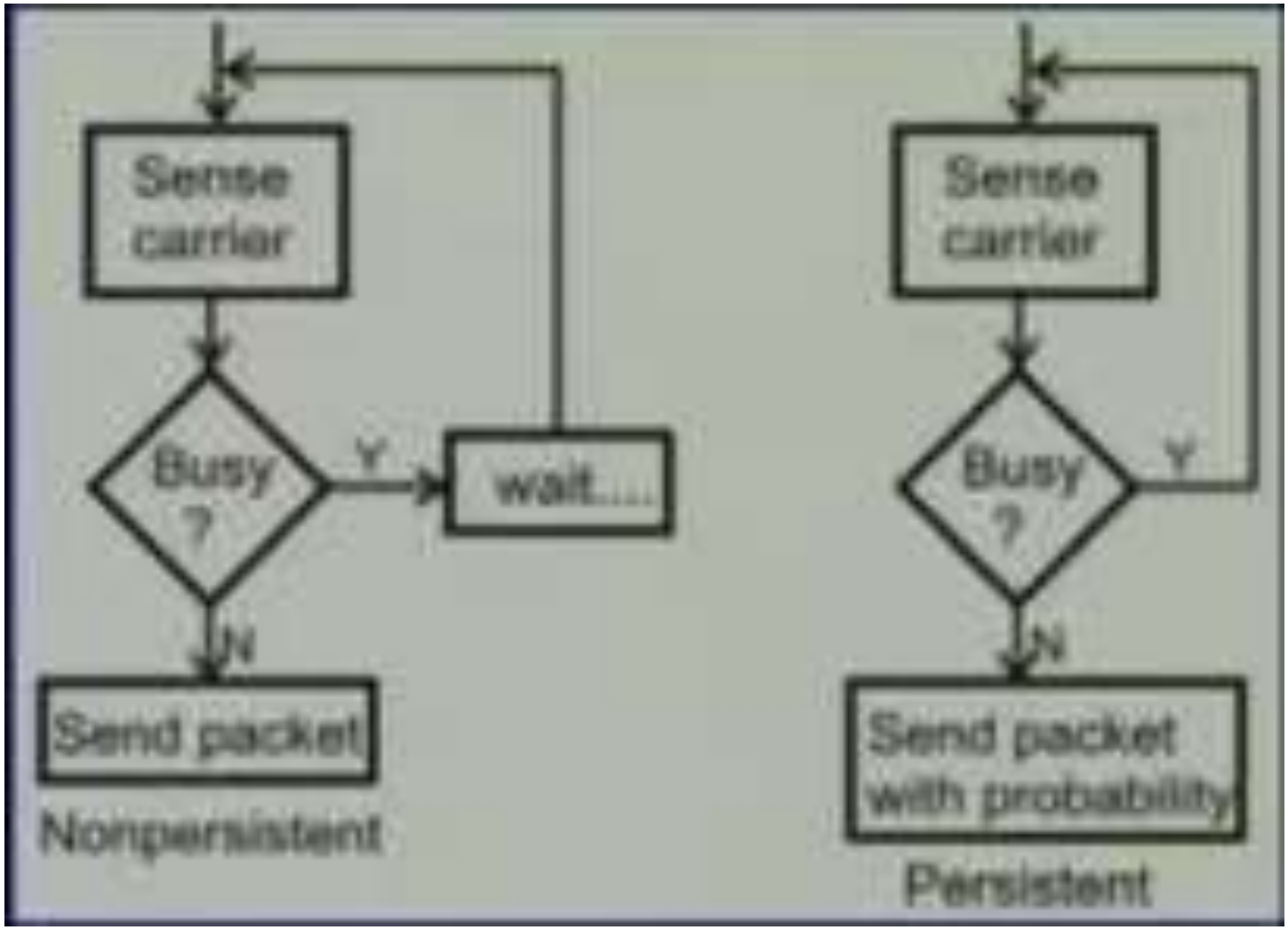
Persistence strategy defines the procedures for a station that senses a busy medium .

(*If the channel is busy; what station should do ?*
If the channel is idle; what station should do ?)

Three methods are derived.

1. 1-Persistence method.
2. Non – Persistence method
3. p – Persistence method.

Persistent and Non-persistent CSMA



Behaviour of Persistence methods; when channel is free.

1-Persistent Method **:**

It is simple and straight forward.

In this method; after the station finds the line idle, it sends frame immediately (with probability one).

It has the highest chance of collision – as two or more stations may find the line idle and will send their frames immediately.

***Non-Persistent Method* :-**

A station that has a frame to send, senses the line.
If the line is idle; it sends immediately.
If line is not idle, it waits for random amount of time and then senses the line again.
This approach reduces the chance of collision; as two or more stations will not wait for same amount of time and retry to send immediately.

Drawback :-

it reduces the efficiency; as the medium remains idle; when there may be many stations with frames to send.

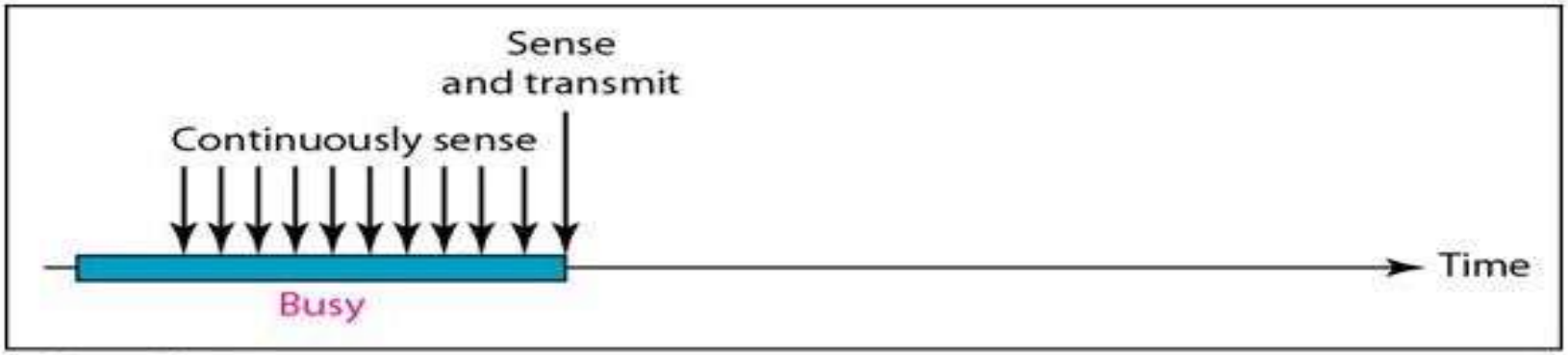
P-Persistent *Method :-*

- If the station finds the line idle , the station may or may not send.
- It sends with probability P and refrains from sending with probability 1-P.
- For ex. If P=0.2

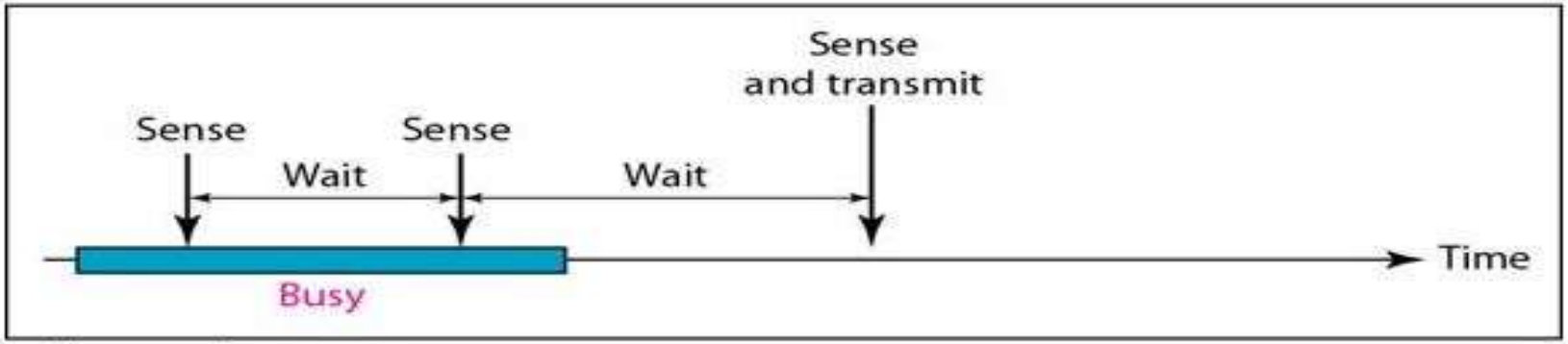
The station generates a random number between 1 - 100.If random number is between 1-20 ,the station will transmit else will refrain from sending.

- ➢ It is used if the channel has time slots; with the slot duration equal to or greater than the maximum propagation time.
- ➢ This approach combines the advantage of both.

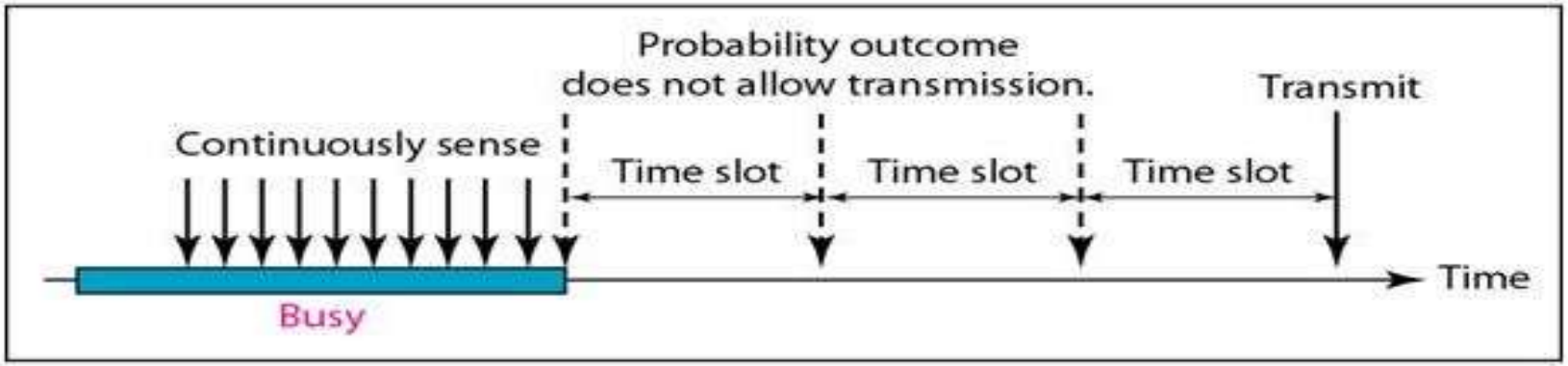
It reduces the chance of collision and increases efficiency.



a. 1-persistent

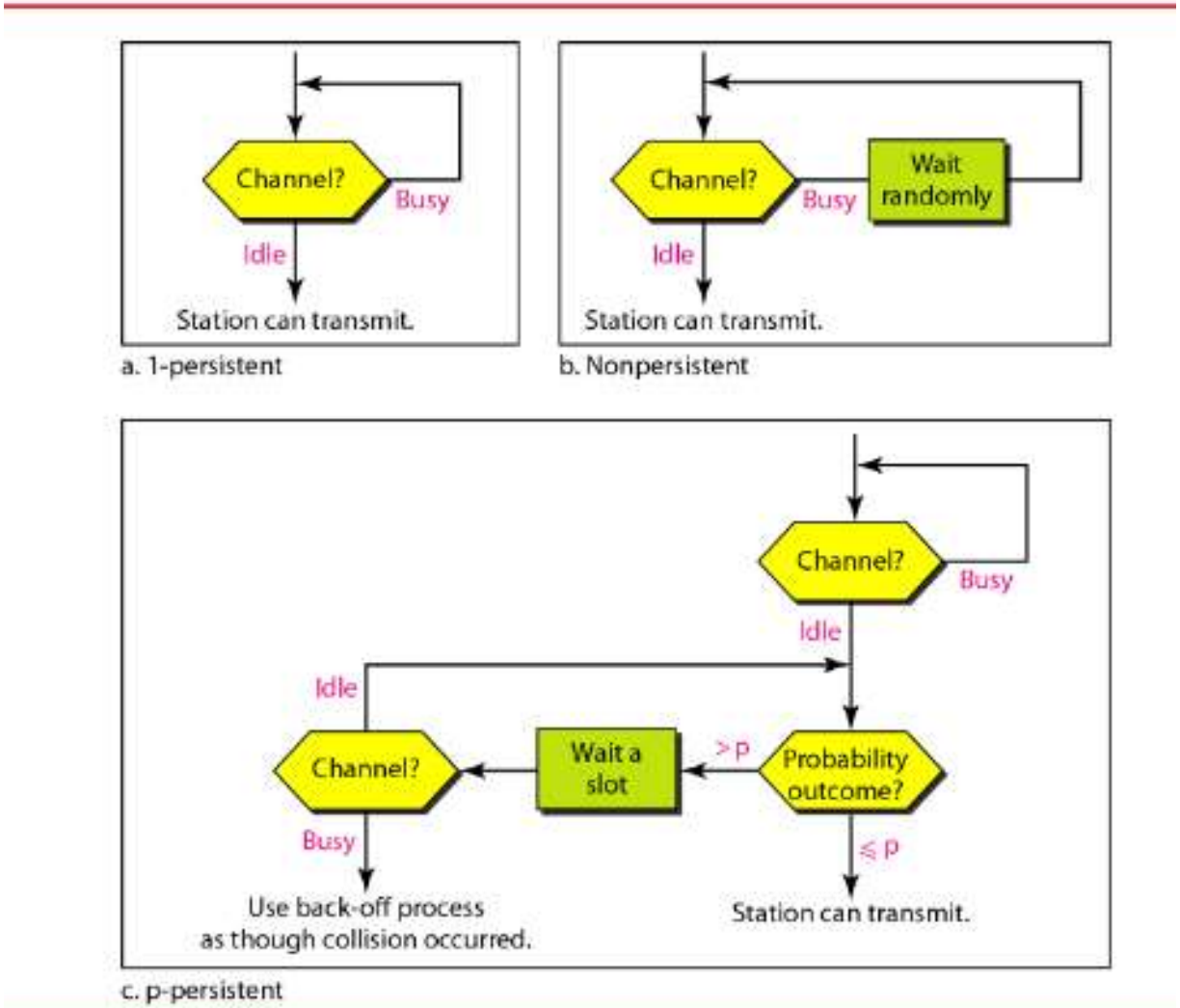


b. Nonpersistent



c. p-persistent

Flow diagram for three persistence methods

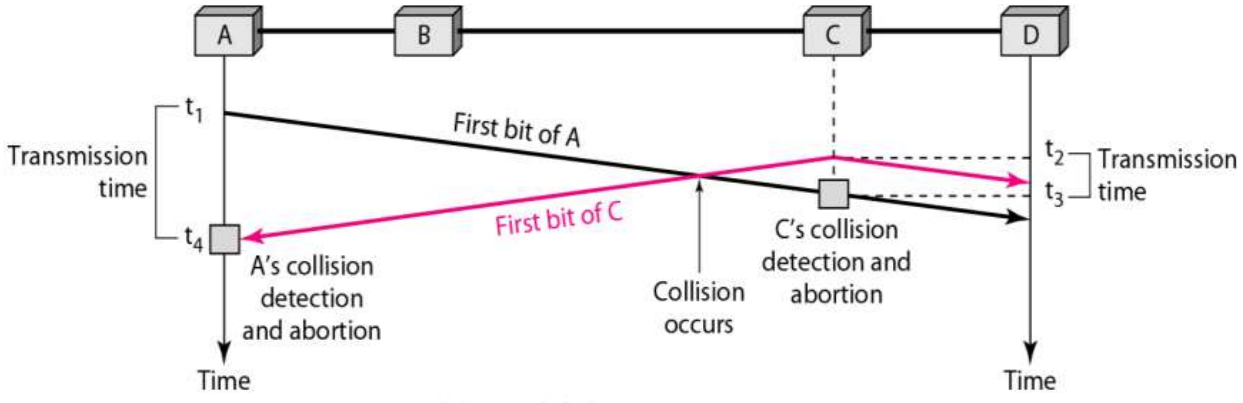


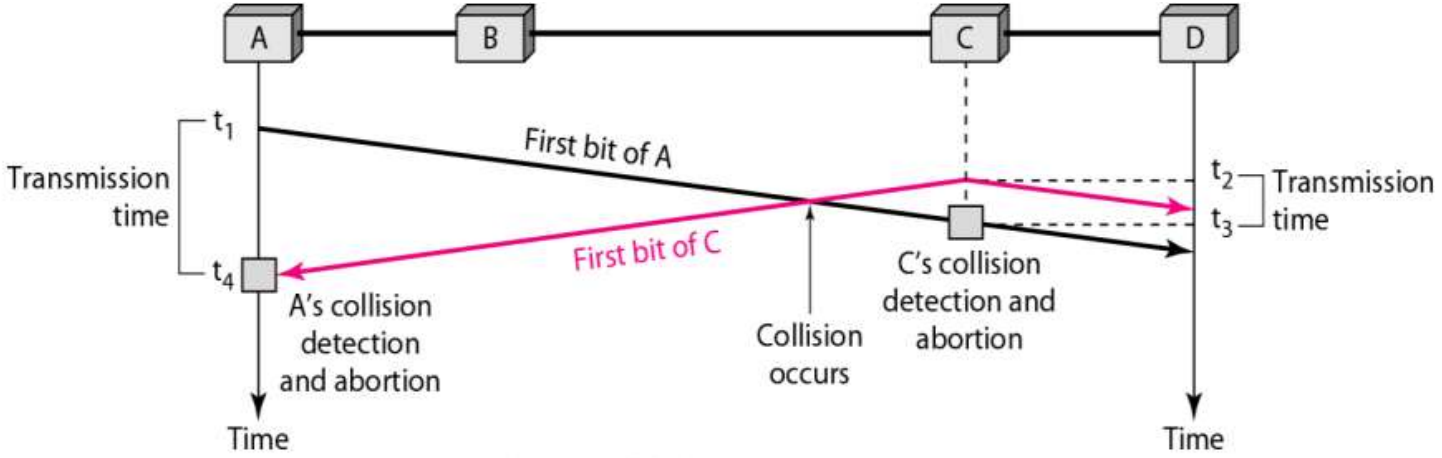
CSMA/CD

- The CSMA method does not specify the procedure following a collision.
- CSMA/CD (Carrier Sense Multiple Access/ Collision Detection) defines the algorithm to handle the collision.
- CSMA/CD (Carrier Sense Multiple Access/ Collision Detection) is a media access control method that was widely used in Early Ethernet technology/LANs when there used to be shared Bus Topology and each node (Computers) were connected By Coaxial Cables. Now a Days Ethernet is Full Duplex and Topology is either Star (connected via Switch or Router) or Point to Point (Direct Connection). Hence CSMA/CD is not used but they are still supported though.
- A station monitors the medium, after it sends a frame, to see if the transmission was successful.If so, the station is finished transmitting. If however, there is a collision, the frame is resent.

Collision of first bit in CSMA/CD

- To better understand CSMA/CD, let us look at the first bits transmitted by the two stations involved in the collision. Although each station continues to send bits in the frame until it detects the collision, we show what happens as the first bits collide. In below Figure, stations A and C are involved in the collision.





- At time t 1, station A has executed its persistence procedure and starts sending the bits of its frame. At time t2, station C has not yet sensed the first bit sent by A. Station C executes its persistence procedure and starts sending the bits in its frame, which propagate both to the left and to the right. The collision occurs sometime after time t2.Station C detects a collision at time t3 when it receives the first bit of A's frame. Station C immediately (or after a short time, but we assume immediately) aborts transmission. Station A detects collision at time t4 when it receives the first bit of C's frame; it also immediately aborts transmission. Looking at the figure, we see that A transmits for the duration t4 - tl; C transmits for the duration t3 - t2.

Minimum Frame size

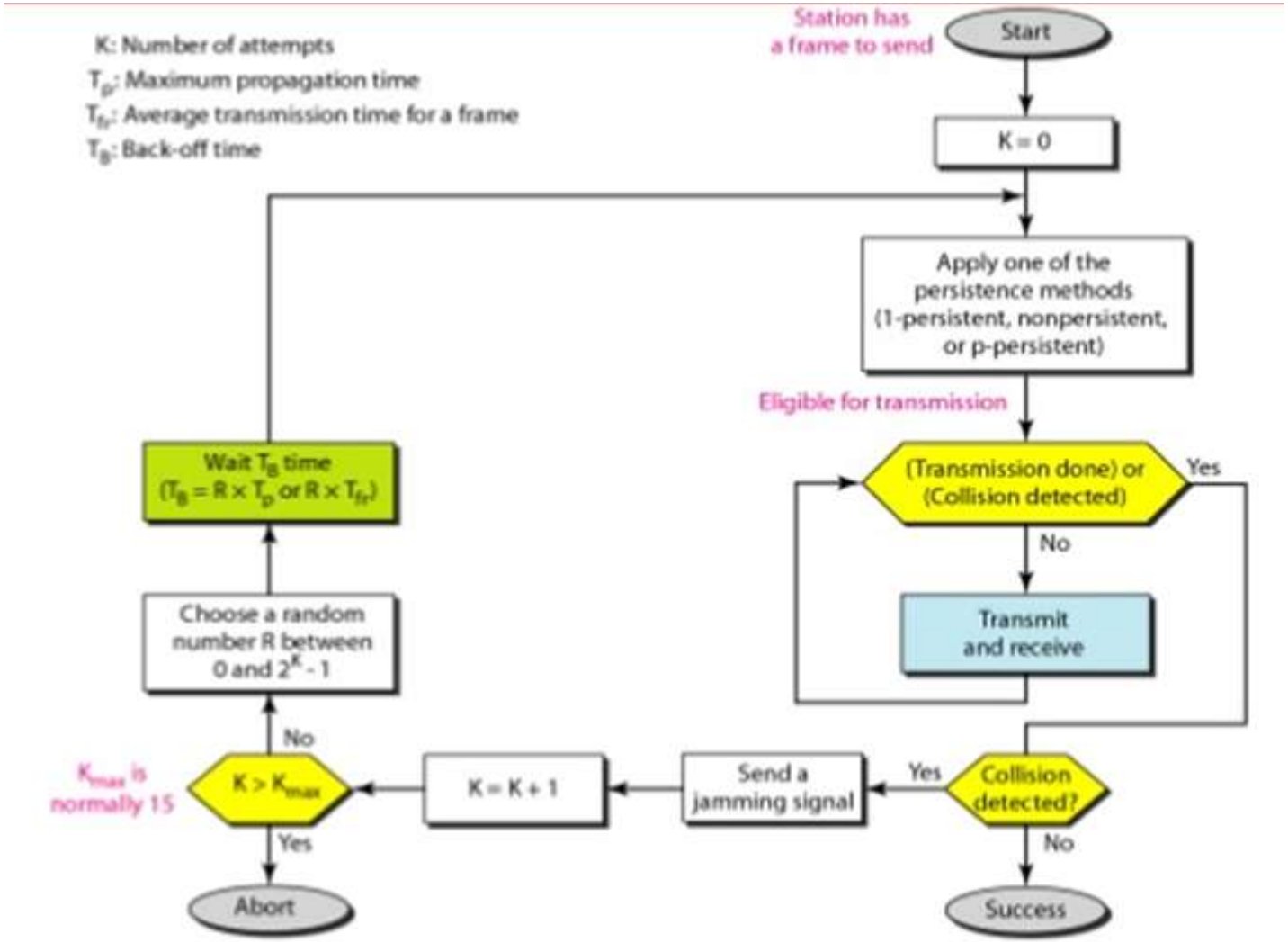
For *CSMAICD* to work, we need a restriction on the frame size. Before sending the last bit of the frame, the sending station must detect a collision, if any, and abort the transmission. This is so because the station, once the entire frame is sent, does not keep a copy of the frame and does not monitor the line for collision detection. Therefore, the frame transmission time *Tfr* must be at least two times the maximum propagation time *Tp*. To understand the reason, let us think about the worst-case scenario. If the two stations involved in a collision are the maximum distance apart, the signal from the first takes time *Tp* to reach the second, and the effect of the collision takes another time *Tp* to reach the first. So the requirement is that the first station must still be transmitting after *2Tp*.

How CSMA/CD works?

- **Step 1:** Check if the sender is ready for transmitting data packets.
- **Step 2:** Check if the transmission link is idle?
  Sender has to keep on checking if the transmission link/medium is idle. For this, it continuously senses transmissions from other nodes. Sender sends dummy data on the link. If it does not receive any collision signal, this means the link is idle at the moment. If it senses that the carrier is free and there are no collisions, it sends the data. Otherwise, it refrains from sending data.
- **Step 3:** Transmit the data & check for collisions.
  Sender transmits its data on the link. CSMA/CD does not use an 'acknowledgment' system. It checks for successful and unsuccessful transmissions through collision signals. During transmission, if a collision signal is received by the node, transmission is stopped. The station then transmits a jam signal onto the link and waits for random time intervals before it resends the frame. After some random time, it again attempts to transfer the data and repeats the above process.
- **Step 4:** If no collision was detected in propagation, the sender completes its frame transmission and resets the counters.

FLOW DIAGRAM FOR CSMA/CD :-

(jamming signal → to inform the station of collision.)



Example- CSMA/CD

A network using CSMA/CD has a bandwidth of 10 Mbps. If the maximum propagation time (including the delays in the devices and ignoring the time needed to send a jamming signal, as we see later) is 25.6 μs, what is the minimum size of the frame?

Solution

The minimum frame transmission time is $T_{fr} = 2 \times T_p = 51.2$ μs. This means, in the worst case, a station needs to transmit for a period of 51.2 μs to detect the collision. The minimum size of the frame is 10 Mbps × 51.2 μs = 512 bits or 64 bytes. This is actually the minimum size of the frame for Standard Ethernet, as we will see later in the chapter.

DIFFERENCES BETWEEN ALOHA & CSMA/CD

- The first difference is the addition of the persistence process. We need to sense the channel before we start sending the frame by using one of the persistence processes .
- The second difference is the frame transmission. In ALOHA, we first transmit the entire frame and then wait for an acknowledgment. In CSMA/CD, transmission and collision detection is a continuous process. We do not send the entire frame and then look for a collision. The station transmits and receives continuously and simultaneously
- The third difference is the sending of a short jamming signal that enforces the collision in case other stations have not yet sensed the collision.

CSMA/CA

- The basic idea behind CSMA/CD is that the station should be able to receive while transmitting to detect a collision from different stations.
- In wired networks, if a collision has occurred then the energy of received signal almost doubles and the station can sense the possibility of collision.
- In case of wireless networks, most of the energy is used for transmission and the energy of received signal increases by only 5-10% if a collision occurs. It can't be used by the station to sense collision. Therefore **CSMA/CA has been specially designed for wireless networks**.

CSMA/CA

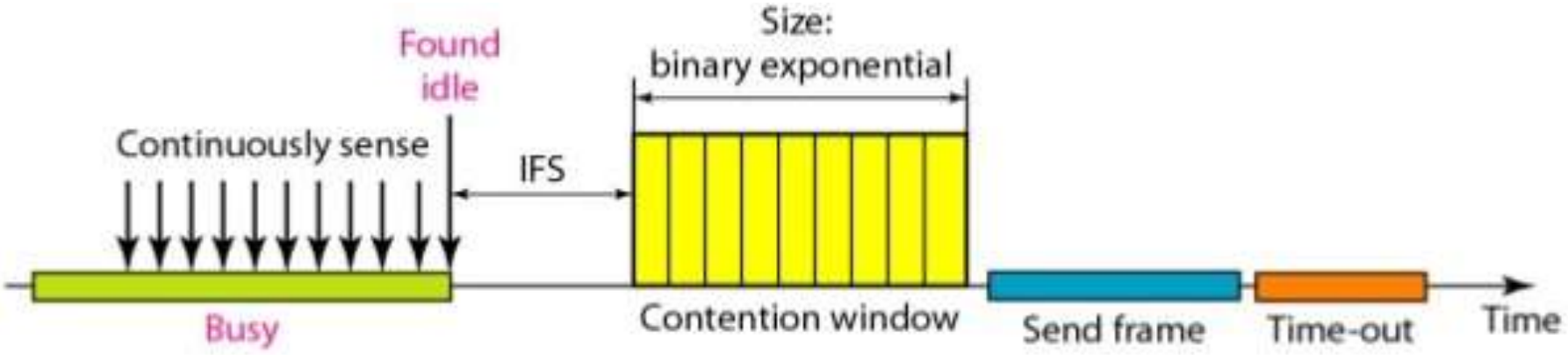
- Sender sends a short frame called Request to send RTS (20 bytes) to the destination. RTS also contains the length of the data frame
- Destination station responds with a short (14 bytes) clear to send (CTS) frame
- After receiving the CTS, the sender starts sending the data frame
- If collision occurs, CTS frame is not received within a certain period of time

CARRIER SENSE MULTIPLE ACCESS / COLLISION AVOIDANCE :

We need to avoid collision on wireless N/w's, because they cannot be detected.

Collision is avoided by the use of three strategies.

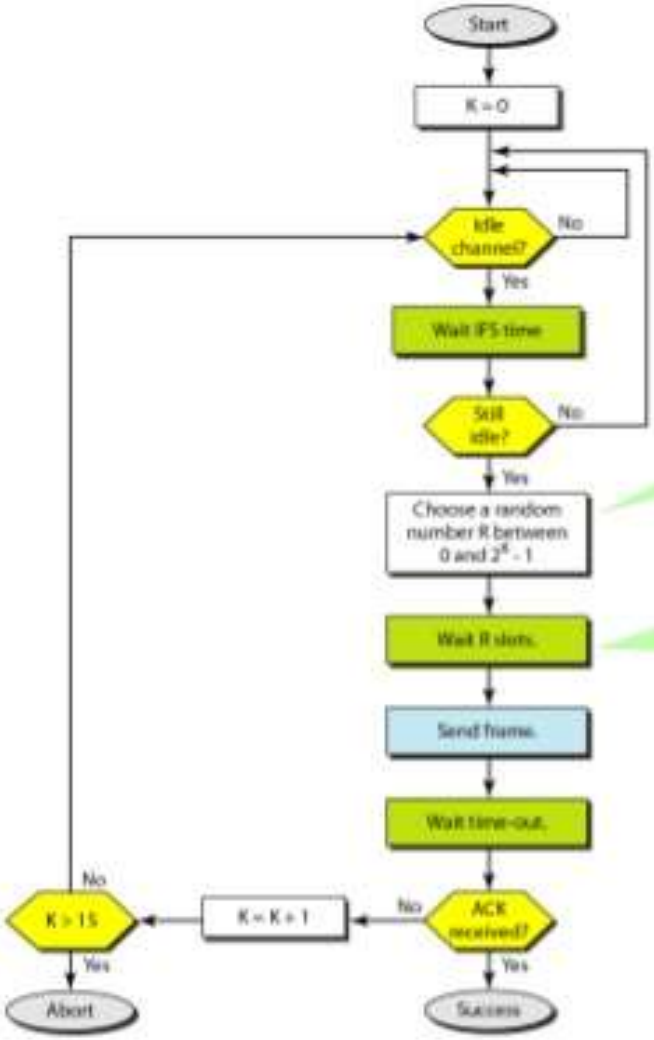
– the interframe space, -The contention window & - the acknowledgement.



CSMA\CA process:

- InterFrame Space (IFS) – When a station finds the channel busy, it waits for a period of time called IFS time. IFS can also be used to define the priority of a station or a frame. Higher the IFS lower is the priority.
- Contention Window – It is the amount of time divided into slots . A station which is ready to send frames chooses random number of slots as wait time.
- Acknowledgements – The positive acknowledgements and time-out timer can help guarantee a successful transmission of the frame.

CSMA/CA: Flow Diagram



contention window size is 2^K-1

After each slot:
- If idle, continue counting
- If busy, stop counting

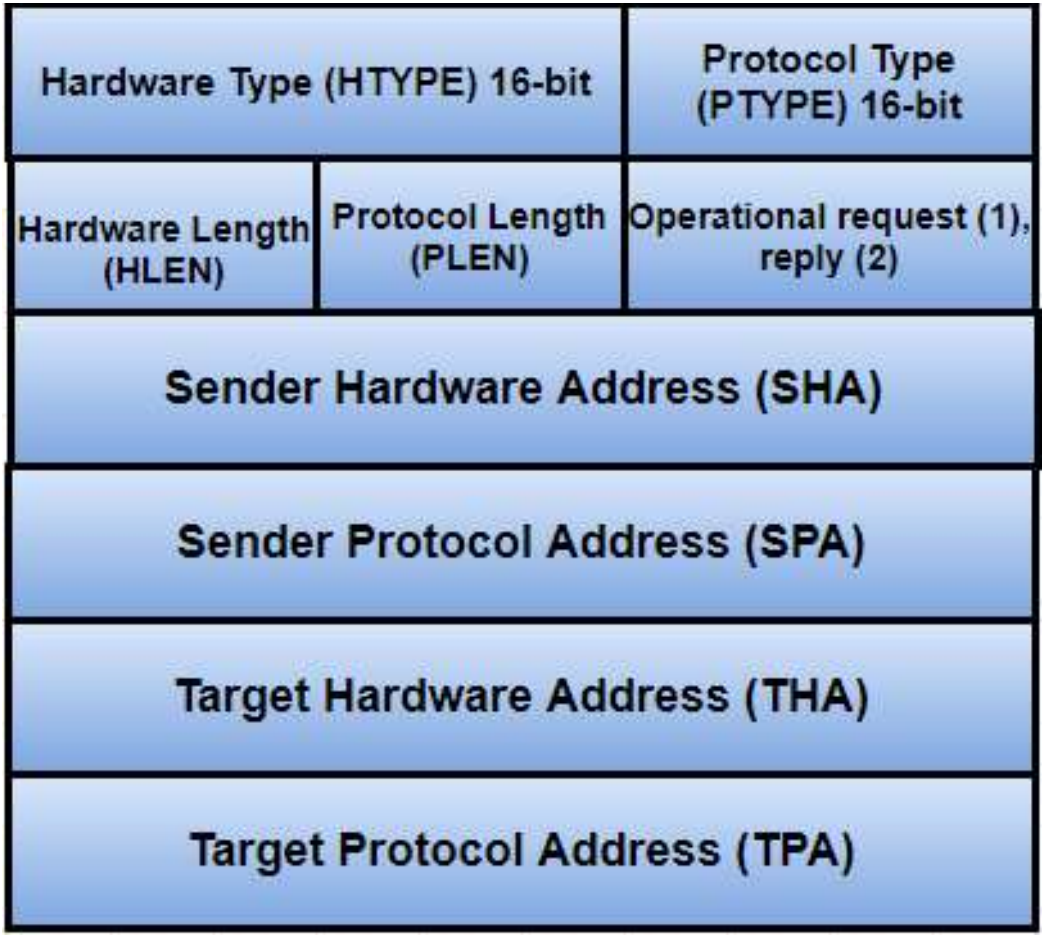
Module 3 ARP: Address Resolution Protocol

- **Address Resolution Protocol** is one of the most important protocols of the network layer in the OSI model which helps in finding the MAC(Media Access Control) address given the IP address of the system i.e. the main duty of the ARP is to convert the 32-bit IP address(for IPv4) to 48-bit address i.e. the MAC address.
- A MAC address consists of six sets of two Hexadecimal characters , each separated by a colon:

  00:1B:44:11:3A:B7 is an example of a MAC address.

(Note: Though ARP is a Network Layer protocol , it is discussed in DLL because of its functionality (finding MAC Addresses from IP))

ARP Header Format:



ARP Header:

- **Hardware Type :** Hardware Type field in the Address Resolution Protocol (ARP) Message specifies the type of hardware used for the local network transmitting the Address Resolution Protocol (ARP) message. Ethernet is the common Hardware Type and he value for Ethernet is 1. The size of this field is 2 bytes.

- **Protocol Type :** Each protocol is assigned a number used in this field. IPv4 0800 in Hexadecimals.
- **Hardware Address Length :** Hardware Address Length in the Address Resolution Protocol (ARP) Message is length in bytes of a hardware (MAC) address. Ethernet MAC addresses are 6 bytes long.

- **Protocol Address Length :** Length in bytes of a logical address (IPv4 Address). IPv4 addresses are 4 bytes long.
- **Opcode :** Opcode field/operational code in the Address Resolution Protocol (ARP) Message specifies the nature of the ARP message. **1 for ARP request** and **2 for ARP reply**.

- **Sender Hardware Address :** Layer 2 address (MAC Address) of the device sending the message.

ARP Header:

- **Sender Protocol Address :** The protocol address (IP address) of the device sending the message

- **Target Hardware Address :** Layer 2 (MAC Address) of the intended receiver. This field is ignored in requests.

- **Target Protocol Address :** The protocol address /IP Address of the intended receiver.

How ARP Works ?

- At the network layer when the source wants to find out the MAC address of the destination device it first looks for the MAC address(Physical Address) in the ARP cache or ARP table. If present there then it will use the MAC address from there for communication.

- If the MAC address is not present in the ARP table then the source device will generate an ARP Request message. In the request message the source puts its own MAC address, its IP address, destination IP address and the destination MAC address is left blank since the source is trying to find this.

….How ARP Works ?

- The source device will broadcast the ARP request message to the local network.
- The broadcast message is received by all the other devices in the LAN network. Now each device will compare the IP address of the destination with its own IP address. If the IP address of destination matches with the device's IP address then the device will send an ARP Reply message. If the IP addresses do not match then the device will simply drop the packet.
- The device whose IP address has matched with the destination IP address in the packet will reply and send the ARP Reply message. This ARP Reply message contains the MAC address of this device. The destination device updates its ARP table and stores the MAC address of the source as it will need to contact the source soon. Now, the source becomes destination(target) for this device and the ARP Reply message is sent.

….How ARP Works ?

- The **ARP reply** message is unicast and it is not broadcasted because the source which is sending the ARP reply to the destination knows the MAC address of the source device.

- When the source receives the **ARP reply** it comes to know about the destination MAC address and it also updates its ARP cache. Now the packets can be sent as the source knows destination MAC address.